

DSP Interface

for S32R41

Rev.0.8.10/16.12.2024

© 2024 NXP Semiconductors

1 Module Index	1
1.1 Software Modules	1
2 Data Structure Index	3
2.1 Data Structures	3
3 Module Documentation	5
3.1 DSP	5
3.1.1 Detailed Description	5
3.2 DSP User Guide	5
3.3 DSP Dispatcher API	5
3.3.1 Detailed Description	6
3.4 DSP Dispatcher Constants	6
3.4.1 Detailed Description	6
3.4.2 Macro Definition Documentation	6
3.4.2.1 RSDK_DSP_SPT_EVT0_INTEN_BIT	6
3.4.2.2 RSDK_DSP_SPT_EVT10_INTEN_BIT	6
3.4.2.3 RSDK_DSP_SPT_EVT11_INTEN_BIT	7
3.4.2.4 RSDK_DSP_SPT_EVT12_INTEN_BIT	7
3.4.2.5 RSDK_DSP_SPT_EVT1_INTEN_BIT	7
3.4.2.6 RSDK_DSP_SPT_EVT2_INTEN_BIT	7
3.4.2.7 RSDK_DSP_SPT_EVT3_INTEN_BIT	7
3.4.2.8 RSDK_DSP_SPT_EVT4_INTEN_BIT	7
3.4.2.9 RSDK_DSP_SPT_EVT5_INTEN_BIT	7
3.4.2.10 RSDK_DSP_SPT_EVT6_INTEN_BIT	7
3.4.2.11 RSDK_DSP_SPT_EVT7_INTEN_BIT	8
3.4.2.12 RSDK_DSP_SPT_EVT8_INTEN_BIT	8
3.4.2.13 RSDK_DSP_SPT_EVT9_INTEN_BIT	8
3.5 DSP Dispatcher Data Types	8
3.5.1 Detailed Description	8
3.5.2 Typedef Documentation	9
3.5.2.1 rsdkDspFuncPtr_t	9
3.5.2.2 rsdkDsplsCb_t	9
3.6 DSP Dispatcher Functions	9
3.6.1 Detailed Description	9
3.6.2 Function Documentation	10
3.6.2.1 RsdKdSpDispatcherInit()	10
3.6.2.2 RsdKdSpDispatcherRun()	10
3.7 DSP Radar Baseband Algo Library API	11
3.7.1 Detailed Description	11
3.8 DSP Radar Algo Functions	11
3.8.1 Detailed Description	12
3.8.2 Function Documentation	12

3.8.2.1 RsdkBbe32CaCfar()	12
3.8.2.2 RsdkBbe32CmprSptInit()	13
3.8.2.3 RsdkBbe32DecTxps()	13
3.8.2.4 RsdkBbe32Doa128()	14
3.8.2.5 RsdkBbe32Doa256()	15
3.8.2.6 RsdkBbe32EncCp4d()	15
3.8.2.7 RsdkBbe32EncCp4dFmta()	16
3.8.2.8 RsdkBbe32EncCp4dFmtb()	16
3.8.2.9 RsdkBbe32EncFlxSpt34()	17
3.8.2.10 RsdkBbe32EncSeqFixOnebuf()	17
3.8.2.11 RsdkBbe32EncSeqFixTwobuf4()	17
3.8.2.12 RsdkBbe32EncSeqFixTwobuf8()	18
3.8.2.13 RsdkBbe32EncSeqFlxOnebuf()	18
3.8.2.14 RsdkBbe32EncSeqFlxTwobuf4()	19
3.8.2.15 RsdkBbe32EncSeqFlxTwobuf8()	19
3.8.2.16 RsdkBbe32EncTxps()	20
3.8.2.17 RsdkBbe32Fft2Real()	20
3.8.2.18 RsdkBbe32FftCplx()	21
3.8.2.19 RsdkBbe32FftInit()	21
3.8.2.20 RsdkBbe32Hist()	22
3.8.2.21 RsdkBbe32OsCfar()	22
3.8.2.22 RsdkBbe32ReordTile16to4Ch()	23
3.8.2.23 RsdkBbe32Tdlm()	23
3.9 DSP Radar Algo Constants	24
3.9.1 Detailed Description	24
3.9.2 Macro Definition Documentation	24
3.9.2.1 BBE_SIMD_WIDTH [1/2]	24
3.9.2.2 BBE_SIMD_WIDTH [2/2]	24
3.10 DSP Radar Algo Data Types	24
3.10.1 Detailed Description	25
3.11 DSP Host Driver API	25
3.11.1 Detailed Description	26
3.12 DSP Host Driver Constants	26
3.12.1 Detailed Description	26
3.12.2 Macro Definition Documentation	26
3.12.2.1 DSPHD_SIZEOF_JOB_LIST	26
3.12.2.2 DSPHD_SIZEOF_MSG	26
3.13 DSP Host Driver Data Types	26
3.13.1 Detailed Description	27
3.13.2 Enumeration Type Documentation	27
3.13.2.1 Dsphd_AckStatusType	27
3.13.2.2 Dsphd_DispatcherStateType	28

3.13.2.3 Dsphd_fpuRoundConfigType	28
3.13.2.4 Dsphd_MsgTypeType	28
3.13.2.5 Dsphd_p1TaskConfigType	29
3.13.2.6 Dsphd_SignalsType	30
3.14 DSP Host Driver Functions	30
3.14.1 Detailed Description	30
3.14.2 Function Documentation	31
3.14.2.1 Dsphd_CreateJobList()	31
3.14.2.2 Dsphd_Init()	31
3.14.2.3 Dsphd_SendMsg()	32
3.15 DSP LAL API	32
3.15.1 Detailed Description	32
3.16 DSP LAL Demo Data	33
3.16.1 Detailed Description	33
3.17 DSP LAL Constants	33
3.17.1 Detailed Description	34
3.17.2 Macro Definition Documentation	34
3.17.2.1 BBE32_ALWAYS_INLINE	34
3.17.2.2 BBE32_NO_INLINE	34
3.17.2.3 EXP_MASK	34
3.17.2.4 INFINITY_SP	34
3.17.2.5 MANTISSA_MASK	34
3.17.2.6 NAN_SP	34
3.17.2.7 OPTIMIZE_0	35
3.17.2.8 OPTIMIZE_1	35
3.17.2.9 OPTIMIZE_2	35
3.17.2.10 OPTIMIZE_3	35
3.17.2.11 OPTIMIZE_S	35
3.17.2.12 RSDK_MAG2N_2XF32_V1	35
3.17.2.13 RSDK_MAG2N_2XF32_V2	36
3.17.2.14 SP_pi	36
3.17.2.15 SP_pi_over_2	36
3.18 DSP LAL Demo Constants	37
3.18.1 Detailed Description	37
3.18.2 Macro Definition Documentation	37
3.18.2.1 BBE_BATCH_WIDTH	37
3.18.2.2 SIZE_VEC	37
3.19 DSP LAL Functions	37
3.19.1 Detailed Description	37
3.20 DSP LAL Demo Functions	38
3.21 Dsp_lal_api_func_add_vec	38
3.21.1 Detailed Description	38

3.21.2 Function Documentation	38
3.21.2.1 Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8()	38
3.21.2.2 Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2()	39
3.21.2.3 Rsdk_LA_add_Vof32p8Vif32p8Vif32p8()	39
3.21.2.4 Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2()	40
3.22 Dsp_lal_api_func_sub_vec	40
3.22.1 Detailed Description	41
3.22.2 Function Documentation	41
3.22.2.1 Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8()	41
3.22.2.2 Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2()	41
3.22.2.3 Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8()	42
3.22.2.4 Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2()	42
3.23 Dsp_lal_api_func_mul_vec_mat	43
3.23.1 Detailed Description	43
3.23.2 Function Documentation	43
3.23.2.1 Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8()	44
3.23.2.2 Rsdk_LA_mul_Mof32p8Mif32p8Mif32p8()	45
3.23.2.3 Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8()	46
3.23.2.4 Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2()	46
3.23.2.5 Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8()	47
3.23.2.6 Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2()	47
3.23.2.7 Rsdk_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8()	48
3.23.2.8 Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8()	49
3.24 Dsp_lal_api_func_div_vec	50
3.24.1 Detailed Description	50
3.24.2 Function Documentation	50
3.24.2.1 Rsdk_LA_div_Vof32cp8Vif32cp8Vif32cp8()	50
3.24.2.2 Rsdk_LA_div_Vof32p8Vif32p8Vif32p8()	51
3.24.2.3 Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2()	51
3.25 Dsp_lal_api_func_xcorr_vec	52
3.25.1 Detailed Description	52
3.25.2 Function Documentation	52
3.25.2.1 Rsdk_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8()	52
3.25.2.2 Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8()	53
3.26 Dsp_lal_api_func_op_vec_mat	53
3.26.1 Detailed Description	54
3.26.2 Function Documentation	54
3.26.2.1 Rsdk_EF_exp_Vof32p8Vif32p8()	54
3.26.2.2 Rsdk_EF_mod_Vof32p8Vif32p8Vif32p8()	55
3.26.2.3 Rsdk_EF_sqrt_Vof32cp8Vif32cp8()	56
3.26.2.4 Rsdk_EF_sqrt_Vof32p8Vif32p8()	56
3.26.2.5 Rsdk_EF_sqrtAbs_Vof32p8Vif32p8()	57

3.26.2.6 Rsdk_LA_hypot_Vof32p8Vif32p8Vif32p8()	57
3.26.2.7 Rsdk_LA_rcp_Vof32cp8Vif32cp8()	58
3.26.2.8 Rsdk_LA_rcp_Vof32p8Vif32p8()	58
3.26.2.9 Rsdk_LA_thresh_Vof32p8Vif32p8Sif32()	59
3.26.2.10 Rsdk_LA_thresh_Vof32p8Vif32p8Vif32p8()	59
3.26.2.11 Rsdk_LA_threshAbs_Vof32p8Vif32cp8Sif32()	60
3.26.2.12 Rsdk_LA_threshAbs_Vof32p8Vif32cp8Vif32p8()	61
3.27 Dsp_lal_api_func_fill	61
3.27.1 Detailed Description	62
3.27.2 Function Documentation	62
3.27.2.1 Rsdk_EF_fill_Vof32cp8Sif32c()	62
3.27.2.2 Rsdk_EF_fill_Vof32cp8Sif32cp8()	63
3.27.2.3 Rsdk_EF_fill_Vof32p8Sif32()	63
3.27.2.4 Rsdk_EF_fill_Vof32p8Sif32p8()	64
3.27.2.5 Rsdk_EF_fillZeros_Mof32cp8()	64
3.27.2.6 Rsdk_EF_fillZeros_Mof32p8()	65
3.27.2.7 Rsdk_EF_fillZeros_Vof32cp8()	65
3.27.2.8 Rsdk_EF_fillZeros_Vof32p8()	66
3.27.2.9 Rsdk_EF_fillZerosDg_Mof32cp8Sif32cp8()	66
3.27.2.10 Rsdk_EF_fillZerosDg_Mof32cp8Vif32cp8()	67
3.27.2.11 Rsdk_EF_fillZerosDg_Mof32p8Sif32p8()	67
3.27.2.12 Rsdk_EF_fillZerosDg_Mof32p8Vif32p8()	68
3.27.2.13 Rsdk_EF_identity_Mof32cp8()	68
3.27.2.14 Rsdk_EF_identity_Mof32p8()	69
3.27.2.15 Rsdk_SP_fillStgRadix2_Vof32cp8()	69
3.28 Dsp_lal_api_func_trig_vec	70
3.28.1 Detailed Description	70
3.28.2 Function Documentation	70
3.28.2.1 Rsdk_EF_acos_Vof32p8Vif32p8()	71
3.28.2.2 Rsdk_EF_asin_Vof32p8Vif32p8()	71
3.28.2.3 Rsdk_EF_atan2_Vof32p8Vif32p8Vif32p8()	72
3.28.2.4 Rsdk_EF_atan_Vof32p8Vif32p8()	72
3.28.2.5 Rsdk_EF_cos_Vof32p8Vif32p8()	73
3.28.2.6 Rsdk_EF_sin_Vof32p8Vif32p8()	73
3.28.2.7 Rsdk_LA_deg2rad_Vof32p8Vif32p8()	74
3.28.2.8 Rsdk_LA_rad2deg_Vof32p8Vif32p8()	74
3.29 Dsp_lal_api_func_log_vec	75
3.29.1 Detailed Description	75
3.29.2 Function Documentation	75
3.29.2.1 Rsdk_EF_log10_Vof32p8Vif32p8()	75
3.29.2.2 Rsdk_EF_log2_Vof32p8Vif32p8()	76
3.29.2.3 Rsdk_EF_log_Vof32p8Vif32p8()	76

3.30 Dsp_lal_api_func_prod_vec	78
3.30.1 Detailed Description	78
3.30.2 Function Documentation	78
3.30.2.1 Rsdk_LA_cross_Vo3f32cp8Vi3f32cp8Vi3f32cp8()	78
3.30.2.2 Rsdk_LA_cross_Vo3f32p8Vi3f32p8Vi3f32p8()	79
3.30.2.3 Rsdk_LA_crossconj_Vo3f32cp8Vi3f32cp8Vi3f32cp8()	79
3.30.2.4 Rsdk_LA_dot_Sof32p8Vif32p8Vif32p8()	80
3.30.2.5 Rsdk_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8()	80
3.30.2.6 Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8()	81
3.31 Dsp_lal_api_func_det_mat	81
3.31.1 Detailed Description	82
3.31.2 Function Documentation	82
3.31.2.1 Rsdk_LA_detSq_Sof32cp8Mif32cp8()	82
3.31.2.2 Rsdk_LA_detSq_Sof32p8Mif32p8()	82
3.32 Dsp_lal_api_func_qr_dec_mat	83
3.32.1 Detailed Description	83
3.32.2 Function Documentation	83
3.32.2.1 Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8()	84
3.32.2.2 Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8()	85
3.33 Dsp_lal_api_func_cholesky_dec_mat	86
3.33.1 Detailed Description	86
3.33.2 Function Documentation	86
3.33.2.1 Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8()	86
3.33.2.2 Rsdk_LA_decLLCholPd_Mof32p8Mif32p8()	87
3.34 Dsp_lal_api_func_conj_vec	87
3.34.1 Detailed Description	87
3.34.2 Function Documentation	87
3.34.2.1 Rsdk_LA_conj_Vof32cp8Vif32cp8()	88
3.35 Dsp_lal_api_func_min_max_vec	88
3.35.1 Detailed Description	88
3.35.2 Function Documentation	88
3.35.2.1 Rsdk_LA_max_Sou32p8Vif32p8()	89
3.35.2.2 Rsdk_LA_maxAbs_Sou32p8Vif32cp8()	89
3.35.2.3 Rsdk_LA_min_Sou32p8Vif32p8()	90
3.35.2.4 Rsdk_LA_minAbs_Sou32p8Vif32cp8()	90
3.36 Dsp_lal_api_func_sum_vec	91
3.36.1 Detailed Description	91
3.36.2 Function Documentation	91
3.36.2.1 Rsdk_LA_sum2_Sof32cp8Vif32cp8()	92
3.36.2.2 Rsdk_LA_sum2_Sof32p8Vif32p8()	92
3.36.2.3 Rsdk_LA_sum3_Sof32cp8Vif32cp8()	93
3.36.2.4 Rsdk_LA_sum3_Sof32p8Vif32p8()	93

3.36.2.5 RsdK_LA_sum_Sof32cp8Vif32cp8()	94
3.36.2.6 RsdK_LA_sum_Sof32p8Vif32p8()	94
3.36.2.7 RsdK_LA_sumRcp_Sof32cp8Vif32cp8()	95
3.36.2.8 RsdK_LA_sumRcp_Sof32p8Vif32p8()	95
3.37 Dsp_lal_api_func_inv_up_triag_mat	96
3.37.1 Detailed Description	96
3.37.2 Function Documentation	96
3.37.2.1 RsdK_LA_invRectUNs_Mof32cp8Mif32cp8()	97
3.37.2.2 RsdK_LA_invRectUNs_Mof32p8Mif32p8()	97
3.37.2.3 RsdK_LA_invUNs_Mof32cp8Mif32cp8()	98
3.37.2.4 RsdK_LA_invUNs_Mof32p8Mif32p8()	99
3.38 Dsp_lal_api_func_inv_low_triag_mat	99
3.38.1 Detailed Description	99
3.38.2 Function Documentation	99
3.38.2.1 RsdK_LA_invLNs_Mof32cp8Mif32cp8()	100
3.38.2.2 RsdK_LA_invLNs_Mof32p8Mif32p8()	100
3.38.2.3 RsdK_LA_invRectLNs_Mof32cp8Mif32cp8()	101
3.38.2.4 RsdK_LA_invRectLNs_Mof32p8Mif32p8()	101
3.39 Dsp_lal_api_func_inv_qr_mat	102
3.39.1 Detailed Description	102
3.39.2 Function Documentation	102
3.39.2.1 RsdK_LA_invHousQRNs_Mof32cp8Mif32cp8()	103
3.39.2.2 RsdK_LA_invHousQRNs_Mof32p8Mif32p8()	103
3.40 Dsp_lal_api_func_inv_gauss_mat	104
3.40.1 Detailed Description	104
3.40.2 Function Documentation	104
3.40.2.1 RsdK_LA_invGausCPNs_Mof32cp8Miof32cp8()	104
3.40.2.2 RsdK_LA_invGausCPNs_Mof32p8Miof32p8()	105
3.40.2.3 RsdK_LA_invNsGaus_Mof32cp8Miof32cp8()	105
3.40.2.4 RsdK_LA_invNsGaus_Mof32p8Miof32p8()	106
3.41 Dsp_lal_api_func_eig_jacobi_mat	107
3.41.1 Detailed Description	107
3.41.2 Function Documentation	107
3.41.2.1 RsdK_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8()	107
3.41.2.2 RsdK_LA_eigSy_Mof32p8Mof32p8Mif32p8()	108
3.42 Dsp_lal_api_func_eig_ql_mat	108
3.42.1 Detailed Description	109
3.42.2 Function Documentation	109
3.42.2.1 RsdK_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8()	109
3.42.2.2 RsdK_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8()	110
3.42.2.3 RsdK_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8()	110
3.43 Dsp_lal_api_func_eig_QRfrancis_mat	111

3.43.1 Detailed Description	111
3.43.2 Function Documentation	111
3.43.2.1 Rsdk_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8()	111
3.43.2.2 Rsdk_LA_eigValUpHess_Vof32cp8Mif32cp8()	112
3.44 Dsp_lal_api_func_mean_vec	113
3.44.1 Detailed Description	113
3.44.2 Function Documentation	113
3.44.2.1 Rsdk_LA_avg_Sof32cp8Vif32cp8()	113
3.44.2.2 Rsdk_LA_avg_Sof32p8Vif32p8()	114
3.44.2.3 Rsdk_LA_havg_Sof32cp8Vif32cp8()	114
3.44.2.4 Rsdk_LA_havg_Sof32p8Vif32p8()	115
3.44.2.5 Rsdk_LA_rms_Sof32p8Vif32cp8()	116
3.44.2.6 Rsdk_LA_rms_Sof32p8Vif32p8()	116
3.44.2.7 Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8()	117
3.44.2.8 Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8()	117
3.45 Dsp_lal_api_func_conversions_vec_mat	118
3.45.1 Detailed Description	118
3.45.2 Function Documentation	118
3.45.2.1 Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8()	118
3.45.2.2 Rsdk_DT_getIm_Vof32p8Vif32cp8()	120
3.45.2.3 Rsdk_DT_getRe_Vof32p8Vif32cp8()	120
3.45.2.4 Rsdk_DT_interl_Mof32cp8Mif32p8Mif32p8()	121
3.46 Dsp_lal_api_func_sort_vec	122
3.46.1 Detailed Description	122
3.46.2 Function Documentation	122
3.46.2.1 Rsdk_AU_sortdOET_Viof32p8_k2()	122
3.46.2.2 Rsdk_AU_sortdOET_Vof32p8Vif32p8()	123
3.46.2.3 Rsdk_AU_sortOET_Viof32p8_k2()	123
3.46.2.4 Rsdk_AU_sortOET_Vof32p8Vif32p8()	124
3.46.2.5 Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8()	124
3.47 Dsp_lal_api_func_sp_smooth	125
3.47.1 Detailed Description	125
3.47.2 Function Documentation	125
3.47.2.1 Rsdk_ML_spatialSm_Mof32cp8Mif32cp8()	125
3.48 Dsp_lal_func_api_dist	126
3.48.1 Detailed Description	126
3.48.2 Function Documentation	126
3.48.2.1 Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32()	126
3.48.2.2 Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32()	127
3.49 Dsp_lal_api_func_prod_tot	128
3.49.1 Detailed Description	128
3.49.2 Function Documentation	128

3.49.2.1 Rsdk_LA_prod_Sof32cp8Vif32cp8()	128
3.49.2.2 Rsdk_LA_prod_Sof32p8Vif32p8()	129
3.50 Dsp_lal_api_func_hessenberg_mat	129
3.50.1 Detailed Description	129
3.50.2 Function Documentation	129
3.50.2.1 Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8()	130
3.50.2.2 Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8()	130
3.51 Dsp_lal_api_func_copy	131
3.51.1 Detailed Description	132
3.51.2 Function Documentation	132
3.51.2.1 Rsdk_DT_copy_Mof32cp8Mif32cp8()	132
3.51.2.2 Rsdk_DT_copy_Mof32p8Mif32p8()	132
3.51.2.3 Rsdk_DT_copy_Vof32cp8Vif32cp8()	133
3.51.2.4 Rsdk_DT_copy_Vof32p8Vif32p8()	134
3.51.2.5 Rsdk_DT_copyInOffset_Vof32cp8Vif32cp8()	134
3.51.2.6 Rsdk_DT_copyInOffset_Vof32p8Vif32p8()	135
3.51.2.7 Rsdk_DT_copyOffset_Vof32cp8Vif32cp8()	136
3.51.2.8 Rsdk_DT_copyOffset_Vof32p8Vif32p8()	137
3.51.2.9 Rsdk_DT_copyOutOffset_Vof32cp8Vif32cp8()	138
3.51.2.10 Rsdk_DT_copyOutOffset_Vof32p8Vif32p8()	139
3.52 Dsp_lal_api_func_polinom	140
3.52.1 Detailed Description	140
3.52.2 Function Documentation	140
3.52.2.1 Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8()	140
3.53 Dsp_lal_api_func_snr	141
3.53.1 Detailed Description	141
3.53.2 Function Documentation	141
3.53.2.1 Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8()	141
3.54 Dsp_lal_demos_api_func_add	142
3.54.1 Detailed Description	142
3.54.2 Function Documentation	142
3.54.2.1 RsdkBbe32LalVectorAdd()	142
4 Data Structure Documentation	143
4.1 dspDispCommParams_t Struct Reference	143
4.1.1 Detailed Description	143
4.1.2 Field Documentation	143
4.1.2.1 asyncChanId	143
4.1.2.2 ctrlChanId	144
4.1.2.3 ipcfInstance	144
4.2 dspDisplnit_t Struct Reference	144
4.2.1 Detailed Description	144

4.2.2 Field Documentation	144
4.2.2.1 commParams	144
4.2.2.2 funcTableSize	145
4.2.2.3 pFuncTable	145
4.2.2.4 prefAggrParams	145
4.2.2.5 sptEvtIntEnBitmap	145
4.2.2.6 sptEvtIsrCb	145
4.3 Dsphd_CommandAckType Struct Reference	145
4.3.1 Detailed Description	146
4.3.2 Field Documentation	146
4.3.2.1 commandNumber	146
4.3.2.2 commandType	146
4.4 Dsphd_JobType Struct Reference	146
4.4.1 Detailed Description	146
4.4.2 Field Documentation	146
4.4.2.1 arg	147
4.4.2.2 id	147
4.5 Dsphd_RuntimeCfgType Struct Reference	147
4.5.1 Detailed Description	147
4.5.2 Field Documentation	147
4.5.2.1 crc	147
4.5.2.2 fpuRoundConfig	148
4.5.2.3 numberOfChirps	148
4.5.2.4 p1TaskConfig	148
4.5.2.5 pDispatcherState	148
4.5.2.6 reportFPUExceptions	148
4.6 Dsphd_TraceCfgType Struct Reference	148
4.6.1 Detailed Description	148
4.6.2 Field Documentation	149
4.6.2.1 bufAddr	149
4.6.2.2 bufSize	149
4.7 dspPrefetchAggrParams_t Struct Reference	149
4.7.1 Detailed Description	149
4.7.2 Field Documentation	149
4.7.2.1 prefetchAggressionData	149
4.7.2.2 prefetchAggressionInstr	150
4.7.2.3 prefetchAggressionSW	150
4.8 rsdkBbe32CaCfarParams_t Struct Reference	150
4.8.1 Detailed Description	150
4.8.2 Field Documentation	150
4.8.2.1 dopplerBins	150
4.8.2.2 guardInterval	151

4.8.2.3 pDetectBitmap	151
4.8.2.4 pRdm	151
4.8.2.5 pThr	151
4.8.2.6 rangeBins	151
4.8.2.7 thresholdFactor	151
4.8.2.8 windowSize	151
4.9 rsdkBbe32CmprDecTxpsParams_t Struct Reference	152
4.9.1 Field Documentation	152
4.9.1.1 mantBwFix	152
4.9.1.2 noChirps	152
4.9.1.3 noRanges	152
4.9.1.4 numExpBits	152
4.9.1.5 numFillBits	153
4.9.1.6 pCcfg	153
4.9.1.7 pDeCmprOut	153
4.9.1.8 pInCmprDat	153
4.9.1.9 pMantBwEncTbl	153
4.9.1.10 pMantBwFlx	153
4.10 rsdkBbe32CmprEncTxpsParams_t Struct Reference	153
4.10.1 Field Documentation	154
4.10.1.1 mantBwFix	154
4.10.1.2 noChirps	154
4.10.1.3 noRanges	154
4.10.1.4 numExpBits	154
4.10.1.5 numFillBits	154
4.10.1.6 pCcfg	155
4.10.1.7 pCmprOut	155
4.10.1.8 pInDat	155
4.10.1.9 pMantBwEncTbl	155
4.10.1.10 pMantBwFlx	155
4.11 rsdkBbe32CmprFixOneBufParams_t Struct Reference	155
4.11.1 Detailed Description	156
4.11.2 Field Documentation	156
4.11.2.1 mantBwFix	156
4.11.2.2 noChirps	156
4.11.2.3 noRanges	156
4.11.2.4 numExpBits	156
4.11.2.5 numFillBits	157
4.11.2.6 pCmprOut	157
4.11.2.7 pInDat	157
4.11.2.8 pMantBwEncTbl	157
4.12 rsdkBbe32CmprFixTwoBuf4Params_t Struct Reference	157

4.12.1 Field Documentation	157
4.12.1.1 mantBwFix	158
4.12.1.2 noChirps	158
4.12.1.3 noRanges	158
4.12.1.4 pCmprExpOut	158
4.12.1.5 pCmprOut	158
4.12.1.6 plnDat	158
4.13 rsdkBbe32CmprFixTwoBuf8Params_t Struct Reference	158
4.13.1 Field Documentation	159
4.13.1.1 mantBwFix	159
4.13.1.2 noChirps	159
4.13.1.3 noRanges	159
4.13.1.4 pCmprExpOut	159
4.13.1.5 pCmprOut	159
4.13.1.6 plnDat	160
4.13.1.7 pMantBwEncTbl	160
4.14 rsdkBbe32CmprFlxOneBufParams_t Struct Reference	160
4.14.1 Field Documentation	160
4.14.1.1 noChirps	160
4.14.1.2 noRanges	160
4.14.1.3 numExpBits	161
4.14.1.4 numFillBits	161
4.14.1.5 pCmprOut	161
4.14.1.6 plnDat	161
4.14.1.7 pMantBwEncTbl	161
4.14.1.8 pMantBwFlx	161
4.15 rsdkBbe32CmprFlxTwoBuf4Params_t Struct Reference	161
4.15.1 Field Documentation	162
4.15.1.1 noChirps	162
4.15.1.2 noRanges	162
4.15.1.3 pCmprExpOut	162
4.15.1.4 pCmprOut	162
4.15.1.5 plnDat	162
4.15.1.6 pMantBwFlx	163
4.16 rsdkBbe32CmprFlxTwoBuf8Params_t Struct Reference	163
4.16.1 Field Documentation	163
4.16.1.1 noChirps	163
4.16.1.2 noRanges	163
4.16.1.3 pCmprExpOut	163
4.16.1.4 pCmprOut	164
4.16.1.5 plnDat	164
4.16.1.6 pMantBwEncTbl	164

4.16.1.7 pMantBwFlx	164
4.17 rsdkBbe32CmprSplnitParams_t Struct Reference	164
4.17.1 Field Documentation	164
4.17.1.1 comprType	165
4.17.1.2 nRange	165
4.17.1.3 numBytesPerChirp	165
4.17.1.4 pMantBwFlx	165
4.18 rsdkBbe32CmprSptParams_t Struct Reference	165
4.18.1 Field Documentation	165
4.18.1.1 noChirps	166
4.18.1.2 noRanges	166
4.18.1.3 numBytesPerChirp	166
4.18.1.4 pCmprOut	166
4.18.1.5 pInDat	166
4.18.1.6 pMantBwFlx	166
4.19 rsdkBbe32DoaParams_t Struct Reference	166
4.19.1 Detailed Description	167
4.19.2 Field Documentation	167
4.19.2.1 noAntennas	167
4.19.2.2 noPeaks	167
4.19.2.3 pDetectBitmap	167
4.19.2.4 pIn	168
4.19.2.5 pOutMag	168
4.19.2.6 pScratch1Area	168
4.19.2.7 pScratch2Area	168
4.19.2.8 pTwiddlesArea	168
4.19.2.9 pWindowArea	169
4.19.2.10 shift	169
4.19.2.11 thresholdFactor	169
4.20 rsdkBbe32Fft2RealParams_t Struct Reference	169
4.20.1 Detailed Description	169
4.20.2 Field Documentation	170
4.20.2.1 pIn1	170
4.20.2.2 pIn2	170
4.20.2.3 pOut1	170
4.20.2.4 pOut2	170
4.20.2.5 pScratch1Area	170
4.20.2.6 pScratch2Area	171
4.20.2.7 pScratch3Area	171
4.20.2.8 pTwiddlesArea	171
4.20.2.9 pWindowArea	171
4.20.2.10 shift	171

4.20.2.11 size	172
4.21 rsdkBbe32FftCplxParams_t Struct Reference	172
4.21.1 Detailed Description	172
4.21.2 Field Documentation	172
4.21.2.1 pInOut	172
4.21.2.2 pScratch1Area	173
4.21.2.3 pScratch2Area	173
4.21.2.4 pTwiddlesArea	173
4.21.2.5 pWindowArea	173
4.21.2.6 shift	173
4.21.2.7 size	174
4.22 rsdkBbe32FftInitParams_t Struct Reference	174
4.22.1 Detailed Description	174
4.22.2 Field Documentation	174
4.22.2.1 pTwiddlesArea	174
4.22.2.2 pWindowArea	175
4.22.2.3 size	175
4.23 rsdkBbe32HistParams_t Struct Reference	175
4.23.1 Detailed Description	175
4.23.2 Field Documentation	175
4.23.2.1 pEdges	175
4.23.2.2 pIn	176
4.23.2.3 pOut	176
4.23.2.4 size	176
4.23.2.5 sizeEdges	176
4.24 rsdkBbe32LalVectorAddParams_t Struct Reference	176
4.24.1 Field Documentation	176
4.24.1.1 pInput1	176
4.24.1.2 pInput2	177
4.24.1.3 pOutput	177
4.24.1.4 size	177
4.25 rsdkBbe32OsCfarParams_t Struct Reference	177
4.25.1 Detailed Description	177
4.25.2 Field Documentation	177
4.25.2.1 dopplerBins	177
4.26 rsdkBbe32ReordTile16to4ChParams_t Struct Reference	178
4.26.1 Detailed Description	178
4.26.2 Field Documentation	178
4.26.2.1 doubleInBufEn	178
4.26.2.2 noElements	178
4.26.2.3 pInDat	178
4.26.2.4 pOutDat	179

4.27 rsdkBbe32TdlmParams_t Struct Reference	179
4.27.1 Field Documentation	179
4.27.1.1 pln	179
4.27.1.2 plnDataAbs	179
4.27.1.3 pOut	179
4.27.1.4 pTotalSum	180
4.27.1.5 samples	180
4.27.1.6 scalingFactor	180
4.27.1.7 thrPercent	180
Index	181

Chapter 1

Module Index

1.1 Software Modules

Here is a list of all modules:

DSP	5
DSP User Guide	5
DSP Dispatcher API	5
DSP Dispatcher Constants	6
DSP Dispatcher Data Types	8
DSP Dispatcher Functions	9
DSP Radar Baseband Algo Library API	11
DSP Radar Algo Functions	11
DSP Radar Algo Constants	24
DSP Radar Algo Data Types	24
DSP Host Driver API	25
DSP Host Driver Constants	26
DSP Host Driver Data Types	26
DSP Host Driver Functions	30
DSP LAL API	32
DSP LAL Demo Data	33
DSP LAL Constants	33
DSP LAL Demo Constants	37
DSP LAL Functions	37
DSP LAL Demo Functions	38
Dsp_lal_api_func_add_vec	38
Dsp_lal_api_func_sub_vec	40
Dsp_lal_api_func_mul_vec_mat	43
Dsp_lal_api_func_div_vec	50
Dsp_lal_api_func_xcorr_vec	52
Dsp_lal_api_func_op_vec_mat	53
Dsp_lal_api_func_fill	61
Dsp_lal_api_func_trig_vec	70
Dsp_lal_api_func_log_vec	75
Dsp_lal_api_func_prod_vec	78
Dsp_lal_api_func_det_mat	81
Dsp_lal_api_func_qr_dec_mat	83
Dsp_lal_api_func_cholesky_dec_mat	86
Dsp_lal_api_func_conj_vec	87

Dsp_lal_api_func_min_max_vec	88
Dsp_lal_api_func_sum_vec	91
Dsp_lal_api_func_inv_up_triangular_mat	96
Dsp_lal_api_func_inv_low_triangular_mat	99
Dsp_lal_api_func_inv_qr_mat	102
Dsp_lal_api_func_inv_gauss_mat	104
Dsp_lal_api_func_eig_jacobi_mat	107
Dsp_lal_api_func_eig_ql_mat	108
Dsp_lal_api_func_eig_QRfrancis_mat	111
Dsp_lal_api_func_mean_vec	113
Dsp_lal_api_func_conversions_vec_mat	118
Dsp_lal_api_func_sort_vec	122
Dsp_lal_api_func_sp_smooth	125
Dsp_lal_func_api_dist	126
Dsp_lal_api_func_prod_tot	128
Dsp_lal_api_func_hessenberg_mat	129
Dsp_lal_api_func_copy	131
Dsp_lal_api_func_polinom	140
Dsp_lal_api_func_snr	141
Dsp_lal_demos_api_func_add	142

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

dspDispCommParams_t	This structure is used to initialize the IPCF communication parameters used between the Dispatcher and the Host Driver	143
dspDisplnit_t	Initialization parameters for the Dispatcher	144
Dsphd_CommandAckType	This structure packs the information needed to send an ACK from the DSP Dispatcher to the core	145
Dsphd_JobType	This structure packs the information needed to call a function remotely on the BBE32 DSP, through the DSP Dispatcher	146
Dsphd_RuntimeCfgType	This structure packs the information needed to configure the runtime configuration of the DSP Dispatcher	147
Dsphd_TraceCfgType	This structure packs the information needed to configure and start application trace logging in the DSP Dispatcher	148
dspPrefetchAggrParams_t	This structure is used to initialize the prefetch aggression parameters for HW Data, Hw Instr, Sw	149
rsdkBbe32CaCfarParams_t	The data types accepted by the BBE32 algos	150
rsdkBbe32CmprDecTxpsParams_t		152
rsdkBbe32CmprEncTxpsParams_t		153
rsdkBbe32CmprFixOneBufParams_t	The data types accepted by the BBE32 algos	155
rsdkBbe32CmprFixTwoBuf4Params_t		157
rsdkBbe32CmprFixTwoBuf8Params_t		158
rsdkBbe32CmprFlxOneBufParams_t		160
rsdkBbe32CmprFlxTwoBuf4Params_t		161
rsdkBbe32CmprFlxTwoBuf8Params_t		163
rsdkBbe32CmprSpInitParams_t		164
rsdkBbe32CmprSptParams_t		165
rsdkBbe32DoaParams_t	The data types accepted by the BBE32 algos	166
rsdkBbe32Fft2RealParams_t	The data types accepted by the BBE32 algos	169

rsdkBbe32FftCplxParams_t	
This struture is used to pass the window and twiddles area, for the RsdkBbe32FftInit() wrapper	172
rsdkBbe32FftInitParams_t	
The data types accepted by the BBE32 algos	174
rsdkBbe32HistParams_t	
The data types accepted by the BBE32 algos	175
rsdkBbe32LalVectorAddParams_t	176
rsdkBbe32OsCfarParams_t	
The data types accepted by the BBE32 algos	177
rsdkBbe32ReordTile16to4ChParams_t	
The data types accepted by the BBE32 algos	178
rsdkBbe32TdImParams_t	179

Chapter 3

Module Documentation

3.1 DSP

Modules

- [DSP User Guide](#)
- [DSP Dispatcher API](#)
DSP Dispatcher API and module description.
- [DSP Radar Baseband Algo Library API](#)
DSP Radar Baseband Algo Library API.
- [DSP Host Driver API](#)
DSP Host Driver API.
- [DSP LAL API](#)
DSP Linear Algebra Library API.

3.1.1 Detailed Description

3.2 DSP User Guide

3.3 DSP Dispatcher API

DSP Dispatcher API and module description.

Modules

- [DSP Dispatcher Constants](#)
DSP Dispatcher Constants.
- [DSP Dispatcher Data Types](#)
DSP Dispatcher Data Types.
- [DSP Dispatcher Functions](#)
DSP Dispatcher Functions.

3.3.1 Detailed Description

DSP Dispatcher API and module description.

Describes the functions, data types and structures that make up the DSP Dispatcher user interface. This module is used to facilitate easy integration of the BBE32 DSP into an application running on the host CPU. The `spt_ug` explains how this module is built and integrated. The diagram below shows a high-level integration view.

3.4 DSP Dispatcher Constants

DSP Dispatcher Constants.

Macros

- `#define RSDK_DSP_SPT_EVT0_INTEN_BIT (1u << 0u)`
- `#define RSDK_DSP_SPT_EVT1_INTEN_BIT (1u << 1u)`
- `#define RSDK_DSP_SPT_EVT2_INTEN_BIT (1u << 2u)`
- `#define RSDK_DSP_SPT_EVT3_INTEN_BIT (1u << 3u)`
- `#define RSDK_DSP_SPT_EVT4_INTEN_BIT (1u << 4u)`
- `#define RSDK_DSP_SPT_EVT5_INTEN_BIT (1u << 5u)`
- `#define RSDK_DSP_SPT_EVT6_INTEN_BIT (1u << 6u)`
- `#define RSDK_DSP_SPT_EVT7_INTEN_BIT (1u << 7u)`
- `#define RSDK_DSP_SPT_EVT8_INTEN_BIT (1u << 8u)`
- `#define RSDK_DSP_SPT_EVT9_INTEN_BIT (1u << 9u)`
- `#define RSDK_DSP_SPT_EVT10_INTEN_BIT (1u << 10u)`
- `#define RSDK_DSP_SPT_EVT11_INTEN_BIT (1u << 11u)`
- `#define RSDK_DSP_SPT_EVT12_INTEN_BIT (1u << 12u)`

3.4.1 Detailed Description

DSP Dispatcher Constants.

3.4.2 Macro Definition Documentation

3.4.2.1 RSDK_DSP_SPT_EVT0_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT0_INTEN_BIT (1u << 0u)
```

Bit used to enable the SPT Event 0 interrupt. Intended to be assigned to `dspDisplnit_t::sptEvtIntEnBitmap`

3.4.2.2 RSDK_DSP_SPT_EVT10_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT10_INTEN_BIT (1u << 10u)
```

Bit used to enable the SPT Event 10 interrupt. Intended to be assigned to `dspDisplnit_t::sptEvtIntEnBitmap`

3.4.2.3 RSDK_DSP_SPT_EVT11_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT11_INTEN_BIT (1u << 11u)
```

Bit used to enable the SPT Event 11 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.4 RSDK_DSP_SPT_EVT12_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT12_INTEN_BIT (1u << 12u)
```

Bit used to enable the SPT Event 12 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.5 RSDK_DSP_SPT_EVT1_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT1_INTEN_BIT (1u << 1u)
```

Bit used to enable the SPT Event 1 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.6 RSDK_DSP_SPT_EVT2_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT2_INTEN_BIT (1u << 2u)
```

Bit used to enable the SPT Event 2 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.7 RSDK_DSP_SPT_EVT3_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT3_INTEN_BIT (1u << 3u)
```

Bit used to enable the SPT Event 3 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.8 RSDK_DSP_SPT_EVT4_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT4_INTEN_BIT (1u << 4u)
```

Bit used to enable the SPT Event 4 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.9 RSDK_DSP_SPT_EVT5_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT5_INTEN_BIT (1u << 5u)
```

Bit used to enable the SPT Event 5 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.10 RSDK_DSP_SPT_EVT6_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT6_INTEN_BIT (1u << 6u)
```

Bit used to enable the SPT Event 6 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.11 RSDK_DSP_SPT_EVT7_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT7_INTEN_BIT (1u << 7u)
```

Bit used to enable the SPT Event 7 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.12 RSDK_DSP_SPT_EVT8_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT8_INTEN_BIT (1u << 8u)
```

Bit used to enable the SPT Event 8 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.4.2.13 RSDK_DSP_SPT_EVT9_INTEN_BIT

```
#define RSDK_DSP_SPT_EVT9_INTEN_BIT (1u << 9u)
```

Bit used to enable the SPT Event 9 interrupt. Intended to be assigned to [dspDisplnit_t::sptEvtIntEnBitmap](#)

3.5 DSP Dispatcher Data Types

DSP Dispatcher Data Types.

Data Structures

- struct [dspDispCommParams_t](#)
This structure is used to initialize the IPCF communication parameters used between the Dispatcher and the Host Driver.
- struct [dspPrefetchAggrParams_t](#)
This structure is used to initialize the prefetch aggression parameters for HW Data, Hw Instr, Sw.
- struct [dspDisplnit_t](#)
Initialization parameters for the Dispatcher.

Typedefs

- typedef void * [rsdkDspFuncPtr_t](#)
Alias datatype for specifying generic DSP function pointers.
- typedef void(* [rsdkDsplsCb_t](#)) (uint32_t arg)
Definition of callback function type to be called by the DSP interrupt handlers.

3.5.1 Detailed Description

DSP Dispatcher Data Types.

3.5.2 Typedef Documentation

3.5.2.1 rsdkDspFuncPtr_t

```
typedef void* rsdkDspFuncPtr_t
```

Alias datatype for specifying generic DSP function pointers.

3.5.2.2 rsdkDsplsrCb_t

```
typedef void(* rsdkDspIsrCb_t) (uint32_t arg)
```

Definition of callback function type to be called by the DSP interrupt handlers.

This is offered to the user to customize functionality of the interrupt handlers.

Parameters

in	arg	ID of the interrupt
----	-----	---------------------

Returns

void

3.6 DSP Dispatcher Functions

DSP Dispatcher Functions.

Functions

- rsdkStatus_t [RsdKdspDispatcherInit](#) ([dspDisplnit_t](#) const *const pDisplnitInfo)
Initialize the DSP Dispatcher to be ready for receiving commands from SPT.
- void [RsdKdspDispatcherRun](#) (void)
Runs the Dispatcher scheduling loop.

3.6.1 Detailed Description

DSP Dispatcher Functions.

•

3.6.2 Function Documentation

3.6.2.1 RsdKdSpDispatcherInit()

```
rsdkStatus_t RsdKdSpDispatcherInit (
    dspDispInit_t const *const pDispInitInfo )
```

Initialize the DSP Dispatcher to be ready for receiving commands from SPT.

Initializes the MPU and exceptions table, enables the interrupts (all except the timer interrupt), sets the handlers, initializes the internal state of the Dispatcher, initializes the prefetch aggressiveness

Parameters

in	pDispInitInfo	Initialization parameters for the Dispatcher: MPU table, SPT events callbacks, prefetch aggressiveness
----	---------------	--

Returns

#RSDK_SUCCESS or a specific error code.

Precondition

It should be called right after DSP boot, before running the RsdKdSpRunDispatcher().

3.6.2.2 RsdKdSpDispatcherRun()

```
void RsdKdSpDispatcherRun (
    void )
```

Runs the Dispatcher scheduling loop.

The Dispatcher runs in an infinite loop: waits for commands from the SPT command queue, parses them and triggers execution of requested DSP algorithms.

Precondition

It should be called right after DSP boot, before running the [RsdKdSpDispatcherRun\(\)](#).

Postcondition

This function never returns during normal operation, but only if it detects certain errors in execution. In those cases it signals an error code through the SPT "DSP" interrupt before exiting.

3.7 DSP Radar Baseband Algo Library API

DSP Radar Baseband Algo Library API.

Modules

- [DSP Radar Algo Functions](#)
DSP Radar Algo Library Functions.
- [DSP Radar Algo Constants](#)
DSP Radar Algo Library Constants.
- [DSP Radar Algo Data Types](#)
DSP Radar Algo Library Data Types.

3.7.1 Detailed Description

DSP Radar Baseband Algo Library API.

Describes the functions contained in the DSP Radar Algo Library.

3.8 DSP Radar Algo Functions

DSP Radar Algo Library Functions.

Functions

- void [RsdKBbe32CaCfar](#) (uintptr_t argList)
1D CA-CFAR implementation iso-doppler dimension Using sliding window approach to avoid redundant summations.
- void [RsdKBbe32OsCfar](#) (uintptr_t argList)
1D OS-CFAR implementation across RDM 'range' dimension
- void [RsdKBbe32FftInit](#) (uintptr_t argList)
Initialization of FFT window and twiddle coefficients for one particular size.
- void [RsdKBbe32FftCplx](#) (uintptr_t argList)
FFT 24bits complex.
- void [RsdKBbe32Fft2Real](#) (uintptr_t argList)
FFT 24bits 2 real inputs.
- void [RsdKBbe32Hist](#) (uintptr_t argList)
Histogram 24bits real.
- void [RsdKBbe32Doa128](#) (uintptr_t argList)
DoA 128 24bits complex input.
- void [RsdKBbe32Doa256](#) (uintptr_t argList)
DoA 256 24bits complex input.
- void [RsdKBbe32ReordTile16to4Ch](#) (uintptr_t argList)
Reorder tile 16 data from 1 buffer of 16 bits elements to 4 data buffer of 16 bits elements.
- void [RsdKBbe32EncSeqFixOnebuf](#) (uintptr_t argList)
Sequential compression, fixed, one buffer of 8 bits elements.

- void [RsdkBbe32EncSeqFixTwobuf4](#) (uintptr_t argList)
Sequential compression, fixed, two buffers of 4 bits elements.
- void [RsdkBbe32EncSeqFixTwobuf8](#) (uintptr_t argList)
Sequential compression, fixed, two buffers of 8 bits elements.
- void [RsdkBbe32EncSeqFlxOnebuf](#) (uintptr_t argList)
Sequential compression, flexible, one buffer of 8 bits elements.
- void [RsdkBbe32EncSeqFlxTwobuf4](#) (uintptr_t argList)
Sequential compression, flexible, two buffers of 4 bits elements.
- void [RsdkBbe32EncSeqFlxTwobuf8](#) (uintptr_t argList)
Sequential compression, flexible, two buffers of 8 bits elements.
- void [RsdkBbe32CmprSptInit](#) (uintptr_t argList)
Init for SPT mode compressions.
- void [RsdkBbe32EncCp4d](#) (uintptr_t argList)
Compression, SPT mode CP4D, one buffer of 8 bits elements.
- void [RsdkBbe32EncCp4dFmta](#) (uintptr_t argList)
Compression, SPT mode CP4DFMTA, one buffer of 8 bits elements.
- void [RsdkBbe32EncCp4dFmtb](#) (uintptr_t argList)
Compression, SPT mode CP4DFMTB, one buffer of 8 bits elements.
- void [RsdkBbe32EncFlxSpt34](#) (uintptr_t argList)
Compression, SPT mode FLXSPT34, one buffer of 8 bits elements.
- void [RsdkBbe32EncTxps](#) (uintptr_t argList)
Compression, transposed output, one buffer of 8 bits elements.
- void [RsdkBbe32DecTxps](#) (uintptr_t argList)
Decompression, transposed output, one buffer of 8 bits elements.
- void [RsdkBbe32TdIm](#) (uintptr_t argList)
Time-domain Interference Mitigation.

3.8.1 Detailed Description

DSP Radar Algo Library Functions.

.

3.8.2 Function Documentation

3.8.2.1 RsdkBbe32CaCfar()

```
void RsdkBbe32CaCfar (
    uintptr_t argList )
```

1D CA-CFAR implementation iso-doppler dimension Using sliding window approach to avoid redundant summations.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CaCfarParams_t containing the CA-CFAR parameters and buffers.
----	----------------	--

Precondition

This implementation assumes the following memory layout of the input RDM buffer passed by [rsdkBbe32CaCfarParams_t::pRdm](#) :

INPUT Memory layout example for RangeBins = Nr, DopplerBins = Nd: Sample format: 16 bit real SPT "Log2 Mag2"

offset bytes:	0	2	...	2*(Nd-2)	2*(Nd-1)
0	R0_D0	R0_D1	...	R0_D<Nd-1>	R0_D<Nd>
2*Nd	R1_D0	R1_D1	...	R1_D<Nd-1>	R1_D<Nd>
	:	:		:	:
	:	:		:	:
2*(Nr-1)*Nd	R<Nr>_D0	R1<Nr>_D1	...	R1<Nr>_D<Nd-1>	R1<Nr>_D<Nd>

3.8.2.2 RsdKBbe32CmprSptInit()

```
void RsdKBbe32CmprSptInit (
    uintptr_t argList )
```

Init for SPT mode compressions.

Performs compression init part for [RsdKBbe32EncCp4d\(\)](#) and computed number of output bytes for chirp for the actual compression., . This includes the part of the BBE32 registers initialisation for SPT compressions([RsdKBbe32EncCp4d](#), [RsdKBbe32EncCp4dFmta](#), [RsdKBbe32EncCp4dFmtb](#), [RsdKBbe32EncFlxSpt34](#)); IMPORTANT: For Flex compression([RsdKBbe32EncFlxSpt34](#)) this needs to be called once per frame. IMPORTANT: For fixed compression([RsdKBbe32EncCp4d](#), [RsdKBbe32EncCp4dFmta](#), [RsdKBbe32EncCp4dFmtb](#)) this needs to be called only once in the beginning of the processing.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprSptInitParams_t containing the compression init parameters.
----	----------------	--

3.8.2.3 RsdKBbe32DecTxps()

```
void RsdKBbe32DecTxps (
    uintptr_t argList )
```

Decompression, transposed output, one buffer of 8 bits elements.

Performs decompression of the transposed compression, resulting data, which represent the original data from 4 antennas. This includes the part of the BBE32 registers initialisation for `decompression(Rsdk_EncInInit(), Rsdk_↵ EncTxpsInInit() and Rsdk_DecTxpsInInit())` and the actual decompression (`Rsdk_DecTxps()`)

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprDecTxpsParams_t containing the decompression parameters and buffers.
----	----------------	---

Precondition

`Rsdk_EncTxpsInInit()` can be run before as it returns the expected number of bytes for the compressed input.

3.8.2.4 RsdkBbe32Doa128()

```
void RsdkBbe32Doa128 (
    uintptr_t argList )
```

DoA 128 24bits complex input.

Performs DoA based on FFT128 on 24bits on complex operands. The algorithm extends an input vector to 128 complex elements(fill with 0), applies the corresponding FFT and computes the $\text{mag2} \Rightarrow (\text{real}^2 + \text{imag}^2)/2$. Using these values, the maximum mag2 value is computed and multiplied with a threshold factor obtaining a threshold value.

This result is then compared with all the local maximum values($\text{localMaxMag2} > \text{threshold}$) of the mag2 for creating a bitmap.

A value $\text{mag2}(i)$ is considered a local maximum value if $\text{mag2}(i) > \text{mag2}(i - 1)$ and $\text{mag2}(i) \geq \text{mag2}(i+1)$.

The start and the end of the resulting mag2 vector are not considered.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32DoaParams_t containing the DoA parameters and buffers.
----	----------------	---

Precondition

[RsdkBbe32FftInInit\(\)](#) must be called for initializing FFT window and twiddles coefficients before calling this function the same size parameter.

The `pOutMag` and `pln` are pointers to the real mag2 output and complex input data respectively, which use `int32_t`.

Even though the operations are performed on 24bits (re, imag), the data is stored in memory on 32 bits. The outputs are sign extended to 32 bits.

The `pDetectBitmap` contains the resulting bitmap and is represented using `uint16_t`.

The shift value should be `RSDK_BBE32_FFT_SHIFT_2` since a division by 4 is required before performing each radix4/radix2 round to prevent saturation.

The available sizes are described in `rsdkBbe32FftSize_t` enum.

The output signals should be scaled with $(\text{shift_value}^4)^2 / 256$.

The twiddles and window coeffi are converted inside the used FFT kernel from q0.15 to q0.23 fixed point.

3.8.2.5 RsdkBbe32Doa256()

```
void RsdkBbe32Doa256 (
    uintptr_t argList )
```

DoA 256 24bits complex input.

Performs DoA based on FFT256 on 24bits on complex operands. The algorithm extends an input vector to 256 complex elements(fill with 0), applies the corresponding FFT and computes the $\text{mag2} \Rightarrow (\text{real}^2 + \text{imag}^2)/2$. Using these values, the maximum mag2 value is computed and multiplied with a threshold factor obtaining a threshold value.

This result is then compared with all the local maximum values($\text{localMaxMag2} > \text{threshold}$) of the mag2 for creating a bitmap.

A value $\text{mag2}(i)$ is considered a local maximum value if $\text{mag2}(i) > \text{mag2}(i - 1)$ and $\text{mag2}(i) \geq \text{mag2}(i+1)$.

The start and the end of the resulting mag2 vector are not considered.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32DoaParams_t containing the DoA parameters and buffers.
----	---------	---

Precondition

[RsdkBbe32FftInit\(\)](#) must be called for initializing FFT window and twiddles coefficients before calling this function the same size parameter.

The pOutMag and pln are pointers to the real mag2 output and complex input data respectively, which use [int32_t](#).

Even though the operations are performed on 24bits (re, imag), the data is stored in memory on 32 bits. The outputs are sign extended to 32 bits.

The pDetectBitmap contains the resulting bitmap and is represented using [uint16_t](#).

The shift value should be [RSDK_BBE32_FFT_SHIFT_2](#) since a division by 4 is required before performing each radix4/radix2 round to prevent saturation.

The available sizes are described in [rsdkBbe32FftSize_t](#) enum.

The output signals should be scaled with $(\text{shift_value}^4)^2 / 256$.

The twiddles and window coefficients are converted inside the used FFT kernel from q0.15 to q0.23 fixed point.

3.8.2.6 RsdkBbe32EncCp4d()

```
void RsdkBbe32EncCp4d (
    uintptr_t argList )
```

Compression, SPT mode CP4D, one buffer of 8 bits elements.

Performs compression(SPT mode CP4D) on the input data representing data from 4 antennas.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32CmprSptParams_t containing the compression parameters and buffers.
----	---------	---

Precondition

[RsdkBbe32CmprSptInit\(\)](#) needs to be ran before(only once) for compression initialization and to obtain the number of bytes per chirp for the compressed output(parameter needs to be copied and passed to this wrapper).

3.8.2.7 RsdkBbe32EncCp4dFmta()

```
void RsdkBbe32EncCp4dFmta (
    uintptr_t argList )
```

Compression, SPT mode CP4DFMTA, one buffer of 8 bits elements.

Performs compression(SPT mode CP4DFMTA) on the input data representing data from 4 antennas.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprSptParams_t containing the compression parameters and buffers.
----	----------------	---

Precondition

[RsdkBbe32CmprSptInit\(\)](#) needs to be ran before(only once) for compression initialization and to obtain the number of bytes per chirp for the compressed output(parameter needs to be copied and passed to this wrapper).

3.8.2.8 RsdkBbe32EncCp4dFmtb()

```
void RsdkBbe32EncCp4dFmtb (
    uintptr_t argList )
```

Compression, SPT mode CP4DFMTB, one buffer of 8 bits elements.

Performs compression(SPT mode CP4DFMTB) on the input data representing data from 4 antennas.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprSptParams_t containing the compression parameters and buffers.
----	----------------	---

Precondition

[RsdkBbe32CmprSptInit\(\)](#) needs to be ran before(only once) for compression initialization and to obtain the number of bytes per chirp for the compressed output(parameter needs to be copied and passed to this wrapper).

3.8.2.9 RsdkBbe32EncFlxSpt34()

```
void RsdkBbe32EncFlxSpt34 (
    uintptr_t argList )
```

Compression, SPT mode FLXSPT34, one buffer of 8 bits elements.

Performs compression(flex compression compatible with SPT3.4) on the input data representing data from 4 antennas.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32CmprSptParams_t containing the compression parameters and buffers.
----	---------	---

Precondition

[RsdkBbe32CmprSptInit\(\)](#) needs to be ran before(once per frame) for compression initialization. Also this is required to obtain the number of bytes per chirp for the compressed output(parameter needs to be copied and passed to this wrapper) and also for obtaining the bid width mantissa table computed(same parameter, p↔ MantBwFlx, need to be used for init and in this wrapper).

3.8.2.10 RsdkBbe32EncSeqFixOnebuf()

```
void RsdkBbe32EncSeqFixOnebuf (
    uintptr_t argList )
```

Sequential compression, fixed, one buffer of 8 bits elements.

Performs Sequential compression, fixed, on the input data representing data from 4 antennas, for cmprOut↔ Mode=ONEBUF. This includes the part of the BBE32 registers initialisation for compression([Rsdk_EncInit\(\)](#)) and the actual compression ([Rsdk_EncSeqFixOnebuf\(\)](#))

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32CmprFixOneBufParams_t containing the compression parameters and buffers.
----	---------	---

Precondition

[Rsdk_EncSeqBytesPerChirp\(\)](#) can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.11 RsdkBbe32EncSeqFixTwobuf4()

```
void RsdkBbe32EncSeqFixTwobuf4 (
    uintptr_t argList )
```

Sequential compression, fixed, two buffers of 4 bits elements.

Performs Sequential compression, fixed, on the input data representing data from 4 antennas, for `cmprOutMode=TWOF4`. This includes the part of the BBE32 registers initialisation for compression(`Rsdk_EncInit()`) and the actual compression (`Rsdk_EncSeqFixTwobuf4()`)

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprFixTwoBuf4Params_t containing the compression parameters and buffers.
----	----------------	--

Precondition

`Rsdk_EncSeqBytesPerChirp()` can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.12 RsdkBbe32EncSeqFixTwobuf8()

```
void RsdkBbe32EncSeqFixTwobuf8 (
    uintptr_t argList )
```

Sequential compression, fixed, two buffers of 8 bits elements.

Performs Sequential compression, fixed, on the input data representing data from 4 antennas, for `cmprOutMode=TWOF8`. This includes the part of the BBE32 registers initialisation for compression(`Rsdk_EncInit()`) and the actual compression (`Rsdk_EncSeqFixTwobuf8()`)

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprFixTwoBuf8Params_t containing the compression parameters and buffers.
----	----------------	--

Precondition

`Rsdk_EncSeqBytesPerChirp()` can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.13 RsdkBbe32EncSeqFlxOnebuf()

```
void RsdkBbe32EncSeqFlxOnebuf (
    uintptr_t argList )
```

Sequential compression, flexible, one buffer of 8 bits elements.

Performs Sequential compression on the input data representing data from 4 antennas, for `cmprOutMode=ONEBUF`. This includes the part of the BBE32 register initialisation for compression(`Rsdk_EncInit()`) and the actual compression (`Rsdk_EncSeqFlxOnebuf()`)

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprFlxOneBufParams_t containing the compression parameters and buffers.
----	----------------	---

Precondition

Rsdk_EncSeqBytesPerChirp() can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.14 RsdkBbe32EncSeqFlxTwobuf4()

```
void RsdkBbe32EncSeqFlxTwobuf4 (
    uintptr_t argList )
```

Sequential compression, flexible, two buffers of 4 bits elements.

Performs Sequential compression, flexible, on the input data representing data from 4 antennas, for cmprOut↔ Mode=TWOBUF4. This includes the part of the BBE32 registers initialisation for compression(Rsdk_EncInit()) and the actual compression (Rsdk_EncSeqFlxTwobuf4())

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprFlxTwoBuf4Params_t containing the compression parameters and buffers.
----	----------------	--

Precondition

Rsdk_EncSeqBytesPerChirp() can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.15 RsdkBbe32EncSeqFlxTwobuf8()

```
void RsdkBbe32EncSeqFlxTwobuf8 (
    uintptr_t argList )
```

Sequential compression, flexible, two buffers of 8 bits elements.

Performs Sequential compression, flexible, on the input data representing data from 4 antennas, for cmprOut↔ Mode=TWOBUF8. This includes the part of the BBE32 registers initialisation for compression(Rsdk_EncInit()) and the actual compression (Rsdk_EncSeqFlxTwobuf8())

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32CmprFlxTwoBuf8Params_t containing the compression parameters and buffers.
----	----------------	--

Precondition

Rsdk_EncSeqBytesPerChirp() can be run before to compute the number of bytes per chirp for the compressed output.

3.8.2.16 RsdkBbe32EncTxps()

```
void RsdkBbe32EncTxps (
    uintptr_t argList )
```

Compression, transposed output, one buffer of 8 bits elements.

Performs transpose compression on the input data representing data from 4 antennas. This includes the part of the BBE32 registers initialisation for compression(Rsdk_EncInit() and Rsdk_EncTxpsInit()) and the actual compression (Rsdk_EncTxps())

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32CmprEncTxpsParams_t containing the compression parameters and buffers.
----	---------	---

Precondition

Rsdk_EncTxpsInit() can be run before as it returns the number of bytes for the compressed output.

3.8.2.17 RsdkBbe32Fft2Real()

```
void RsdkBbe32Fft2Real (
    uintptr_t argList )
```

FFT 24bits 2 real inputs.

Performs FFT on 24bits 2 real operands in 1 complex operand. Lower half of each Fft is returned.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32Fft2RealParams_t containing the FFT parameters and buffers.
----	---------	--

Precondition

[RsdkBbe32FftInit\(\)](#) must be called for initializing FFT window and twiddles coefficients before calling this function the same size parameter.

The pOut1 and pOut2 are pointers that serve both as two output complex lower half FFTs(one for each signal) and

uses `int32_t`.

Even though the operations are performed on 24bits (re, imag), the data is stored in memory on 32 bits. The outputs are sign extended to 32 bits.

The shift value should be `RSDK_BBE32_FFT_SHIFT_2` since a division by 4 is required before performing each radix4/radix2 round to prevent saturation.

The available sizes are described in `rsdkBbe32FftSize_t` enum.

OPTIONAL: For maintaining the accuracy of the results, the two output FFTs can be scaled with $\text{shift_value}^{\text{radix_stages}}$.

OPTIONAL: `pIn1` and `pOut1` can point to the same memory location, `pIn2` and `pOut2` can point to the same memory location.

There are 4 stages for 128 and 256 input size buffers; 5 stages for 512 and 1024 input size; 6 stages for 2048 input size.

The twiddles and window coefficients are converted inside the used complex FFT kernel from q0.15 to q0.23 fixed point.

3.8.2.18 RsdkBbe32FftCplx()

```
void RsdkBbe32FftCplx (
    uintptr_t argList )
```

FFT 24bits complex.

Performs FFT on 24bits complex operands.

Parameters

in	<code>argList</code>	- pointer to a data structure of type <code>rsdkBbe32FftCplxParams_t</code> containing the FFT parameters and buffers.
----	----------------------	--

Precondition

`RsdkBbe32FftInit()` must be called for initializing FFT window and twiddles coefficients before calling this function the same size parameter. The FFT is computed inplace, meaning that the `pInOut` serves both as an input complex signal, and as the output complex FFT and uses `int32_t` for real and imaginary parts.

Even though the operations are performed on 24bits (re, imag), the data is stored in memory on 32 bits. The output is sign extended to 32 bits. The shift value should be `RSDK_BBE32_FFT_SHIFT_2` since a division by 4 is required before performing each radix4/radix2 round to prevent saturation.

The output signals should be scaled with $\text{shift_value}^{\text{radix_stages}}$.

The available sizes are described in `rsdkBbe32FftSize_t` enum.

OPTIONAL: For maintaining the accuracy of the result, the output FFT can be scaled with $\text{shift_value}^{\text{radix_stages}}$. The twiddles and window coefficients are converted from q0.15 to q0.23 fixed point.

3.8.2.19 RsdkBbe32FftInit()

```
void RsdkBbe32FftInit (
    uintptr_t argList )
```

Initialization of FFT window and twiddle coefficients for one particular size.

Copies the window and twiddle coefficients(recommended in SRAM) to the two pointers provided(recommened in DRAM).

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32FftInitParams_t containing the FFT size, twiddles and windows destination areas.
----	----------------	---

3.8.2.20 RsdkBbe32Hist()

```
void RsdkBbe32Hist (
    uintptr_t argList )
```

Histogram 24bits real.

Performs Histogram on 24 bits 2 real inputs (data and edges).

It sorts data into bins with bin edges specified in the edges vector. The output values (except the last two scalar values) represent how many input values respect the following inequation: $\text{edge}(i) \leq \text{in} < \text{edge}(i + 1)$; $i=0\dots M-1$. The second last scalar value(int32_t) represents how many input values respect the following inequation: $\text{edge}(M-2) \leq \text{in} \leq \text{edge}(M-1)$. The last scalar value(int32_t) is not useful data.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32HistParams_t containing the Histogram parameters and buffers.
----	----------------	--

Precondition

The pOut an pln and pEdges are pointers to real output and input data which use int32_t.

The output are sign extended to 32 bits.

Values of the edges(threshold values) must be stored in ascending order.

The size(N) and sizeEdges(M) are restricted to being multiple of 16.

The useful output data has size equal to (sizeEdges - 1).

3.8.2.21 RsdkBbe32OsCfar()

```
void RsdkBbe32OsCfar (
    uintptr_t argList )
```

1D OS-CFAR implementation across RDM 'range' dimension

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32OsCfarParams_t containing the OS-CFAR parameters.
----	----------------	--

This function reads the input samples (from ORAM) and the sliding window thresholds (from ORAM) computed in the RsdkSptOsCfar kernel and then computes a new threshold vector.

The newly formed threshold vector is compared with the input samples element wise. If the sample is greater than the threshold the corresponding bit is set to 1 (1byte = 8 elements). The result is written back in ORAM.

Precondition

!!! THIS FUNCTION SHOULD ONLY BE USED INSIDE THE RsdKsptOsCfar KERNEL

The input is taken from ORAM and the output is written to ORAM.

The memory map is fixed. Input samples: OR_0_0_0, Input thresholds for BBE32: OR_2_0_0, Output: OR_0_0_0

If the memory layout is changed in the RsdKsptOsCfar kernel then RsdKBbe32OsCfar needs to be changed in the following way:

op_br0 - controls the bank selection for RLU0

op_br1 - controls the bank selection for RLU1

op_bw0 - controls the bank selection for WLU

RESTRICTION: op_br0, op_br1 and op_bw0 SHOULD POINT TO DIFFERENT CONTROLLERS

3.8.2.22 RsdKBbe32ReordTile16to4Ch()

```
void RsdKBbe32ReordTile16to4Ch (
    uintptr_t argList )
```

Reorder tile 16 data from 1 buffer of 16 bits elements to 4 data buffer of 16 bits elements.

Performs a reorder from a buffer containing data from 4 channels tile 16 (16 elem ch1, 16 elem ch2, 16 elem ch3, 16 elem ch4, 16 elem ch1, 16 elem ch2, 16 elem ch3, 16 elem ch4,) to 4 buffers of 16 bits elements.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32ReordTile16to4ChParams_t containing the reorder parameters and buffers.
----	---------	--

Precondition

noElements, representing the number of input elements must be multiple of 64.

3.8.2.23 RsdKBbe32TdIm()

```
void RsdKBbe32TdIm (
    uintptr_t argList )
```

Time-domain Interference Mitigation.

Performs time-domain interference mitigation by zeroing the interference (no reconstruction) in time domain by means of a sort on an average filter.

Parameters

in	<i>argList</i>	- pointer to a data structure of type rsdkBbe32TdlmParams_t containing the algo parameters and buffers.
----	----------------	---

3.9 DSP Radar Algo Constants

DSP Radar Algo Library Constants.

Macros

- `#define BBE_SIMD_WIDTH (16u)`
- `#define BBE_SIMD_WIDTH (16u)`

3.9.1 Detailed Description

DSP Radar Algo Library Constants.

•

3.9.2 Macro Definition Documentation

3.9.2.1 BBE_SIMD_WIDTH [1/2]

```
#define BBE_SIMD_WIDTH (16u)
```

3.9.2.2 BBE_SIMD_WIDTH [2/2]

```
#define BBE_SIMD_WIDTH (16u)
```

3.10 DSP Radar Algo Data Types

DSP Radar Algo Library Data Types.

Data Structures

- struct [rsdkBbe32CaCfarParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32OsCfarParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32DoaParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32FftInitParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32FftCplxParams_t](#)
This structure is used to pass the window and twiddles area, for the [RsdkBbe32FftInit\(\)](#) wrapper.
- struct [rsdkBbe32Fft2RealParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32HistParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32ReordTile16to4ChParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32CmprFixOneBufParams_t](#)
The data types accepted by the BBE32 algos.
- struct [rsdkBbe32CmprFixTwoBuf4Params_t](#)
- struct [rsdkBbe32CmprFixTwoBuf8Params_t](#)
- struct [rsdkBbe32CmprFlxOneBufParams_t](#)
- struct [rsdkBbe32CmprFlxTwoBuf4Params_t](#)
- struct [rsdkBbe32CmprFlxTwoBuf8Params_t](#)
- struct [rsdkBbe32CmprSplInitParams_t](#)
- struct [rsdkBbe32CmprSptParams_t](#)
- struct [rsdkBbe32CmprEncTxpsParams_t](#)
- struct [rsdkBbe32CmprDecTxpsParams_t](#)
- struct [rsdkBbe32TdImParams_t](#)

3.10.1 Detailed Description

DSP Radar Algo Library Data Types.

.

3.11 DSP Host Driver API

DSP Host Driver API.

Modules

- [DSP Host Driver Constants](#)
DSP Host Driver Constants .
- [DSP Host Driver Data Types](#)
DSP Host Driver Data Types .
- [DSP Host Driver Functions](#)
DSP Host Driver Functions .

3.11.1 Detailed Description

DSP Host Driver API.

Describes the functions contained in the DSP Host Driver Library.

3.12 DSP Host Driver Constants

DSP Host Driver Constants .

Macros

- #define `DSPHD_SIZEOF_MSG` 12
- #define `DSPHD_SIZEOF_JOB_LIST` ((16u * 10u) + 2u)

3.12.1 Detailed Description

DSP Host Driver Constants .

3.12.2 Macro Definition Documentation

3.12.2.1 DSPHD_SIZEOF_JOB_LIST

```
#define DSPHD_SIZEOF_JOB_LIST ((16u * 10u) + 2u)
```

Size of the job list buffer(s) that must be allocated by the user.

3.12.2.2 DSPHD_SIZEOF_MSG

```
#define DSPHD_SIZEOF_MSG 12
```

Size of the DSP message buffer that must be allocated by the user.

3.13 DSP Host Driver Data Types

DSP Host Driver Data Types .

Data Structures

- struct [Dsphd_JobType](#)
This structure packs the information needed to call a function remotely on the BBE32 DSP, through the DSP Dispatcher .
- struct [Dsphd_TraceCfgType](#)
This structure packs the information needed to configure and start application trace logging in the DSP Dispatcher .
- struct [Dsphd_RuntimeCfgType](#)
This structure packs the information needed to configure the runtime configuration of the DSP Dispatcher .
- struct [Dsphd_CommandAckType](#)
This structure packs the information needed to send an ACK from the DSP Dispatcher to the core .

Enumerations

- enum [Dsphd_AckStatusType](#) { [DSPHD_ACK_WAITING](#) , [DSPHD_ACK_RECEIVED](#) , [DSPHD_ACK_OUT_OF_ORDER](#) , [DSPHD_ACK_TIMEOUT](#) }
This structure packs the information needed for ACK status from DSP Dispatcher.
- enum [Dsphd_DispatcherStateType](#) { [DSPHD_DISP_STATE_INIT](#) = 424242u , [DSPHD_DISP_STATE_RUNNING](#) , [DSPHD_DISP_STATE_ERROR](#) , [DSPHD_DISP_STATE_IDLE](#) }
Defines the internal state of the DSP Dispatcher.
- enum [Dsphd_p1TaskConfigType](#) { [DSPHD_CFG_P1_TASK_PREEMPT](#) , [DSPHD_CFG_P1_TASK_NON_PREEMPT](#) , [DSPHD_CFG_P1_TASK_EXTEND_ENUM_TO_32_BITS](#) = 0x7FFFFFFFu }
Defines the types of messages that P1 Task can be configured to.
- enum [Dsphd_fpuRoundConfigType](#) { [DSPHD_CFG_FPU_ROUND_TO_NEAREST](#) , [DSPHD_CFG_FPU_ROUND_TRUNC](#) , [DSPHD_CFG_FPU_ROUND_CEIL](#) , [DSPHD_CFG_FPU_ROUND_FLOOR](#) , [DSPHD_CFG_FPU_EXTEND_ENUM_TO_32_BITS](#) = 0x7FFFFFFFu }
Defines the types of messages that correspond to FPU Rounding.
- enum [Dsphd_MsgTypeType](#) { [DSPHD_MSG_RUN_ASYNC_JOB](#) = 424242u , [DSPHD_MSG_UPDATE_CE_JOB_LIST](#) , [DSPHD_MSG_UPDATE_RC_JOB_LIST](#) , [DSPHD_MSG_UPDATE_LONG_JOB_LIST](#) , [DSPHD_MSG_CLEAR_CE_JOB_LIST](#) , [DSPHD_MSG_CLEAR_RC_JOB_LIST](#) , [DSPHD_MSG_CLEAR_LONG_JOB_LIST](#) , [DSPHD_MSG_UPDATE_DISPATCHER_CONFIG](#) , [DSPHD_MSG_CHECK_DISPATCHER_ALIVE](#) , [DSPHD_MSG_DSP_TRACE_START](#) , [DSPHD_MSG_DSP_TRACE_END](#) , [DSPHD_MSG_LAST](#) , [DSPHD_MSG_EXTEND_ENUM_TO_32_BITS](#) = 0x7FFFFFFFu }
Defines the types of messages that can be sent from the host CPU application to the DSP Dispatcher.
- enum [Dsphd_SignalsType](#) { [DSPHD_SIG_ASYNC_DONE](#) = 0x7C0DE001 , [DSPHD_SIG_ASYNC_DONE_EXTEND_ENUM_TO_32_BITS](#) = 0x7FFFFFFFu }
Defines the signals that can be used for the 2nd IPCF channel for different events (ASYNC_DONE as an example)

3.13.1 Detailed Description

DSP Host Driver Data Types .

3.13.2 Enumeration Type Documentation

3.13.2.1 Dsphd_AckStatusType

enum [Dsphd_AckStatusType](#)

This structure packs the information needed for ACK status from DSP Dispatcher.

Enumerator

DSPHD_ACK_WAITING	Waiting for ACK
DSPHD_ACK_RECEIVED	ACK has been received from the DSP Dispatcher before timeout
DSPHD_ACK_OUT_OF_ORDER	ACK is not for the command we sent. This should never happen. If it does something is seriously wrong.
DSPHD_ACK_TIMEOUT	ACK has not been received in the expected time from DSP Dispatcher

3.13.2.2 Dsphd_DispatcherStateType

enum [Dsphd_DispatcherStateType](#)

Defines the internal state of the DSP Dispatcher.

Can be used to detect Dispatcher "idle" or "error" states

Enumerator

DSPHD_DISP_STATE_INIT	Dispatcher initialized
DSPHD_DISP_STATE_RUNNING	Dispatcher running
DSPHD_DISP_STATE_ERROR	Dispatcher has reported unrecoverable error and halted
DSPHD_DISP_STATE_IDLE	Dispatcher idle. DSP is in low power wait state.

3.13.2.3 Dsphd_fpuRoundConfigType

enum [Dsphd_fpuRoundConfigType](#)

Defines the types of messages that correspond to FPU Rounding.

Enumerator

DSPHD_CFG_FPU_ROUND_TO_NEAREST	FPU Round to nearest
DSPHD_CFG_FPU_ROUND_TRUNC	FPU Round towards 0
DSPHD_CFG_FPU_ROUND_CEIL	FPU Round towards +Infinity
DSPHD_CFG_FPU_ROUND_FLOOR	FPU Round towards -Infinity
DSPHD_CFG_FPU_EXTEND_ENUM_TO_32_BITS	

3.13.2.4 Dsphd_MsgTypeType

enum [Dsphd_MsgTypeType](#)

Defines the types of messages that can be sent from the host CPU application to the DSP Dispatcher.

Enum values are starting from a large random number instead of 0, to avoid false interpretation of uninitialized memory

Enumerator

DSPHD_MSG_RUN_ASYNC_JOB	Command to execute an asynchronous job (function) immediately as the message is received
DSPHD_MSG_UPDATE_CE_JOB_LIST	Command to update the list of jobs that are executed periodically on each chirp end (jobs that are triggered by the CSI2 LINE_DONE interrupt)
DSPHD_MSG_UPDATE_RC_JOB_LIST	Command to update the list of jobs that are executed periodically on each radar cycle (jobs that are triggered by the CSI2 VSYNC interrupt)
DSPHD_MSG_UPDATE_LONG_JOB_LIST	Command to update the list of jobs that are executed periodically with lower priority than the ones defined by DSPHD_MSG_UPDATE_RC_JOB_LIST (e.g. self test, measuring core load etc)
DSPHD_MSG_CLEAR_CE_JOB_LIST	Command to clear the list of jobs that are executed periodically on each chirp end (jobs that are triggered by the CSI2 LINE_DONE interrupt)
DSPHD_MSG_CLEAR_RC_JOB_LIST	Command to clear the list of jobs that are executed periodically on each radar cycle (jobs that are triggered by the CSI2 VSYNC interrupt)
DSPHD_MSG_CLEAR_LONG_JOB_LIST	Command to clear the list of jobs that are executed periodically with lower priority than the ones defined by DSPHD_MSG_UPDATE_RC_JOB_LIST (e.g. self test, measuring core load etc)
DSPHD_MSG_UPDATE_DISPATCHER_CONFIG	Command that configures the dsp dispatcher parameters at runtime.
DSPHD_MSG_CHECK_DISPATCHER_ALIVE	Command that "pings" the Dispatcher and waits for an ACK to verify the Dispatcher is not stuck.
DSPHD_MSG_DSP_TRACE_START	Command to start logging events to the DSP Dispatcher trace log.
DSPHD_MSG_DSP_TRACE_END	Command to stop logging events to the DSP Dispatcher trace log.
DSPHD_MSG_LAST	
DSPHD_MSG_EXTEND_ENUM_TO_32_BITS	

3.13.2.5 Dsphd_p1TaskConfigType

```
enum Dsphd_p1TaskConfigType
```

Defines the types of messages that P1 Task can be configured to.

Enumerator

DSPHD_CFG_P1_TASK_PREEMPT	Preempt P1 Task behavior and dispatcher operation. All interrupts enabled P1Task Preemptive with highest priority. P1 Task sleeps after every chirp and other tasks can run in the meantime.
DSPHD_CFG_P1_TASK_NON_PREEMPT	Non-Preempt P1 Task behavior and dispatcher operation. All interrupts related to other tasks are disabled except RFS until numberOfChirps are processed. Chirp End interrupt waited via polling. Dispatcher never leaves P1 Task for the duration of expected chirps
DSPHD_CFG_P1_TASK_EXTEND_ENUM_TO_↔ 32_BITS	

3.13.2.6 Dsphd_SignalsType

enum [Dsphd_SignalsType](#)

Defines the signals that can be used for the 2nd IPCF channel for different events (ASYNC_DONE as an example)

Enumerator

DSPHD_SIG_ASYNC_DONE	Signal ASYNC (P3) task kernel done
DSPHD_SIG_ASYNC_DONE_EXTEND_ENUM_TO_32_BITS	

3.14 DSP Host Driver Functions

DSP Host Driver Functions .

Functions

- Std_ReturnType [Dsphd_CreateJobList](#) ([Dsphd_JobType](#) *jobPtr, uint8 *jobPtrListBuffer, uint16 numJobs, uint8_t isCrcEnabled)
Creates a list of jobs formatted for the DSP Dispatcher.
- Std_ReturnType [Dsphd_SendMsg](#) ([Dsphd_MsgTypeType](#) msgType, uint32 msgPayload)
Sends a message to the BBE32 DSP Dispatcher .
- Std_ReturnType [Dsphd_Init](#) (void)
Initializes the DSP Host driver .

3.14.1 Detailed Description

DSP Host Driver Functions .

3.14.2 Function Documentation

3.14.2.1 Dsphd_CreateJobList()

```
Std_ReturnType Dsphd_CreateJobList (
    Dsphd_JobType * jobPtr,
    uint8 * jobPtrListBuffer,
    uint16 numJobs,
    uint8_t isCrcEnabled )
```

Creates a list of jobs formatted for the DSP Dispatcher.

It reads the job information from a user-defined list, copies it to a shared buffer and computes CRC for each entry.

Parameters

in	<i>jobPtr</i>	List of jobs to be sent to the Dispatcher
in	<i>jobPtrListBuffer</i>	Pointer to a buffer of size DSPHD_SIZEOF_JOB_LIST that must be allocated by the user in a shared memory area, which is accessible to the BBE32 DSP.
in	<i>numJob</i>	Number of jobs in the user-defined list
in	<i>isCrcEnabled</i>	CRC check on/off for the jobs (0/other values but 1 = OFF, 1 = ON)

Returns

#RSDK_SUCCESS or a specific error code.

Precondition

It should be called whenever the user wishes to change the list of periodic jobs running on the DSP. Calling this function alone does not change the DSP function schedule, it must be followed by another call to [Dsphd_SendMsg\(\)](#), using either the [DSPHD_MSG_UPDATE_RC_JOB_LIST](#) or [DSPHD_MSG_UPDATE_LONG_JOB_LIST](#) message types.

3.14.2.2 Dsphd_Init()

```
Std_ReturnType Dsphd_Init (
    void )
```

Initializes the DSP Host driver .

Initializes the IPCF communication with the BBE32 DSP Dispatcher.

Returns

#RSDK_SUCCESS or a specific error code.

3.14.2.3 Dsphd_SendMsg()

```
Std_ReturnType Dsphd_SendMsg (
    Dsphd_MsgTypeType msgType,
    uint32 msgPayload )
```

Sends a message to the BBE32 DSP Dispatcher .

The message follows a simple "type+payload" protocol

Parameters

in	msgType	type of message
in	msgPayload	32-bit information field which can contain a scalar or 32-bit pointer usable by the BBE32 DSP

Returns

#RSDK_SUCCESS or a specific error code.

Precondition

For [DSPHD_MSG_UPDATE_RC_JOB_LIST](#) or [DSPHD_MSG_UPDATE_LONG_JOB_LIST](#) message types, the msgPayload must be a pointer to the pJobListBuffer initialized by [Dsphd_CreateJobList\(\)](#)

3.15 DSP LAL API

DSP Linear Algebra Library API.

Modules

- [DSP LAL Demo Data](#)
DSP LAL BBE32EP Demo Data .
- [DSP LAL Constants](#)
DSP LAL BBE32EP Constants .
- [DSP LAL Functions](#)
DSP LAL BBE32EP Functions .

3.15.1 Detailed Description

DSP Linear Algebra Library API.

Describes the functions contained in the DSP BBE32 LAL Library. DSP LAL library is a floating-point library running in batch mode (8 parallel streaming data), see DSP Linear Algebra Library User Guide.

3.16 DSP LAL Demo Data

DSP LAL BBE32EP Demo Data .

Data Structures

- struct [rsdkBbe32LalVectorAddParams_t](#)

3.16.1 Detailed Description

DSP LAL BBE32EP Demo Data .

3.17 DSP LAL Constants

DSP LAL BBE32EP Constants .

Modules

- [DSP LAL Demo Constants](#)

DSP LAL BBE32EP Demo Constants .

Macros

- #define [OPTIMIZE_0](#) __attribute__((optimize("-O0")))

*Optimization options. Used to modify default project code optimization level.
Can be added after function prototype.*
- #define [OPTIMIZE_1](#) __attribute__((optimize("-O1")))
- #define [OPTIMIZE_2](#) __attribute__((optimize("-O2")))
- #define [OPTIMIZE_3](#) __attribute__((optimize("-O3")))
- #define [OPTIMIZE_S](#) __attribute__((optimize("-Os")))
- #define [BBE32_ALWAYS_INLINE](#) __attribute__((always_inline))
- #define [BBE32_NO_INLINE](#) __attribute__((noinline))
- #define [SP_pi](#) ((float32_t)(3.14159265358979323846F)) /* PI */
- #define [SP_pi_over_2](#) ((float32_t)(1.57079632679489661923F)) /* PI/2 */
- #define [EXP_MASK](#) (BBE_MOVVA16C(0x7F800000)) /* Infinity IEEE754 == the exponent is 111...1 */
- #define [MANTISSA_MASK](#) (BBE_MOVVA16C(0x007FFFFF)) /* first subnormal IEEE754 == the mask is 000...0 and mantissa is 111...1 */
- #define [INFINITY_SP](#) ((float32_t)(1e+300 * 1e+300))
- #define [NAN_SP](#) (INFINITY_SP * 0.0F)
- #define [RSDK_MAG2N_2XF32_V1](#)(__out, __inl1, __inl2, __rtmp, __itmp)

A macro that returns __out the absolute square, square modulus, square magnitude, square norm vector of two input registers of __inl1 and __inl2.
- #define [RSDK_MAG2N_2XF32_V2](#)(__out, __inl1, __inl2, __rtmp, __itmp)

A macro that returns __out the absolute square, square modulus, square magnitude, square norm vector of two input registers of __inl1 and __inl2.

3.17.1 Detailed Description

DSP LAL BBE32EP Constants .

3.17.2 Macro Definition Documentation

3.17.2.1 BBE32_ALWAYS_INLINE

```
#define BBE32_ALWAYS_INLINE __attribute__((always_inline))
```

3.17.2.2 BBE32_NO_INLINE

```
#define BBE32_NO_INLINE __attribute__((noinline))
```

3.17.2.3 EXP_MASK

```
#define EXP_MASK (BBE_MOVVA16C(0x7F800000)) /* Infinity IEEE754 == the exponent is 111...1 */
```

3.17.2.4 INFINITY_SP

```
#define INFINITY_SP ((float32_t)(1e+300 * 1e+300))
```

3.17.2.5 MANTISSA_MASK

```
#define MANTISSA_MASK (BBE_MOVVA16C(0x007FFFFF)) /* first subnormal IEEE754 == the mask is  
000...0 and mantissa is 111...1 */
```

3.17.2.6 NAN_SP

```
#define NAN_SP (INFINITY_SP * 0.0F)
```

3.17.2.7 OPTIMIZE_0

```
#define OPTIMIZE_0 __attribute__((optimize("-O0")))
```

Optimization options. Used to modify default project code optimization level.
Can be added after function prototype.

3.17.2.8 OPTIMIZE_1

```
#define OPTIMIZE_1 __attribute__((optimize("-O1")))
```

3.17.2.9 OPTIMIZE_2

```
#define OPTIMIZE_2 __attribute__((optimize("-O2")))
```

3.17.2.10 OPTIMIZE_3

```
#define OPTIMIZE_3 __attribute__((optimize("-O3")))
```

3.17.2.11 OPTIMIZE_S

```
#define OPTIMIZE_S __attribute__((optimize("-Os")))
```

3.17.2.12 RSDK_MAG2N_2XF32_V1

```
#define RSDK_MAG2N_2XF32_V1(  
    __out,  
    __inl1,  
    __inl2,  
    __rtmp,  
    __itmp )
```

Value:

```
{  
    BBE_DSELN_2XF32I((__itmp), (__rtmp), (__inl2), (__inl1), BBE_DSELI_DEINTERLEAVE_2);  
    (__out) = BBE_MULN_2XF32((__rtmp), (__rtmp));  
    BBE_MULAN_2XF32((__out), (__itmp), (__itmp));  
}
```

A macro that returns `__out` the absolute square, square modulus, square magnitude, square norm vector of two input registers of `__inl1` and `__inl2`.

Parameters

<code>__out</code>	Output register <code>xb_vecN_2xf32</code> .
<code>__in1</code>	Input first register <code>xb_vecN_2xf32</code> .
<code>__in2</code>	Input second register <code>xb_vecN_2xf32</code> .
<code>__rtmp</code>	Scratch register output real part <code>xb_vecN_2xf32</code> (not initialized).
<code>__itmp</code>	Scratch register output imaginary part <code>xb_vecN_2xf32</code> (not initialized).

3.17.2.13 RSDK_MAG2N_2XF32_V2

```
#define RSDK_MAG2N_2XF32_V2(
```

```
    __out,  
    __in1,  
    __in2,  
    __rtmp,  
    __itmp )
```

Value:

```
{  
    (__in1) = BBE_MULN_2XF32((__in1), (__in1));  
    (__in2) = BBE_MULN_2XF32((__in2), (__in2));  
    BBE_DSELN_2XF32I((__itmp), (__rtmp), (__in2), (__in1), BBE_DSELI_DEINTERLEAVE_2);  
    (__out) = BBE_ADDN_2XF32((__rtmp), (__itmp));  
}
```

A macro that returns `__out` the absolute square, square modulus, square magnitude, square norm vector of two input registers of `__in1` and `__in2`.

Parameters

<code>__out</code>	Output register <code>xb_vecN_2xf32</code> .
<code>__in1</code>	Input first register <code>xb_vecN_2xf32</code> .
<code>__in2</code>	Input second register <code>xb_vecN_2xf32</code> .
<code>__rtmp</code>	Scratch register output real part <code>xb_vecN_2xf32</code> (not initialized).
<code>__itmp</code>	Scratch register output imaginary part <code>xb_vecN_2xf32</code> (not initialized).

3.17.2.14 SP_pi

```
#define SP_pi ((float32_t)(3.14159265358979323846F)) /* PI */
```

3.17.2.15 SP_pi_over_2

```
#define SP_pi_over_2 ((float32_t)(1.57079632679489661923F)) /* PI/2 */
```

3.18 DSP LAL Demo Constants

DSP LAL BBE32EP Demo Constants .

Macros

- `#define BBE_BATCH_WIDTH (8U)`
- `#define SIZE_VEC (6U)`

3.18.1 Detailed Description

DSP LAL BBE32EP Demo Constants .

3.18.2 Macro Definition Documentation

3.18.2.1 BBE_BATCH_WIDTH

```
#define BBE_BATCH_WIDTH (8U)
```

3.18.2.2 SIZE_VEC

```
#define SIZE_VEC (6U)
```

3.19 DSP LAL Functions

DSP LAL BBE32EP Functions .

Modules

- [DSP LAL Demo Functions](#)
DSP LAL BBE32EP Demo Functions .

3.19.1 Detailed Description

DSP LAL BBE32EP Functions .

3.20 DSP LAL Demo Functions

DSP LAL BBE32EP Demo Functions .

DSP LAL BBE32EP Demo Functions .

3.21 Dsp_lal_api_func_add_vec

Functions

- void [Rsdk_LA_add_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Add vectors - real single precision.
- void [Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Add vectors - real single precision.
- void [Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Add vectors - complex single precision.
- void [Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2](#) (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Add vectors - complex single precision.

3.21.1 Detailed Description

3.21.2 Function Documentation

3.21.2.1 RsdK_LA_add_Vof32cp8Vif32cp8Vif32cp8()

```
void RsdK_LA_add_Vof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Add vectors - complex single precision.

Function implements add operation for 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

add(vector1,vector2) = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..1365]

3.21.2.2 Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2()

```
void Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Add vectors - complex single precision.

Function implements add operation for 2 complex data vectors.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

add(vector1,vector2) = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..1365]

3.21.2.3 Rsdk_LA_add_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_add_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Add vectors - real single precision.

Function implements add operation for 2 real data vectors.

Input and output can be in-place memory vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

add(vector1,vector2) = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.21.2.4 Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2()

```
void Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Add vectors - real single precision.

Function implements add operation for 2 real data vectors.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

add(vector1,vector2) = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.22 Dsp_lal_api_func_sub_vec**Functions**

- void [Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector subtraction - real single precision.
- void [Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector subtraction - real single precision.
- void [Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector subtraction - complex single precision.
- void [Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector subtraction - complex single precision.

3.22.1 Detailed Description

3.22.2 Function Documentation

3.22.2.1 Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8()

```
void Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Vector subtraction - complex single precision.

Function performs subtraction of 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

vector = vector1 - vector2

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..1365]

3.22.2.2 Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2()

```
void Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Vector subtraction - complex single precision.

Function performs subtraction of 2 complex data vectors.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{vector} = \text{vector1} - \text{vector2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..1365]

3.22.2.3 Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Vector subtraction - real single precision.

Function performs subtraction of 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{vector} = \text{vector1} - \text{vector2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.22.2.4 Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2()

```
void Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Vector subtraction - real single precision.

Function performs subtraction of 2 real data vectors.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

vector = vector1 - vector2

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.23 Dsp_lal_api_func_mul_vec_mat

Functions

- void [Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Multiply vectors - real single precision.
- void [Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Multiply vectors - real single precision.
- void [Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Multiply vectors - complex single precision.
- void [Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2](#) (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t in1h, uint32_t in1w, uint32_t in2w)
Multiply vectors - complex single precision.
- void [Rsdk_LA_mul_Mof32p8Mif32p8Mif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t in1h, uint32_t in1w, uint32_t in2w)
Multiply matrices - real single precision.
- void [Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8](#) (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t in1h, uint32_t in1w, uint32_t in2w)
Multiply matrices - complex single precision.
- void [Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t in1h, uint32_t in1w, uint32_t in2w)
Multiply 8 parallel streams of NxM matrices by transposed of 8 parallel streams of NxQ matrices - real single precision.
- void [Rsdk_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t in1h, uint32_t in1w, uint32_t in2w)
Multiply 8 parallel streams of NxM matrices by the transposed of the complex conjugate of 8 parallel streams of NxQ matrices - complex single precision.

3.23.1 Detailed Description

3.23.2 Function Documentation

3.23.2.1 Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8()

```
void Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t in1h,
    uint32_t in1w,
    uint32_t in2w )
```

Multiply matrices - complex single precision.

Function implements multiply operation for 2 complex data matrices.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxQ.
in	<i>pInput1</i>	First operand contains 8 parallel complex matrices, size NxM.
in	<i>pInput2</i>	First operand contains 8 parallel complex matrices, size MxQ.
in	<i>in1h</i>	Rows of input matrix pInput1 and rows of output matrix pOutput, N.
in	<i>in1w</i>	Columns of input matrix pInput1 and rows of input matrix pInput2, M.
in	<i>in2w</i>	Columns of output matrix, Q.

Note

mul(matrix1,matrix2) = matrix

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

in1h range is [2..N]

in1w range is [2..M]

in2w range is [2..Q]

$(N * Q + N * M + M * Q) \leq 4095$

3.23.2.2 RsdK_LA_mul_Mof32p8Mif32p8Mif32p8()

```
void RsdK_LA_mul_Mof32p8Mif32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t in1h,
    uint32_t in1w,
    uint32_t in2w )
```

Multiply matrices - real single precision.

Function implements multiply operation for 2 real data matrices.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxQ.
in	<i>pInput1</i>	First operand contains 8 parallel real matrices, size NxM.
in	<i>pInput2</i>	First operand contains 8 parallel real matrices, size MxQ.
in	<i>in1h</i>	Rows of input matrix pInput1 and rows of output matrix pOutput, N.
in	<i>in1w</i>	Columns of input matrix pInput2 and rows of input matrix pInput2, M.
in	<i>in2w</i>	Columns of output matrix, Q.

Note

mul(matrix1,matrix2) = matrix

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

in1h range is [2..N]
 in1w range is [2..M]
 in2w range is [2..Q]
 $(N * Q + N * M + M * Q) \leq 8191$

3.23.2.3 RsdK_LA_mul_Vof32cp8Vif32cp8Vif32cp8()

```
void RsdK_LA_mul_Vof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Multiply vectors - complex single precision.

Function implements multiply operation for 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{mul}(\text{vector1}, \text{vector2}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
 size range is [1..1365]

3.23.2.4 RsdK_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2()

```
void RsdK_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Multiply vectors - complex single precision.

Function implements multiply operation for 2 complex data vectors.

Function only works for N modulo 4 = 0.

The implementation uses loop unrolling.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{mul}(\text{vector1}, \text{vector2}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..1365]

3.23.2.5 Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Multiply vectors - real single precision.

Function implements multiply operation for 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{mul}(\text{vector1}, \text{vector2}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.23.2.6 Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2()

```
void Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2 (
    xb_vecN_2xf32 * __restrict pOutput,
    xb_vecN_2xf32 * pInput1,
```

```
xb_vecN_2xf32 * pInput2,
uint32_t size )
```

Multiply vectors - real single precision.

Function implements multiply operation for 2 real data vectors.

Function only works for N modulo 4 = 0.

The implementation uses loop unrolling.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{mul}(\text{vector1}, \text{vector2}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..2730]

3.23.2.7 RsdK_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8()

```
void RsdK_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t in1h,
    uint32_t in1w,
    uint32_t in2w )
```

Multiply 8 parallel streams of NxM matrices by the transposed of the complex conjugate of 8 parallel streams of NxQ matrices - complex single precision.

All matrices from second input parallel stream are transposed and complex conjugate,

then each matrix from first input parallel stream is multiplied by corresponding matrix in second parallel stream.

Streams are independent.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxQ.
in	<i>pInput1</i>	First operand contains 8 parallel complex matrices, size NxM.
in	<i>pInput2</i>	Second operand contains 8 parallel complex matrices, size QxM.
in	<i>in1h</i>	Rows of the input matrix pInput1 and rows of the output matrix pOutput, N.
in	<i>in1w</i>	Columns of the input matrix pInput1 and columns of the transposed input matrix pInput2, M.
in	<i>in2w</i>	Columns of the output matrix and rows of the transposed input matrix pInput2, Q.

Note

$\text{mul}(\text{matrix1}, \text{conj}(\text{matrix2}^T)) = \text{matrix}$

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

in1h range is $[2..N]$

in1w range is $[2..M]$

in2w range is $[2..Q]$

$(N * Q + N * M + M * Q) \leq 4095$

3.23.2.8 Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8()

```
void Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t in1h,
    uint32_t in1w,
    uint32_t in2w )
```

Multiply 8 parallel streams of NxM matrices by transposed of 8 parallel streams of NxQ matrices - real single precision.

All matrices from second input parallel stream are transposed, then each matrix from first input parallel stream is multiplied by corresponding matrix in second parallel stream. Streams are independent. Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxQ.
in	<i>pInput1</i>	First operand contains 8 parallel real matrices, size NxM.
in	<i>pInput2</i>	Second operand contains 8 parallel real matrices, size QxM.
in	<i>in1h</i>	Rows of the input matrix pInput1 and rows of the output matrix pOutput, N.
in	<i>in1w</i>	Columns of the input matrix pInput1 and columns of the transposed input matrix pInput2, M.
in	<i>in2w</i>	Columns of the output matrix and rows of the transposed input matrix pInput2, Q.

Note

$\text{mul}(\text{matrix1}, \text{matrix2}^T) = \text{matrix}$

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

in1h range is [2..N]

in1w range is [2..M]

in2w range is [2..Q]

$(N * Q + N * M + M * Q) \leq 8191$

3.24 Dsp_lal_api_func_div_vec

Functions

- void [Rsdk_LA_div_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector division - real single precision.
- void [Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Vector division - real single precision. The fastest version.
- void [Rsdk_LA_div_Vof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Vector division - complex single precision.

3.24.1 Detailed Description

3.24.2 Function Documentation

3.24.2.1 RsdK_LA_div_Vof32cp8Vif32cp8Vif32cp8()

```
void RsdK_LA_div_Vof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Vector division - complex single precision.

Function performs division of 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{div}(\text{vector1}, \text{vector2}) = \text{vector1} ./ \text{vector2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..1365]

3.24.2.2 Rsdk_LA_div_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_div_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Vector division - real single precision.

Function performs division of 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{div}(\text{vector1}, \text{vector2}) = \text{vector1} ./ \text{vector2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.24.2.3 Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2()

```
void Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Vector division - real single precision. The fastest version.

Function performs division of 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{div}(\text{vector1}, \text{vector2}) = \text{vector1} * \text{rcp}(\text{vector2})$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.25 Dsp_lal_api_func_xcorr_vec

Functions

- void [Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size1, uint32_t size2)
Cross correlation on vectors - real single precision.
- void [Rsdk_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *restrict pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size1, uint32_t size2)
Cross correlation on vectors - complex single precision.

3.25.1 Detailed Description

3.25.2 Function Documentation

3.25.2.1 RsdK_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8()

```
void RsdK_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 *restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size1,
    uint32_t size2 )
```

Cross correlation on vectors - complex single precision.

Function implements cross correlation operation 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size $2 * \max(M, N) - 1$.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size M.
in	<i>size1</i>	Size of input and output vectors, N.
in	<i>size2</i>	Size of input and output vectors, M.

Note

$\text{xcorr}(\text{vector1}, \text{vector2}) = \text{vector}$

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256

KBytes.
size1 range is [1..N]
size2 range is [1..M]
 $(N + M + 2 * \max(M,N)) < 4096$

3.25.2.2 Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size1,
    uint32_t size2 )
```

Cross correlation on vectors - real single precision.

Function implements cross correlation operation for 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size $2 * \max(M,N) - 1$.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size M.
in	<i>size1</i>	Size of input and output vectors, N.
in	<i>size2</i>	Size of input and output vectors, M.

Note

$\text{xcorr}(\text{vector1}, \text{vector2}) = \text{vector}$
The following sizes /are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size1 range is [1..N]
size2 range is [1..M]
 $(N + M + 2 * \max(M,N)) < 8192$

3.26 Dsp_lal_api_func_op_vec_mat

Functions

- void [Rsdk_EF_mod_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Modulo - real single precision.
- void [Rsdk_EF_sqrtAbs_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Root square - real single precision.
- void [Rsdk_EF_sqrt_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Root square - real single precision.

- void [RsdK_EF_sqrt_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Root square - complex single precision.

- void [RsdK_LA_hypot_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)

Hypotenuse - real single precision.

- void [RsdK_LA_rcp_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Reciprocal - real single precision.

- void [RsdK_LA_rcp_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Reciprocal - complex single precision.

- void [RsdK_LA_threshAbs_Vof32p8Vif32cp8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput1, xb_vecN_2xf32 const *pInput2, uint32_t size)

*Threshold using a real vector to a complex vector
absolute values - complex single precision.*

- void [RsdK_LA_threshAbs_Vof32p8Vif32cp8Sif32](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput1, float32_t const *pInput2, uint32_t size)

*Threshold using a real scalar to a complex vector
absolute values - complex single precision.*

- void [RsdK_LA_thresh_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput1, xb_vecN_2xf32 const *pInput2, uint32_t size)

*Threshold using a real threshold vector to a real vector
values - real single precision.*

- void [RsdK_LA_thresh_Vof32p8Vif32p8Sif32](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput1, float32_t const *pInput2, uint32_t size)

*Threshold using a real scalar to a real vector
values - real single precision.*

- void [RsdK_EF_exp_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Exponential - real single precision.

3.26.1 Detailed Description

3.26.2 Function Documentation

3.26.2.1 RsdK_EF_exp_Vof32p8Vif32p8()

```
void RsdK_EF_exp_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Exponential - real single precision.

Function calculates the exponential of a real data vectors.

The exponential values are computed based on the identity $e^x = 2^y$, where $y = (x / \ln(2))$, which can be expanded to $e^x = 2^{(int)y} * 2^{(fractional)y}$.

2^y will be approximated using Taylor series, implemented using Horner's method to reduce the number of operations.

The Taylor series approximation is expanded around the value 0.5f.

For input values $< 8.5f$, the function produces an approximation error $< 1e-3$.

For input values $> 8.5f$, the approximation error increases constantly with the increasing of the input values.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, representing the exponential values, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\exp(\text{vector}) = \text{vector}$

$v = (a_1, a_2, \dots, a_n)$;

$\exp(v) = (e^{a_1}, e^{a_2}, \dots, e^{a_n})$;

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.26.2.2 RsdK_EF_mod_Vof32p8Vif32p8Vif32p8()

```
void RsdK_EF_mod_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Modulo - real single precision.

Function implements the modulo operation between 2 real data vectors.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{mod}(\text{vector1}, \text{vector2}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..2730]

3.26.2.3 RsdK_EF_sqrt_Vof32cp8Vif32cp8()

```
void RsdK_EF_sqrt_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Root square - complex single precision.

Function calculates the square root of a complex data vectors.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{csqrt}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2047]

3.26.2.4 RsdK_EF_sqrt_Vof32p8Vif32p8()

```
void RsdK_EF_sqrt_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Root square - real single precision.

Function calculates the square root of a real data vectors.
Values must be real positives.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{sqrt}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.26.2.5 Rsdk_EF_sqrtAbs_Vof32p8Vif32p8()

```
void Rsdk_EF_sqrtAbs_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Root square - real single precision.

Function calculates the square root of a real data vectors.
The square root of the absolute input value is computed.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{sqrt}(\text{abs}(\text{vector})) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.26.2.6 Rsdk_LA_hypot_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_hypot_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Hypotenuse - real single precision.

Function implements the hypotenuse operation between 2 real data vectors.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{hypotenuse}(\text{vector1}, \text{vector2}) = \text{sqrt}(a*a + b*b) = \text{vector}$

If one of the input is +/-INF and the other one is NaN, the output is +INF (IEEE-754 compliant) The following

size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.26.2.7 Rsdk_LA_rcp_Vof32cp8Vif32cp8()

```
void Rsdk_LA_rcp_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Reciprocal - complex single precision.

Function calculates the reciprocal of a complex data vectors.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{rcp}(\text{vector}) = 1./\text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2047]

3.26.2.8 Rsdk_LA_rcp_Vof32p8Vif32p8()

```
void Rsdk_LA_rcp_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Reciprocal - real single precision.

Function calculates the reciprocal of a real data vectors.
Input and output must not be in-place memory vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{rcp}(\text{vector}) = 1./\text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.26.2.9 Rsdk_LA_thresh_Vof32p8Vif32p8Sif32()

```
void Rsdk_LA_thresh_Vof32p8Vif32p8Sif32 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 const * pInput1,
    float32_t const * pInput2,
    uint32_t size )
```

Threshold using a real scalar to a real vector
values - real single precision.

Function perform the threshold of the real input vector
against the real scalar threshold.

Each element of the output vector is either 0 (input element < threshold)
or 1 (input element >= threshold).

One threshold is used for all 8 parallel streams.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	Operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Operand contains a pointer to the real scalar threshold.
in	<i>size</i>	Size of the input and output vectors, N.

Note

$\text{thresh}(\text{vector}, \text{scalar}) = \text{vector}$

$v = a_1, a_2, \dots a_N$

$s = \text{thr}$

$\text{absThrsh}(v, s) = (a_1 \geq \text{thr}), (a_2 \geq \text{thr}), \dots (a_N \geq \text{thr})$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.26.2.10 Rsdk_LA_thresh_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_thresh_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    uint32_t size )
```

Threshold using a real threshold vector to a real vector
values - real single precision.

Function perform the threshold of the real input vector
against the real threshold vector.
Each element of the output vector is either 0 (input element < threshold)
or 1 (input element >= threshold).
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	Operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Operand contains 8 parallel threshold real vectors, size 1.
in	<i>size</i>	Size of the input and output vectors, N.

Note

$\text{thresh}(\text{vector}, \text{vector}) = \text{vector}$
 $v1 = a1, a2, \dots aN$
 $v2 = b1, b2, \dots bN$
 $\text{thresh}(v1, v2) = (a1 \geq b1), (a2 \geq b2), \dots (an \geq bN)$
 The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
 size range is [1..4095]

3.26.2.11 RsdK_LA_threshAbs_Vof32p8Vif32cp8Sif32()

```
void RsdK_LA_threshAbs_Vof32p8Vif32cp8Sif32 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xcf32 const * pInput1,
    float32_t const * pInput2,
    uint32_t size )
```

Threshold using a real scalar to a complex vector
absolute values - complex single precision.

Function performs the threshold of the magnitude of the complex input vector
against the square of the scalar threshold.
Each element of the output vector is either 0 (input element < threshold^2)
or 1 (input element >= threshold^2).
One threshold is used for all 8 parallel streams.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	Operand contains a pointer to the real scalar threshold.
in	<i>size</i>	Size of the input and output vectors, N.

Note

$\text{absThrsh}(\text{vector}, \text{scalar}) = \text{vector}$

$v = a_1, a_2, \dots, a_N$

$s = \text{thr}$

$\text{absThrsh}(v, s) = (|a_1|^2 \geq \text{thr}^2), (|a_2|^2 \geq \text{thr}^2), \dots (|a_N|^2 \geq \text{thr}^2)$, where $|a_N|$ is the magnitude of the complex number a_N , computed as

$\sqrt{\text{real}(a_N)^2 + \text{imag}(a_N)^2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.26.2.12 Rsdk_LA_threshAbs_Vof32p8Vif32cp8Vif32p8()

```
void Rsdk_LA_threshAbs_Vof32p8Vif32cp8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xcf32 const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    uint32_t size )
```

Threshold using a real vector to a complex vector
absolute values - complex single precision.

Function performs the threshold of the magnitude of the complex input vector against the square of the threshold vector.

Each element of the output vector is either 0 (input element < threshold²) or 1 (input element ≥ threshold²).

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput1</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	Operand contains 8 parallel threshold real vectors, size 1.
in	<i>size</i>	Size of the input and output vectors, N.

Note

$\text{absThrsh}(\text{vector}, \text{vector}) = \text{vector}$

$v_1 = a_1, a_2, \dots, a_N$

$v_2 = b_1, b_2, \dots, b_N$

$\text{absThrsh}(v_1, v_2) = (|a_1|^2 \geq b_1^2), (|a_2|^2 \geq b_2^2), \dots (|a_N|^2 \geq b_N^2)$,

where $|a_N|$ is the magnitude of the complex number a_N , computed as

$\sqrt{\text{real}(a_N)^2 + \text{imag}(a_N)^2}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.27 Dsp_lal_api_func_fill**Functions**

- void [Rsdk_EF_fillZeros_Vof32p8](#) (xb_vecN_2xf32 *pOutput, uint32_t size)

- Fill with zero vectors - real single precision.*
- void [RsdK_EF_fillZeros_Vof32cp8](#) (xb_vecN_2xf32 *pOutput, uint32_t size)
- Fill with zero vectors - complex single precision.*
- void [RsdK_EF_fill_Vof32p8Sif32](#) (xb_vecN_2xf32 *pOutput, float32_t const *pInput, uint32_t size)
- Fill 8 streams of vectors, size N, with one value - real single precision.*
- void [RsdK_EF_fill_Vof32cp8Sif32c](#) (xb_vecN_2xf32 *pOutput, xtcomplexfloat const *pInput, uint32_t size)
- Fill 8 streams of vectors, size N, with one value - complex single precision.*
- void [RsdK_EF_fill_Vof32p8Sif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of vectors, size N, with one value - real single precision.*
- void [RsdK_EF_fill_Vof32cp8Sif32cp8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of vectors, size N, with one value - complex single precision.*
- void [RsdK_SP_fillStgRadix2_Vof32cp8](#) (xb_vecN_2xf32 *pOutput, uint32_t stage)
- Fill 8 parallel identical vectors with FFT radix 2 stage twiddle factors.*
- void [RsdK_EF_fillZeros_Mof32p8](#) (xb_vecN_2xf32 *pOutput, uint32_t rows, uint32_t cols)
- Fill with zero matrices - real single precision.*
- void [RsdK_EF_fillZeros_Mof32cp8](#) (xb_vecN_2xf32 *pOutput, uint32_t rows, uint32_t cols)
- Fill with zero matrices - complex single precision.*
- void [RsdK_EF_fillZerosDg_Mof32p8Sif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of NxN matrix with 0 and the diagonal elements to a scalar.*
- void [RsdK_EF_fillZerosDg_Mof32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of NxN matrix with 0 and the diagonal with a vector of values - real single precision.*
- void [RsdK_EF_fillZerosDg_Mof32cp8Sif32cp8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of NxN matrix with (0,0i) and the diagonal elements to a scalar - complex single precision.*
- void [RsdK_EF_fillZerosDg_Mof32cp8Vif32cp8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
- Fill 8 streams of NxN matrix with (0,0i) and the diagonal with a vector of values - complex single precision.*
- void [RsdK_EF_identity_Mof32p8](#) (xb_vecN_2xf32 *pOutput, uint32_t size)
- Identity matrices - real single precision.*
- void [RsdK_EF_identity_Mof32cp8](#) (xb_vecN_2xf32 *pOutput, uint32_t size)
- Identity matrices - complex single precision.*

3.27.1 Detailed Description

3.27.2 Function Documentation

3.27.2.1 RsdK_EF_fill_Vof32cp8Sif32c()

```
void RsdK_EF_fill_Vof32cp8Sif32c (
    xb_vecN_2xf32 * pOutput,
    xtcomplexfloat const * pInput,
    uint32_t size )
```

Fill 8 streams of vectors, size N, with one value - complex single precision.

Set all elements of the vectors with a complex scalar.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors size N.
in	<i>pInput</i>	Operand is a complex scalar.
in	<i>size</i>	Number of vector elements(N).

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.27.2.2 Rsdk_EF_fill_Vof32cp8Sif32cp8()

```
void Rsdk_EF_fill_Vof32cp8Sif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of vectors, size N, with one value - complex single precision.

Set all elements of the vectors with a complex scalar.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors size N.
in	<i>pInput</i>	Operand contains 8 complex scalars.
in	<i>size</i>	Number of vector elements(N).

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4094]

3.27.2.3 Rsdk_EF_fill_Vof32p8Sif32()

```
void Rsdk_EF_fill_Vof32p8Sif32 (
    xb_vecN_2xf32 * pOutput,
    float32_t const * pInput,
    uint32_t size )
```

Fill 8 streams of vectors, size N, with one value - real single precision.

Set all elements of vectors with a real scalar.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors size N.
in	<i>pInput</i>	Operand is a real scalar.
in	<i>size</i>	Number of vector elements(N).

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..8191]

3.27.2.4 Rsdk_EF_fill_Vof32p8Sif32p8()

```
void Rsdk_EF_fill_Vof32p8Sif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of vectors, size N, with one value - real single precision.

Set all elements of vectors with a real scalar.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors size N.
in	<i>pInput</i>	Operand contains 8 real scalars.
in	<i>size</i>	Number of vector elements(N).

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..8190]

3.27.2.5 Rsdk_EF_fillZeros_Mof32cp8()

```
void Rsdk_EF_fillZeros_Mof32cp8 (
    xb_vecN_2xcf32 * pOutput,
    uint32_t rows,
    uint32_t cols )
```

Fill with zero matrices - complex single precision.

Function fills matrices with zeros.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxM.
in	<i>rows</i>	Rows of output matrices, N.
in	<i>cols</i>	Columns of output matrices, M.

Note

fillZeros() = matrix

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

rows range is [1..N]

cols range is [1..M]

$(N * M) < 4096$

3.27.2.6 Rsdk_EF_fillZeros_Mof32p8()

```
void Rsdk_EF_fillZeros_Mof32p8 (
    xb_vecN_2xf32 * pOutput,
    uint32_t rows,
    uint32_t cols )
```

Fill with zero matrices - real single precision.

Function fills matrices with zeros.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxM.
in	<i>rows</i>	Rows of output matrices, N.
in	<i>cols</i>	Columns of output matrices, M.

Note

fillZeros() = matrix

The following sizes are estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

rows range is [1..N]

cols range is [1..M]

$(N * M) < 8192$

3.27.2.7 Rsdk_EF_fillZeros_Vof32cp8()

```
void Rsdk_EF_fillZeros_Vof32cp8 (
    xb_vecN_2xcf32 * pOutput,
    uint32_t size )
```

Fill with zero vectors - complex single precision.

Function fills vectors with zeros.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of output vectors, N.

Note

fillZeros() = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.27.2.8 RsdK_EF_fillZeros_Vof32p8()

```
void RsdK_EF_fillZeros_Vof32p8 (
    xb_vecN_2xf32 * pOutput,
    uint32_t size )
```

Fill with zero vectors - real single precision.

Function fills vectors with zeros.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of output vectors, N.

Note

fillZeros() = vector

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..8191]

3.27.2.9 RsdK_EF_fillZerosDg_Mof32cp8Sif32cp8()

```
void RsdK_EF_fillZerosDg_Mof32cp8Sif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of NxN matrix with (0,0i) and the diagonal elements to a scalar - complex single precision.

Set diagonals' elements to a scalar and the rest of the matrices' elements to (0,0i).

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex square matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 complex scalars.
in	<i>size</i>	Size of output matrix(N, rows, cols).

Note

fillZeros() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..63]

3.27.2.10 Rsdk_EF_fillZerosDg_Mof32cp8Vif32cp8()

```
void Rsdk_EF_fillZerosDg_Mof32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of NxN matrix with (0,0i) and the diagonal with a vector of values - complex single precision.

Set diagonals' elements to the vector and the rest of matrices' elements to (0,0i).

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex square matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of output matrices(N, rows, cols).

Note

fillZeros() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..63]

3.27.2.11 Rsdk_EF_fillZerosDg_Mof32p8Sif32p8()

```
void Rsdk_EF_fillZerosDg_Mof32p8Sif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of NxN matrix with 0 and the diagonal elements to a scalar.

- real single precision.

Set diagonals' elements to a scalar and the rest of the matrices' elements to 0.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real square matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 real scalars.
in	<i>size</i>	Size of output matrices(N, rows, cols).

Note

fillZerosDg() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [2..90]

3.27.2.12 Rsdk_EF_fillZerosDg_Mof32p8Vif32p8()

```
void Rsdk_EF_fillZerosDg_Mof32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput,
    uint32_t size )
```

Fill 8 streams of NxN matrix with 0 and the diagonal with a vector of values - real single precision.

Set diagonals' elements to the vector and the rest of matrices' elements to 0.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real square matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of output matrices(N, rows, cols).

Note

fillZeros() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [2..90]

3.27.2.13 Rsdk_EF_identity_Mof32cp8()

```
void Rsdk_EF_identity_Mof32cp8 (
    xb_vecN_2xcf32 * pOutput,
    uint32_t size )
```

Identity matrices - complex single precision.

Function creates identity matrices.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the output matrices.

Note

identity() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..63]

3.27.2.14 Rsdk_EF_identity_Mof32p8()

```
void Rsdk_EF_identity_Mof32p8 (
    xb_vecN_2xf32 * pOutput,
    uint32_t size )
```

Identity matrices - real single precision.

Function creates identity matrices.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the output matrices, N.

Note

identity() = matrix

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..90]

3.27.2.15 Rsdk_SP_fillStgRadix2_Vof32cp8()

```
void Rsdk_SP_fillStgRadix2_Vof32cp8 (
    xb_vecN_2xcf32 * pOutput,
    uint32_t stage )
```

Fill 8 parallel identical vectors with FFT radix 2 stage twiddle factors.

- complex single precision.

Result contains 8 parallel identical complex vectors, size 2 to power of S, containing roots of unity often called FFT twiddle factors corresponding to a particular stage. Implemented algorithm is from the book Approximations for Digital Computers, Cecil Hastings, p140.

Parameters

out	pOutput	Result contains 8 parallel complex vectors size 2^S .
in	stage	Stage number (S).

Note

fill twiddle = vector

$N = 2^S$

fill twiddle() = 8, $\cos(-2\pi \cdot 0 \cdot 1/N)$, $\sin(-2\pi \cdot 0 \cdot 1/N)$, ... ;

8, $\cos(-2\pi \cdot 1 \cdot 1/N)$, $\sin(-2\pi \cdot 1 \cdot 1/N)$, ... ;

8, , ... ;

8, $\cos(-2\pi \cdot (N-1) \cdot 1/N)$, $\sin(-2\pi \cdot (N-1) \cdot 1/N)$, ... ;

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

stage range is [2..11]

3.28 Dsp_lal_api_func_trig_vec

Functions

- void [Rsdk_LA_rad2deg_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Element-wise convert angle in radians to degrees.
- void [Rsdk_LA_deg2rad_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Element-wise convert angle in degrees to radians.
- void [Rsdk_EF_asin_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Arcsine - real single precision, high accuracy.
- void [Rsdk_EF_acos_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Arccosine - real single precision, high accuracy.
- void [Rsdk_EF_sin_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sine - real single precision, high accuracy.
- void [Rsdk_EF_cos_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Cosine - real single precision, high accuracy.
- void [Rsdk_EF_atan_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Arctangent - real single precision, high accuracy.
- void [Rsdk_EF_atan2_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInputY, xb_vecN_2xf32 *pInputX, uint32_t size)
Arctangent2 - real single precision.

3.28.1 Detailed Description

3.28.2 Function Documentation

3.28.2.1 Rsdk_EF_acos_Vof32p8Vif32p8()

```
void Rsdk_EF_acos_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Arccosine - real single precision, high accuracy.

Function calculates the inverse cosine of a real data vectors.

It returns arccos(x) in domain $[0, \pi]$ for x in domain $[-1, 1]$.

Computed using formula $\text{acos}(x) = \pi/2 - \text{asin}(x)$.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{acos}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is $[1..4095]$

3.28.2.2 Rsdk_EF_asin_Vof32p8Vif32p8()

```
void Rsdk_EF_asin_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Arcsine - real single precision, high accuracy.

Function calculates the inverse sinus of a real data vectors.

It returns arcsin(x) in domain $[-\pi/2, \pi/2]$ for x in domain $[-1, 1]$.

Implemented algorithm from Approximations for Digital Computers, Cecil Hastings, p163.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{asin}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.28.2.3 RsdK_EF_atan2_Vof32p8Vif32p8Vif32p8()

```
void RsdK_EF_atan2_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInputY,
    xb_vecN_2xf32 * pInputX,
    uint32_t size )
```

Arctangent2 - real single precision.

Function calculates 4 quadrant inverse tangent of 2 real data vectors.
It returns $\text{atan2}(y/x)$ in domain $[-\pi, \pi]$ for x and y in domain $(-\infty, \infty)$.
Function uses $\text{atan}()$ - high accuracy.
Output must not be in-place memory vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInputY</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInputX</i>	Second operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{atan2}(\text{vector2}, \text{vector1}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2730]

3.28.2.4 RsdK_EF_atan_Vof32p8Vif32p8()

```
void RsdK_EF_atan_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Arctangent - real single precision, high accuracy.

Function calculates the inverse tangent of a real data vectors.
It returns $\text{atan}(x)$ in domain $[-\pi/2, \pi/2]$ for x in domain $(-\infty, \infty)$ using a Maclaurin expansion.
This avoids the expensive division needed in the Pade approximation alternative and is slightly faster.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{atan}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.28.2.5 Rsdk_EF_cos_Vof32p8Vif32p8()

```
void Rsdk_EF_cos_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Cosine - real single precision, high accuracy.

Function calculates the cosine of a real data vectors.

It returns $\cos(x)$ in domain $[-1,1]$ for x in domain $[-\pi,\pi]$.

Computed using formula $\cos(x) = \sin(x + \pi/2)$.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\cos(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.28.2.6 Rsdk_EF_sin_Vof32p8Vif32p8()

```
void Rsdk_EF_sin_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sine - real single precision, high accuracy.

Function calculates the sinus of a real data vector.

It returns $\sin(x)$ in domain $[-1, 1]$ for x in domain $[-\pi, \pi]$.

Implemented algorithm from Approximations for Digital Computers, Cecil Hastings, p140.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\sin(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.28.2.7 RsdK_LA_deg2rad_Vof32p8Vif32p8()

```
void RsdK_LA_deg2rad_Vof32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Element-wise convert angle in degrees to radians.

Element-wise convert angle in degrees to radians by multiplying each element of a real single precision vector with pre-calculated constant $\pi / 180.0$

$\text{angleInRadians} = (\text{angleInDegrees}) * \pi / 180.0$

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

size range is [1..4095]

3.28.2.8 RsdK_LA_rad2deg_Vof32p8Vif32p8()

```
void RsdK_LA_rad2deg_Vof32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
```

```
xb_vecN_2xf32 * pInput,
uint32_t size )
```

Element-wise convert angle in radians to degrees.

Element-wise convert angle in radians degrees by multiplying each element of a real single precision vector with pre-calculated constant $180.0 / \text{PI}$

$\text{angleInRadians} = (\text{angleInDegrees}) * 180.0 / \text{PI}$

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.29 Dsp_lal_api_func_log_vec

Functions

- void [Rsdk_EF_log_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Natural logarithm - real single precision, high accuracy.
- void [Rsdk_EF_log2_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Logarithm base 2 - real single precision, high accuracy.
- void [Rsdk_EF_log10_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Logarithm base 10 - real single precision, high accuracy.

3.29.1 Detailed Description

3.29.2 Function Documentation

3.29.2.1 Rsdk_EF_log10_Vof32p8Vif32p8()

```
void Rsdk_EF_log10_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Logarithm base 10 - real single precision, high accuracy.

Function calculates the base 10 logarithm of a real data vectors using the formula $\log_{10}(x) = \log(x) / \log(10)$.

$\log(x)$ is implemented from Approximations for Digital Computers, Cecil Hastings, p176.

Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\log_{10}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.29.2.2 RsdK_EF_log2_Vof32p8Vif32p8()

```
void RsdK_EF_log2_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Logarithm base 2 - real single precision, high accuracy.

Function calculates the base 2 logarithm of a real data vectors using the formula $\log_2(x) = \log(x)/\log(2)$.
 $\log(x)$ is implemented from Approximations for Digital Computers, Cecil Hastings, p176.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\log_2(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.29.2.3 RsdK_EF_log_Vof32p8Vif32p8()

```
void RsdK_EF_log_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Natural logarithm - real single precision, high accuracy.

Function calculates the natural logarithm of a real data vectors using the Taylor series approximation, 4 coefficients. Implemented algorithm from Approximations for Digital Computers, Cecil Hastings, p176. Input and output must not be in-place memory vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\log(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.30 Dsp_lal_api_func_prod_vec

Functions

- void [Rsdk_LA_cross_Vo3f32p8Vi3f32p8Vi3f32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2)
Cross product of 2 vectors - real single precision.
- void [Rsdk_LA_cross_Vo3f32cp8Vi3f32cp8Vi3f32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2)
Cross product of 2 vectors - complex single precision.
- void [Rsdk_LA_crossconj_Vo3f32cp8Vi3f32cp8Vi3f32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2)
Complex conjugated cross product of 2 vectors - complex single precision.
- void [Rsdk_LA_dot_Sof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Dot product of 2 vectors - real single precision.
- void [Rsdk_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Dot product of 2 vectors - complex single precision.
- void [Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Dot product of 2 vectors - complex single precision.

3.30.1 Detailed Description

3.30.2 Function Documentation

3.30.2.1 Rsdk_LA_cross_Vo3f32cp8Vi3f32cp8Vi3f32cp8()

```
void Rsdk_LA_cross_Vo3f32cp8Vi3f32cp8Vi3f32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2 )
```

Cross product of 2 vectors - complex single precision.

Function computes the cross(vectorial) product for 2 complex data vectors of size 3.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size 3.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size 3.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size 3.

Note

$\text{cross}(\text{vector1}, \text{vector2}) = \text{vector1} \times \text{vector2}$

3.30.2.2 Rsdk_LA_cross_Vo3f32p8Vi3f32p8Vi3f32p8()

```
void Rsdk_LA_cross_Vo3f32p8Vi3f32p8Vi3f32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2 )
```

Cross product of 2 vectors - real single precision.

Function computes the cross(vectorial) product of 2 real data vectors of size 3.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size 3.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size 3.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size 3.

Note

$\text{cross}(\text{vector1}, \text{vector2}) = \text{vector1} \times \text{vector2}$

3.30.2.3 Rsdk_LA_crossconj_Vo3f32cp8Vi3f32cp8Vi3f32cp8()

```
void Rsdk_LA_crossconj_Vo3f32cp8Vi3f32cp8Vi3f32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2 )
```

Complex conjugated cross product of 2 vectors - complex single precision.

Function computes the conjugated cross(vectorial) product for 2 complex data vectors of size 3.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size 3.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size 3.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size 3.

Note

$\text{crossconj}(\text{vector1}, \text{vector2}) = \text{conj}(\text{vector1} \times \text{vector2})$

3.30.2.4 RsdK_LA_dot_Sof32p8Vif32p8Vif32p8()

```
void RsdK_LA_dot_Sof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Dot product of 2 vectors - real single precision.

Function computes the dot(scalar) product of 2 real data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 real scalars, size 1.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{dot}(\text{vector1}, \text{vector2}) = \sum(\text{vector1}[i] * \text{vector2}[i])$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..4095]

3.30.2.5 RsdK_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8()

```
void RsdK_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Dot product of 2 vectors - complex single precision.

Function computes the dot(scalar) product for 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 complex scalars, size 1.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{dot}(\text{vector1}, \text{vector2}) = \text{sum}(\text{conj}(\text{vector1}[i]) * \text{vector2}[i])$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2047]

3.30.2.6 Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8()

```
void Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Dot product of 2 vectors - complex single precision.

Function computes the dot(scalar) product for 2 complex data vectors.

Parameters

out	<i>pOutput</i>	Result contains 8 complex scalars, size 1.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>pInput2</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{dot}(\text{vector1}, \text{vector2}) = \text{sum}(\text{vector1}[i] * \text{conj}(\text{vector2}[i]))$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [1..2047]

3.31 Dsp_lal_api_func_det_mat

Functions

- void [Rsdk_LA_detSq_Sof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, xb_vecN_2xf32 *pBuffer1, xb_vecN_2xf32 *pBuffer2, uint32_t size)

Determinant of a matrix - real single precision.

- void [Rsdk_LA_detSq_Sof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput, xb_vecN_2xcf32 *pBuffer1, xb_vecN_2xcf32 *pBuffer2, uint32_t size)

Determinant of a matrix - complex single precision.

3.31.1 Detailed Description

3.31.2 Function Documentation

3.31.2.1 Rsdk_LA_detSq_Sof32cp8Mif32cp8()

```
void Rsdk_LA_detSq_Sof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput,
    xb_vecN_2xcf32 * pBuffer1,
    xb_vecN_2xcf32 * pBuffer2,
    uint32_t size )
```

Determinant of a matrix - complex single precision.

Function computes the determinant of a complex matrix using the upper triangular matrix resulted after the QR decomposition of the input matrix.

Parameters

out	<i>pOutput</i>	Result contains 8 complex scalars.
in	<i>pInput</i>	First operand contains 8 parallel complex matrices, size NxN.
in, out	<i>pBuffer1</i>	Scratch area contains 8 parallel complex matrices, size NxN. It may have the same address as the input buffer - inplace mode. In this case the input buffer is overwritten.
in, out	<i>pBuffer2</i>	Scratch area contains 8 parallel complex vectors, size N.
in	<i>size</i>	Number of rows and columns in the input matrix, N.

Note

$\det(\text{matrix}) = \text{scalar}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

If used in inplace mode:

size range is [2..63]

else:

size range is [2..44]

3.31.2.2 Rsdk_LA_detSq_Sof32p8Mif32p8()

```
void Rsdk_LA_detSq_Sof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    xb_vecN_2xf32 * pBuffer1,
    xb_vecN_2xf32 * pBuffer2,
    uint32_t size )
```

Determinant of a matrix - real single precision.

Function computes the determinant of a real matrix using the upper triangular matrix resulted after the QR decomposition of the input matrix.

Parameters

out	<i>pOutput</i>	Result contains 8 real scalars.
in	<i>pInput</i>	First operand contains 8 parallel real matrices, size NxN.
in, out	<i>pBuffer1</i>	Scratch area contains 8 parallel real matrices, size NxN. It may have the same address as the input buffer - inplace mode. In this case the input buffer is overwritten.
in, out	<i>pBuffer2</i>	Scratch area contains 8 parallel real vectors, size N.
in	<i>size</i>	Number of rows and columns in the input matrix, N.

Note

$\det(\text{matrix}) = \text{scalar}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

If used in inplace mode:

size range is [2..90]

else:

size range is [2..63]

3.32 Dsp_lal_api_func_qr_dec_mat

Functions

- void [Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutputR, xb_vecN_2xf32 *pOutputQ, xb_vecN_2xf32 *pBuffer, xb_vecN_2xf32 *pInput, uint32_t rows, uint32_t cols)

QR decomposition of a matrix - real single precision.

- void [Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutputR, xb_vecN_2xcf32 *pOutputQ, xb_vecN_2xcf32 *pBuffer, xb_vecN_2xcf32 *pInput, uint32_t rows, uint32_t cols)

QR decomposition of a matrix - complex single precision.

3.32.1 Detailed Description

3.32.2 Function Documentation

3.32.2.1 Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8()

```
void Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutputR,
    xb_vecN_2xcf32 * pOutputQ,
    xb_vecN_2xcf32 * pBuffer,
    xb_vecN_2xcf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

QR decomposition of a matrix - complex single precision.

Function computes the QR decomposition in parallel for each complex matrix in batch using Householder reflection.

Function assumes that all matrices have linearly independent columns.

Usual R, Q are not unique, but if input matrix is of full rank, then there exists a single R that has all positive diagonal elements.

If input matrix is square non singular, then the decomposition is unique.

Corresponding rows and columns in Q and R can flip their signs.

pOutputR and pInput can point to the same address - inplace mode. In this case the input buffer is overwritten.

outputQ*outputR = input

$Q \cdot R = \text{input}$

Parameters

out	<i>pOutputR</i>	First result contains 8 parallel complex R matrices, size MxN. The R-matrix is an upper triangular matrix.
out	<i>pOutputQ</i>	Second result contains 8 parallel complex Q matrices, size MxM. The Q-matrix has orthonormal columns($Q^H \cdot Q = I$). If input is a square matrix Q is orthogonal($Q \cdot Q^H = Q^H \cdot Q = I$).
in, out	<i>pBuffer</i>	Scratch area for a vector with the length equal with the number of lines in the input matrix, size M.
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size MxN.
in	<i>rows</i>	Row size of input matrix, Q matrix and R matrix and column size for Q matrix - (M).
in	<i>cols</i>	Column size of input matrix and of R matrix - (N).

Note

$qr(\text{matrixM}) = \text{matrixQ}, \text{matrixR}$ with $M = Q \cdot R$

rows $\geq$ cols

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

If used in inplace mode:

rows range is [2..M]

cols range is [2..N]

$(M * (N + M + 1)) < 4095$

else:

rows range is [2..M]

cols range is [2..N]

$(M * (2 * N + M + 1)) < 4095$

3.32.2.2 Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8()

```
void Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutputR,
    xb_vecN_2xf32 * pOutputQ,
    xb_vecN_2xf32 * pBuffer,
    xb_vecN_2xf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

QR decomposition of a matrix - real single precision.

Function computes the QR decomposition in parallel for each real matrix in batch using Householder transformations.

Function assumes that all matrices have linearly independent columns.

Usual R, Q are not unique, but if input matrix is of full rank, then there exists a single R that has all positive diagonal elements.

If the input matrix is square non singular, the decomposition is unique.

Corresponding rows and columns in Q and R can flip their signs.

pOutputR and pInput can point to the same address - inplace mode. In this case the input buffer is overwritten.

outputQ*outputR = input

$Q \cdot R = \text{input}$

Parameters

out	<i>pOutputR</i>	First result contains 8 parallel real R matrices, size MxN. The R-matrix is an upper triangular matrix
out	<i>pOutputQ</i>	Second result contains 8 parallel real Q matrices, size MxM. The Q-matrix has orthonormal columns($Q^T \cdot Q = I$). If input is a square matrix, Q is orthonormal($Q \cdot Q^T = Q^T \cdot Q = I$).
in, out	<i>pBuffer</i>	Scratch area for a vector with the length equal with the number of lines in the input matrix, size M.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size MxN.
in	<i>rows</i>	Row size of input matrix, Q matrix and R matrix and column size for Q matrix - (M).
in	<i>cols</i>	Column size of input matrix and of R matrix - (N).

Note

$\text{qr}(\text{matrixM}) = \text{matrixQ}, \text{matrixR}$ with $M = Q \cdot R$

rows $\geq$ cols

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.

If used in inplace mode:

rows range is [2..M]

cols range is [2..N]

$(M * (N + M + 1)) < 8192$

else:

rows range is [2..M]

cols range is [2..N]

$(M * (2 * N + M + 1)) < 8192$

3.33 Dsp_lal_api_func_cholesky_dec_mat

Functions

- void [Rsdk_LA_decLLCholPd_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Cholesky decomposition of a matrix - LL - real single precision.
- void [Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Cholesky decomposition of a matrix - LL - complex single precision.

3.33.1 Detailed Description

3.33.2 Function Documentation

3.33.2.1 Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8()

```
void Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Cholesky decomposition of a matrix - LL - complex single precision.

The function applies Cholesky decomposition on complex input matrix and returns lower triangular matrix implementing the Cholesky-Crout algorithm.

Function assumes the input matrix is square and Hermitian.

Function should not be used for input matrices that are not positive definite or semi-definite.

Decomposition is unique when the input matrix is positive definite, then the output matrix has real and positive diagonal entries.

If the input matrix is positive semi-definite the decomposition is not unique and the output diagonal entries can be zeros.

$\text{input} = L * L^H$

Parameters

out	<i>pOutput</i>	Result contains 8 parallel low-triangular real matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices.

Note

$\text{Cholesky}(\text{inmatrix}) = \text{outmatrix}$ with $\text{inmatrix} = \text{outmatrix} * \text{outmatrix}^H$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..45]

3.33.2.2 Rsdk_LA_decLLCholPd_Mof32p8Mif32p8()

```
void Rsdk_LA_decLLCholPd_Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Cholesky decomposition of a matrix - LL - real single precision.

The function applies Cholesky decomposition on real input matrix and returns lower triangular matrix implementing the Cholesky-Crout algorithm.

Function assumes the input matrix is square and symmetric.

Function should not be used for input matrices that are not positive definite or semi-definite.

Decomposition is unique when the input matrix is positive definite and the output matrix has real and positive diagonal entries.

If the input matrix is positive semi-definite the decomposition is not unique and the output diagonal entries can be zeros.

$\text{input} = L * L^T$

Parameters

out	<i>pOutput</i>	Result contains 8 parallel low-triangular real matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices.

Note

$\text{Cholesky}(\text{inmatrix}) = \text{outmatrix}$ with $\text{inmatrix} = \text{outmatrix} * \text{outmatrix}^T$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..63]

3.34 Dsp_lal_api_func_conj_vec

Functions

- void [Rsdk_LA_conj_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)

Conjugate - complex single precision.

3.34.1 Detailed Description

3.34.2 Function Documentation

3.34.2.1 Rsdk_LA_conj_Vof32cp8Vif32cp8()

```
void Rsdk_LA_conj_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Conjugate - complex single precision.

Function implements the conjugate operation on a vector.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

$\text{conj}(\text{vector}) = \text{vector}$

The following size is estimated in the ideal condition that the function uses the whole memory of 256 KBytes.
size range is [2..2047]

3.35 Dsp_lal_api_func_min_max_vec

Functions

- void [Rsdk_LA_min_Sou32p8Vif32p8](#) (xb_vecN_2x32Uv *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Min - real single precision.
- void [Rsdk_LA_minAbs_Sou32p8Vif32cp8](#) (xb_vecN_2x32Uv *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Min - complex single precision.
- void [Rsdk_LA_max_Sou32p8Vif32p8](#) (xb_vecN_2x32Uv *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Max - real single precision.
- void [Rsdk_LA_maxAbs_Sou32p8Vif32cp8](#) (xb_vecN_2x32Uv *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Max - complex single precision.

3.35.1 Detailed Description

3.35.2 Function Documentation

3.35.2.1 Rsdk_LA_max_Sou32p8Vif32p8()

```
void Rsdk_LA_max_Sou32p8Vif32p8 (
    xb_vecN_2x32Uv *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Max - real single precision.

Function implements maximum operation for real data vectors.

The function returns the index of the maximum value.

In case of multiple appearances of the maximum value, the index of the first apperance is returned. Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar values, the positions of the maximum values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

max(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.35.2.2 Rsdk_LA_maxAbs_Sou32p8Vif32cp8()

```
void Rsdk_LA_maxAbs_Sou32p8Vif32cp8 (
    xb_vecN_2x32Uv *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Max - complex single precision.

Function implements maximum operation for complex data vectors.

The function returns the index of element with the largest magnitude.

In case of multiple appearances of the maximum magnitude, the index of the first apperance is returned.

Phase is not used.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar values, the positions of the maximum magnitude values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\max(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.35.2.3 Rsdk_LA_min_Sou32p8Vif32p8()

```
void Rsdk_LA_min_Sou32p8Vif32p8 (
    xb_vecN_2x32Uv *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Min - real single precision.

Function implements minimum operation for real data vectors.

The function returns the index of the minimum value.

In case of multiple appearances of the minimum value, the first position is returned.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar values, the positions of the minimum values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\min(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.35.2.4 Rsdk_LA_minAbs_Sou32p8Vif32cp8()

```
void Rsdk_LA_minAbs_Sou32p8Vif32cp8 (
    xb_vecN_2x32Uv *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Min - complex single precision.

Function implements minimum operation for complex data vectors.

The function returns the index of element with the smallest magnitude.

In case of multiple appearances of the minimum magnitude, the index of the first appearance is returned.

Phase is not used.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar values, the positions of the minimum magnitude values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\min(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.36 Dsp_lal_api_func_sum_vec

Functions

- void [Rsdk_LA_sum_Sof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum - real single precision.
- void [Rsdk_LA_sum_Sof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum - complex single precision.
- void [Rsdk_LA_sum2_Sof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum of all squared vector elements - real single precision.
- void [Rsdk_LA_sum2_Sof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum of all squared vector elements - complex single precision.
- void [Rsdk_LA_sum3_Sof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum of all cubes vector elements - real single precision.
- void [Rsdk_LA_sum3_Sof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sum of all cubes vector elements - complex single precision.
- void [Rsdk_LA_sumRcp_Sof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
Sum of all reciprocal vector elements - real single precision.
- void [Rsdk_LA_sumRcp_Sof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
Sum of all reciprocal vector elements - complex single precision.

3.36.1 Detailed Description

3.36.2 Function Documentation

3.36.2.1 Rsdk_LA_sum2_Sof32cp8Vif32cp8()

```
void Rsdk_LA_sum2_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Sum of all squared vector elements - complex single precision.

Function implements sum of all squared elements operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\text{sum2}(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.36.2.2 Rsdk_LA_sum2_Sof32p8Vif32p8()

```
void Rsdk_LA_sum2_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sum of all squared vector elements - real single precision.

Function implements sum of all squared elements operation for real data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\text{sum2}(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256

kBytes: $N \leq 8000$.

3.36.2.3 Rsdk_LA_sum3_Sof32cp8Vif32cp8()

```
void Rsdk_LA_sum3_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Sum of all cubes vector elements - complex single precision.

Function implements sum of all cubes elements operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

$\text{sum3}(\text{vector}) = \text{scalar}$

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.36.2.4 Rsdk_LA_sum3_Sof32p8Vif32p8()

```
void Rsdk_LA_sum3_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sum of all cubes vector elements - real single precision.

Function implements sum of all cubes elements operation for real data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

sum3(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.36.2.5 Rsdk_LA_sum_Sof32cp8Vif32cp8()

```
void Rsdk_LA_sum_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Sum - complex single precision.

Function implements sum operation for complex data vectors.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

sum(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.36.2.6 Rsdk_LA_sum_Sof32p8Vif32p8()

```
void Rsdk_LA_sum_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sum - real single precision.

Function implements sum operation for real data vectors.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

sum(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.36.2.7 Rsdk_LA_sumRcp_Sof32cp8Vif32cp8()

```
void Rsdk_LA_sumRcp_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t size )
```

Sum of all reciprocal vector elements - complex single precision.

Reciprocal summation is the addition of a sequence of each 1 divided by element in vector, called addends or summands. The result is their reciprocal sum or reciprocal total.

Parameters

out	<i>pOutput</i>	Result, contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	Operand, contains 8 parallel complex vectors, size N.
in	<i>size</i>	Number of vector elements(N).

Note

element reciprocal sum (vector) = scalar

n = size

v = n, a1, a2..., an;

a1 = x + iy;

ersum(v) = $1/a1 + 1/a2 + \dots + 1/an$;

$1/a1 = x/(x^2 + y^2) - iy/(x^2 + y^2)$; The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.36.2.8 Rsdk_LA_sumRcp_Sof32p8Vif32p8()

```
void Rsdk_LA_sumRcp_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
```

```
xb_vecN_2xf32 const * pInput,
uint32_t size )
```

Sum of all reciprocal vector elements - real single precision.

Reciprocal summation is the addition of a sequence of each 1 divided by element in vector, called addends or summands. The result is their reciprocal sum or reciprocal total.

The parallel stream result is NAN, if one input in that stream is zero.

Parameters

out	<i>pOutput</i>	Result, contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	Operand, contains 8 parallel real vectors, size N.
in	<i>size</i>	Number of vector elements(N).

Note

element reciprocal sum (vector) = scalar

n = size

v = n, a1, a2..., an;

ersum(v) = 1/a1 + 1/a2 + ... + 1/an; The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 8000.

3.37 Dsp_lal_api_func_inv_up_triang_mat

Functions

- void [Rsdk_LA_invUNs_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Upper triangular matrix inverse - real single precision.
- void [Rsdk_LA_invUNs_Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Upper triangular matrix inverse - complex single precision.
- void [Rsdk_LA_invRectUNs_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t rows, uint32_t cols)
Upper triangular rectangular matrix inverse - real single precision.
- void [Rsdk_LA_invRectUNs_Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t rows, uint32_t cols)
Upper triangular rectangular matrix inverse - complex single precision.

3.37.1 Detailed Description

3.37.2 Function Documentation

3.37.2.1 Rsdk_LA_invRectUNs_Mof32cp8Mif32cp8()

```
void Rsdk_LA_invRectUNs_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

Upper triangular rectangular matrix inverse - complex single precision.

Function implements matrix inverse for a rectangular upper triangular matrix.

Function assumes input matrix is rectangular upper triangular, non-singular and has inverse.

Function assumes output matrix is a NxM matrix filled with zeros.

Output cannot be in-place memory.

Number of rows of the input matrix should be $\geq$ number of columns

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxM.
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size MxN.
in	<i>rows</i>	Number of rows in the input matrix and number of columns in the output matrix, $M(\geq 2)$, $M \geq N$.
in	<i>cols</i>	Number of columns in the input matrix and number of rows in the output matrix, $N(\geq 2)$.

Returns

void

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $M \times N < 46 \times 46$.

3.37.2.2 Rsdk_LA_invRectUNs_Mof32p8Mif32p8()

```
void Rsdk_LA_invRectUNs_Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

Upper triangular rectangular matrix inverse - real single precision.

Function implements matrix inverse for a rectangular upper triangular matrix.

Function assumes input matrix is rectangular, upper triangular, non-singular and has inverse.

Function assumes output matrix is a NxM matrix filled with zeros.

Output cannot be in-place memory.

Number of rows of the input matrix should be $\geq$ number of columns

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxM.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size MxN.
in	<i>rows</i>	Number of rows in the input matrix and number of columns in the output matrix, $M(\geq 2)$, $M \geq N$.
in	<i>cols</i>	Number of columns in the input matrix and number of rows in the output matrix, $N(\geq 2)$.

Returns

void

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $M \times N < 64 \times 64$.

3.37.2.3 Rsdk_LA_invUNs_Mof32cp8Mif32cp8()

```
void Rsdk_LA_invUNs_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Upper triangular matrix inverse - complex single precision.

Function implements matrix inverse for an upper triangular matrix.

Function assumes input matrix is an upper triangular, non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxN
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, $N(\geq 2)$.

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 45$.

3.37.2.4 Rsdk_LA_invUNs_Mof32p8Mif32p8()

```
void Rsdk_LA_invUNs_Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Upper triangular matrix inverse - real single precision.

Function implements matrix inverse for an upper triangular matrix.

Function assumes input matrix is an upper triangular, non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 64$.

3.38 Dsp_lal_api_func_inv_low_triang_mat

Functions

- void [Rsdk_LA_invLNs_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Lower triangular matrix inverse - real single precision.
- void [Rsdk_LA_invLNs_Mof32cp8Mif32cp8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Lower triangular matrix inverse - complex single precision.
- void [Rsdk_LA_invRectLNs_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t rows, uint32_t cols)
Lower triangular rectangular matrix inverse - real single precision.
- void [Rsdk_LA_invRectLNs_Mof32cp8Mif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t rows, uint32_t cols)
Lower triangular rectangular matrix inverse - complex single precision.

3.38.1 Detailed Description

3.38.2 Function Documentation

3.38.2.1 Rsdk_LA_invLNs_Mof32cp8Mif32cp8()

```
void Rsdk_LA_invLNs_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Lower triangular matrix inverse - complex single precision.

Function implements matrix inverse for a lower triangular matrix.

Function assumes input matrix is a lower triangular, non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxN
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 45$.

3.38.2.2 Rsdk_LA_invLNs_Mof32p8Mif32p8()

```
void Rsdk_LA_invLNs_Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Lower triangular matrix inverse - real single precision.

Function implements matrix inverse for a lower triangular matrix.

Function assumes input matrix is a lower triangular, non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxN.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 64$.

3.38.2.3 RsdK_LA_invRectLNs_Mof32cp8Mif32cp8()

```
void RsdK_LA_invRectLNs_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

Lower triangular rectangular matrix inverse - complex single precision.

Function implements matrix inverse for a lower triangular rectangular matrix.

Function assumes input matrix is rectangular, lower triangular, non-singular and has inverse.

Function assumes output matrix is a NxM matrix filled with zeros.

Output cannot be in-place memory.

Number of rows of the input matrix should be $\leq$ number of columns

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices,size NxM
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size MxN.
in	<i>rows</i>	Number of rows in the input matrix and number of columns in the output matrix, $M(\geq 2)$, $M \leq N$.
in	<i>cols</i>	Number of columns in the input matrix and number of rows in the output matrix, $N(\geq 2)$.

Returns

void

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $M \times N < 46 \times 46$.

3.38.2.4 RsdK_LA_invRectLNs_Mof32p8Mif32p8()

```
void RsdK_LA_invRectLNs_Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
```

```

    xb_vecN_2xf32 * pInput,
    uint32_t rows,
    uint32_t cols )

```

Lower triangular rectangular matrix inverse - real single precision.

Function implements matrix inverse for a lower triangular rectangular matrix.

Function assumes input matrix is rectangular, lower triangular, non-singular and has inverse.

Function assumes output matrix is a NxM matrix filled with zeros.

Output cannot be in-place memory.

Number of rows of the input matrix should be $\leq$ number of columns

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxM.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size MxN.
in	<i>rows</i>	Number of rows in the input matrix and number of columns in the output matrix, $M(\geq 2)$, $M \leq N$.
in	<i>cols</i>	Number of columns in the input matrix and number of rows in the output matrix, $N(\geq 2)$.

Returns

void

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $M \times N < 64 \times 64$.

3.39 Dsp_lal_api_func_inv_qr_mat

Functions

- void [Rsdk_LA_invHousQRNs_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 *pInput, xb_vecN_2xf32 *pBuffer1, xb_vecN_2xf32 *pBuffer2, uint32_t size)
Matrix inverse - real single precision.
- void [Rsdk_LA_invHousQRNs_Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 *pInput, xb_vecN_2xcf32 *pBuffer1, xb_vecN_2xcf32 *pBuffer2, uint32_t size)
Matrix inverse - complex single precision.

3.39.1 Detailed Description

3.39.2 Function Documentation

3.39.2.1 Rsdk_LA_invHousQRNs_Mof32cp8Mif32cp8()

```
void Rsdk_LA_invHousQRNs_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 * pInput,
    xb_vecN_2xcf32 * pBuffer1,
    xb_vecN_2xcf32 * pBuffer2,
    uint32_t size )
```

Matrix inverse - complex single precision.

Function implements matrix inverse using QR decomposition with Householder reflection.

Function multiplies the inverse of R matrix with the inverse of the Q matrix.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices,size NxN
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxN.
in, out	<i>pBuffer1</i>	Scratch area for 8 parallel complex matrices used for Q matrices, size N*N.
in, out	<i>pBuffer2</i>	Scratch area for 8 parallel complex matrices used for R matrices and R inverse matrices, size N*N.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

inv(matrix) = matrix

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 32$.

3.39.2.2 Rsdk_LA_invHousQRNs_Mof32p8Mif32p8()

```
void Rsdk_LA_invHousQRNs_Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 * pInput,
    xb_vecN_2xf32 * pBuffer1,
    xb_vecN_2xf32 * pBuffer2,
    uint32_t size )
```

Matrix inverse - real single precision.

Function implements matrix inverse using QR decomposition with Householder reflection.

Function multiplies the inverse of R matrix with the inverse of the Q matrix.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size N×N.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size N×N.
in, out	<i>pBuffer1</i>	Scratch area for 8 parallel real matrices used for Q matrices, size N×N.
in, out	<i>pBuffer2</i>	Scratch area for 8 parallel real matrices used for R matrices and R inverse matrices, size N×N.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

$\text{inv}(\text{matrix}) = \text{matrix}$

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 45$.

3.40 Dsp_lal_api_func_inv_gauss_mat

Functions

- void [Rsdk_LA_invGausCPNs_Mof32p8Miof32p8](#) (xb_vecN_2xf32 *restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Matrix inverse - real single precision.
- void [Rsdk_LA_invNsGaus_Mof32p8Miof32p8](#) (xb_vecN_2xf32 *restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Matrix inverse - real single precision.
- void [Rsdk_LA_invGausCPNs_Mof32cp8Miof32cp8](#) (xb_vecN_2xcf32 *restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Matrix inverse - complex single precision.
- void [Rsdk_LA_invNsGaus_Mof32cp8Miof32cp8](#) (xb_vecN_2xcf32 *restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Matrix inverse - complex single precision.

3.40.1 Detailed Description

3.40.2 Function Documentation

3.40.2.1 Rsdk_LA_invGausCPNs_Mof32cp8Miof32cp8()

```
void Rsdk_LA_invGausCPNs_Mof32cp8Miof32cp8 (
    xb_vecN_2xcf32 *restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Matrix inverse - complex single precision.

Function implements matrix inverse using Gaussian Elimination with column pivoting.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices,size NxN
in, out	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

inv(matrix) = matrix

Function modifies input matrix. If one needs them for ulterior processing, they should copy the input to another buffer before calling this function. The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 35$.

3.40.2.2 RsdK_LA_invGausCPNs_Mof32p8Miof32p8()

```
void RsdK_LA_invGausCPNs_Mof32p8Miof32p8 (
    xb_vecN_2xf32 *restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Matrix inverse - real single precision.

Function implements matrix inverse using Gaussian Elimination with column pivoting.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxN.
in, out	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

inv(matrix) = matrix

Function modifies input matrix. If one needs them for ulterior processing, they should copy the input to another buffer before calling this function. The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 50$.

3.40.2.3 RsdK_LA_invNsGaus_Mof32cp8Miof32cp8()

```
void RsdK_LA_invNsGaus_Mof32cp8Miof32cp8 (
    xb_vecN_2xcf32 *restrict pOutput,
```

```
xb_vecN_2xcf32 * pInput,
uint32_t size )
```

Matrix inverse - complex single precision.

Function implements matrix inverse using Gaussian Elimination with no pivoting.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxN
in, out	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

inv(matrix) = matrix

Function modifies input matrix. If one needs them for ulterior processing, they should copy the input to another buffer before calling this function. The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 35.

3.40.2.4 Rsdk_LA_invNsGaus_Mof32p8Miof32p8()

```
void Rsdk_LA_invNsGaus_Mof32p8Miof32p8 (
    xb_vecN_2xf32 *restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Matrix inverse - real single precision.

Function implements matrix inverse using Gaussian Elimination with no pivoting.

Function assumes input matrix is non-singular and has inverse.

Output can be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxN.
in, out	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxN.
in	<i>size</i>	Number of rows and columns in the input and output matrices, N(>=2).

Note

inv(matrix) = matrix

Function modifies input matrix. If one needs them for ulterior processing, they should copy the input to another buffer before calling this function. The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 50.

3.41 Dsp_lal_api_func_eig_jacobi_mat

Functions

- void [RsdK_LA_eigSy_Mof32p8Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutputD, xb_vecN_2xf32 *pOutputV, xb_vecN_2xf32 *pInput, uint32_t size, uint32_t passes)
Eigen solver for symmetric NxN matrices - real single precision.
- void [RsdK_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutputD, xb_vecN_2xcf32 *pOutputV, xb_vecN_2xcf32 *pInput, uint32_t size, uint32_t passes)
Eigen solver for complex hermitian NxN matrices - complex single precision.

3.41.1 Detailed Description

3.41.2 Function Documentation

3.41.2.1 RsdK_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8()

```
void RsdK_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutputD,
    xb_vecN_2xcf32 * pOutputV,
    xb_vecN_2xcf32 * pInput,
    uint32_t size,
    uint32_t passes )
```

Eigen solver for complex hermitian NxN matrices - complex single precision.

Functions computes the eigenvalues and eigenvectors for the input matrices.

The eigen-decomposition: $A=V*D*V^T$ is done executing successive Jacobi rotations over the input matrix.

The function assumes the the matrices are complex hermitian.The algorithm will only use the values on and above the leading diagonal of the input matrix.

The output D matrix contains on the diagonal the eigenvalues, as result of the Jacobi rotations.

The output V matrix columns are the eigenvectors of the input matrix.

The eigen vector in column k of the matrix V has the corresponding eigenvalue in the diagonal element (k,k) of the matrix D.

Output can be in-place memory.

pOutputD and pInput can point to the same address.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D complex matrices - size NxN.
out	<i>pOutputV</i>	Result contains the 8 parallel V complex matrices - size NxN.
in	<i>pInput</i>	Operand contains the 8 parallel complex symmetric matrices - size NxN.
in	<i>size</i>	Number of rows and columns of input, D and V matrices: N (N>=2).
in	<i>passes</i>	Number of Jacobi iterations applied to the input matrix.

Note

`eig(matrix) = matrix matrix`

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 35$.

3.41.2.2 Rsdk_LA_eigSy_Mof32p8Mof32p8Mif32p8()

```
void Rsdk_LA_eigSy_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutputD,
    xb_vecN_2xf32 * pOutputV,
    xb_vecN_2xf32 * pInput,
    uint32_t size,
    uint32_t passes )
```

Eigen solver for symmetric NxN matrices - real single precision.

Functions computes the eigenvalues and eigenvectors for the input matrices.

The eigen-decomposition: $A=V*D*V^t$ is done executing successive Jacobi rotations over the input matrix.

The function assumes the the matrices are real symmetric. The algorithm will only use the values on and above the leading diagonal of the input matrix.

The output D matrix contains on the diagonal the eigenvalues, as result of the Jacobi rotations.

The output V matrix columns are the eigenvectors of the input matrix.

The eigenvector in column k of the matrix V has the corresponding eigenvalue in the diagonal element (k,k) of the matrix D.

Output can be in-place memory.

pOutputD and pInput can point to the same address.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D real matrices - size NxN.
out	<i>pOutputV</i>	Result contains the 8 parallel V real matrices - size NxN.
in	<i>pInput</i>	Operand contains the 8 parallel real symmetric matrices - size NxN.
in	<i>size</i>	Number of rows and columns of input, D and V matrices: $N(N \geq 2)$.
in	<i>passes</i>	Number of Jacobi iterations applied to the input matrix.

Note

`eig(matrix) = matrix matrix`

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 50$.

3.42 Dsp_lal_api_func_eig_ql_mat**Functions**

- void [Rsdk_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8](#) (xb_vecN_2xf32 *__restrict pOutputD, xb_vecN_2xf32 *__restrict pOutputV, xb_vecN_2xf32 *pInput, int32_t iterations, int32_t size)

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

- void `Rsdk_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8` (xb_vecN_2xf32 *__restrict pOutputD, xb_vecN_2xf32 *__restrict pOutputV, xb_vecN_2xf32 *pInput, int32_t iterations, int32_t size)

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

- void `Rsdk_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8` (xb_vecN_2xf32 *__restrict pOutputD, xb_vecN_2xf32 *pInput, int32_t iterations, int32_t size)

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

3.42.1 Detailed Description

3.42.2 Function Documentation

3.42.2.1 RsdK_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8()

```
void RsdK_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutputD,
    xb_vecN_2xf32 *__restrict pOutputV,
    xb_vecN_2xf32 * pInput,
    int32_t iterations,
    int32_t size )
```

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

Functions computes the eigenvalues and eigenvectors for the input matrices.

The QL decomposition is done executing Givens rotations.

A QL iteration step is performed with implicit shifts.

The function assumes the the matrices are in symmetric tri-diagonal form.

The eigen vector in column k of the matrix V has the corresponding eigenvalue in the element k of the vector D.

The eigenvector matrix pOutputV is passed to this function as the orthogonal Householder matrix used to transform an initial symmetric matrix to symmetric tri-diagonal form.

Typically only 2.5*N iterations are required but the cautious limit is 30*N.

Function is restricted to matrices with values < 1e+18 due to overflows.

Output cannot be in-place memory.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D real vectors, representing the eigenvalues - size 1xN.
in, out	<i>pOutputV</i>	Operand contains the 8 parallel real Householder matrix transforms - size NxN. Result contains the 8 parallel V real matrices, representing the eigenvectors - size NxN.
in	<i>pInput</i>	Operand contains the 8 parallel real matrices - size NxN.
in	<i>iterations</i>	Maximum number of iterations to be applied to the input matrix.
in	<i>size</i>	Number of rows and columns of input and V matrices and D vectors: N (N>=2).

Note

`eig(matrix) = vector(eigenvalues) matrix(eigenvectors)`

The maximum size of N is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N < 64.

3.42.2.2 Rsdk_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8()

```
void Rsdk_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutputD,
    xb_vecN_2xf32 *__restrict pOutputV,
    xb_vecN_2xf32 * pInput,
    int32_t iterations,
    int32_t size )
```

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

Functions computes the eigenvalues and eigenvectors for the input matrices.

The QL decomposition is done executing Givens rotations.

A QL iteration step is performed with implicit shifts.

The function assumes the the matrices are in symmetric tri-diagonal form.

The eigen vector in column k of the matrix V has the corresponding eigenvalue in the element k of the vector D.

The eigenvector matrix the eigenvectors of the symmetric tri-diagonal pInput.

Typically only 2.5*N iterations are required but the cautious limit is 30*N.

Function is restricted to matrices with values < 1e+18 due to overflows.

Output cannot be in-place memory.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D real vectors, representing the eigenvalues - size 1xN.
out	<i>pOutputV</i>	Result contains the 8 parallel V real matrices, representing the eigenvectors - size NxN.
in	<i>pInput</i>	Operand contains the 8 parallel real matrices - size NxN.
in	<i>iterations</i>	Maximum number of iterations to be applied to the input matrix.
in	<i>size</i>	Number of rows and columns of input and V matrices and D vectors: N (N>=2).

Note

$\text{eig}(\text{matrix}) = \text{vector}(\text{eigenvalues}) \text{matrix}(\text{eigenvectors})$

The maximum size of N is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 64$.

3.42.2.3 Rsdk_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8()

```
void Rsdk_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutputD,
    xb_vecN_2xf32 * pInput,
    int32_t iterations,
    int32_t size )
```

Eigen solver for real symmetric tri-diagonal NxN matrices using QL decomposition - real single precision.

Functions computes the eigenvalues for the input matrices.

The QL decomposition is done executing Givens rotations.

A QL iteration step is performed with implicit shifts.

The function assumes the the matrices are in symmetric tri-diagonal form.

Typically only 2.5*N iterations are required but the cautious limit is 30*N.

Function is restricted to matrices with values < 1e+18 due to overflows.

Output cannot be in-place memory.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D real vectors, representing the eigenvalues - size 1xN.
in	<i>pInput</i>	Operand contains the 8 parallel real matrices - size NxN.
in	<i>iterations</i>	Maximum number of iterations to be applied to the input matrix.
in	<i>size</i>	Number of rows and columns of input and V matrices and D vectors: N (N>=2).

Note

`eigVal(matrix) = vector(eigenvalues)`

The maximum size of N is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 64$.

3.43 Dsp_lal_api_func_eig_QRfrancis_mat

Functions

- void [Rsdk_LA_eigValUpHess_Vof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *__restrict pOutputD, xb_vecN_2xcf32 *pInput, uint32_t *iterations, uint32_t size)

Eigenvalues for complex upper Hessenberg form NxN matrices using Francis Algorithm - complex single precision.

- void [Rsdk_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *__restrict pOutputD, xb_vecN_2xcf32 *__restrict pOutputV, xb_vecN_2xcf32 *pInput, uint32_t *iterations, uint32_t size)

Eigen solver for complex upper Hessenberg form NxN matrices using Francis Algorithm - complex single precision.

3.43.1 Detailed Description

3.43.2 Function Documentation

3.43.2.1 RsdK_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8()

```
void RsdK_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutputD,
    xb_vecN_2xcf32 *__restrict pOutputV,
    xb_vecN_2xcf32 * pInput,
    uint32_t * iterations,
    uint32_t size )
```

Eigen solver for complex upper Hessenberg form NxN matrices using Francis Algorithm - complex single precision.

Functions computes the eigenvalues and eigenvectors for the input matrices.

The eigen-decomposition is done executing Francis QR iterations.

A QR iteration step is performed with implicit shifts.

The function assumes the the matrices are in upper Hessenberg form.

The eigen vector in column k of the matrix V has the corresponding eigenvalue in the element k of the vector D.

Typically only 3*N iterations are required but the cautious limit is 30*N.

Function is restricted to matrices with values < 1e+18 due to overflows.

Output cannot be in-place memory.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D complex vectors, representing the eigenvalues - size 1xN.
out	<i>pOutputV</i>	Result contains the 8 parallel V complex matrices, representing the eigenvectors - size NxN.
in	<i>pInput</i>	Operand contains the 8 parallel complex matrices - size NxN.
in, out	<i>iterations</i>	Maximum number of iterations to be applied to the input matrix. At the end of the algorithm it will contain the number of iterations remaining.
in	<i>size</i>	Number of rows and columns of input and V matrices and D vectors: N (N>=2).

Note

`eig(matrix) = vector(eigenvalues) matrix(eigenvectors)`

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 45$.

3.43.2.2 RsdK_LA_eigValUpHess_Vof32cp8Mif32cp8()

```
void RsdK_LA_eigValUpHess_Vof32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutputD,
    xb_vecN_2xcf32 * pInput,
    uint32_t * iterations,
    uint32_t size )
```

Eigenvalues for complex upper Hessenberg form NxN matrices using Francis Algorithm - complex single precision.

Functions computes the eigenvalues for the input matrices.

The computation is done executing Francis QR iterations.

A QR iteration step is performed with implicit shifts.

The function assumes the the matrices are in upper Hessenberg form.

Typically only 3*N iterations are required but the cautious limit is 30*N.

Function is restricted to matrices with values < 1e+18 due to overflows.

Output cannot be in-place memory.

Parameters

out	<i>pOutputD</i>	Result contains the 8 parallel D complex vectors, representing the eigenvalues - size 1xN.
in	<i>pInput</i>	Operand contains the 8 parallel complex matrices - size NxN.
in, out	<i>iterations</i>	Maximum number of iterations to be applied to the input matrix. At the end of the algorithm it will contain the number of iterations remaining.
in	<i>size</i>	Number of rows and columns of input and D vectors: N (N>=2).

Note

eigVal(matrix) = vector(eigenvalues)

The maximum size of N is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N < 64$.

3.44 Dsp_lal_api_func_mean_vec

Functions

- void **Rsdk_LA_avg_Sof32p8Vif32p8** (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Arithmetic Mean - real single precision.
- void **Rsdk_LA_avg_Sof32cp8Vif32cp8** (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Arithmetic Mean - complex single precision.
- void **Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8** (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Weighted Arithmetic Mean - real single precision.
- void **Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8** (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput1, xb_vecN_2xcf32 *pInput2, uint32_t size)
Weighted Arithmetic Mean - complex single precision.
- void **Rsdk_LA_havg_Sof32p8Vif32p8** (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Harmonic Mean - real single precision.
- void **Rsdk_LA_havg_Sof32cp8Vif32cp8** (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Harmonic Mean - complex single precision.
- void **Rsdk_LA_rms_Sof32p8Vif32p8** (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Root Mean Square or Quadratic Mean - real single precision.
- void **Rsdk_LA_rms_Sof32cp8Vif32cp8** (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xcf32 *pInput, uint32_t size)
Root Mean Square or Quadratic Mean - complex single precision.

3.44.1 Detailed Description

3.44.2 Function Documentation

3.44.2.1 Rsdsk_LA_avg_Sof32cp8Vif32cp8()

```
void Rsdsk_LA_avg_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Arithmetic Mean - complex single precision.

Function implements arithmetic mean operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Arithmetic mean(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.44.2.2 RsdK_LA_avg_Sof32p8Vif32p8()

```
void RsdK_LA_avg_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Arithmetic Mean - real single precision.

Function implements arithmetic mean operation for real data vectors.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Arithmetic mean(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.44.2.3 RsdK_LA_havg_Sof32cp8Vif32cp8()

```
void RsdK_LA_havg_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Harmonic Mean - complex single precision.

Function implements harmonic mean operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Harmonic mean(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.44.2.4 Rsdk_LA_havg_Sof32p8Vif32p8()

```
void Rsdk_LA_havg_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Harmonic Mean - real single precision.

Function implements harmonic mean operation for real data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Harmonic mean(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.44.2.5 Rsdk_LA_rms_Sof32p8Vif32cp8()

```
void Rsdk_LA_rms_Sof32p8Vif32cp8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Root Mean Square or Quadratic Mean - complex single precision.

Function implements root mean square or quadratic mean operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Root Mean Square(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.44.2.6 Rsdk_LA_rms_Sof32p8Vif32p8()

```
void Rsdk_LA_rms_Sof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Root Mean Square or Quadratic Mean - real single precision.

Function implements root mean square or quadratic mean operation for real data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput</i>	First operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input vector, N.

Note

Root Mean Square(vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256

kBytes: $N \leq 8000$.

3.44.2.7 Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8()

```
void Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput1,
    xb_vecN_2xcf32 * pInput2,
    uint32_t size )
```

Weighted Arithmetic Mean - complex single precision.

Function implements weighted arithmetic mean operation for complex data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar complex values.
in	<i>pInput1</i>	First operand contains 8 parallel complex vectors representing the values, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel complex vectors representing the weights, size N.
in	<i>size</i>	Size of input vector, N.

Note

Weighted arithmetic mean(vector vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 2000$.

3.44.2.8 Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8()

```
void Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Weighted Arithmetic Mean - real single precision.

Function implements weighted arithmetic mean operation for real data vectors.
Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains a pointer to 8 parallel scalar real values.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors representing the values, size N.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors representing the weights, size N.
in	<i>size</i>	Size of input vector, N.

Note

Weighted arithmetic mean(vector vector) = scalar

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.45 Dsp_lal_api_func_conversions_vec_mat

Functions

- void [Rsdk_DT_getRe_Vof32p8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Convert complex vectors into real vectors by extracting the real part of each vector element - single precision.
- void [Rsdk_DT_getIm_Vof32p8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Convert complex vectors into real vectors by extracting the imaginary part of each vector element - single precision.
- void [Rsdk_DT_interl_Mof32cp8Mif32p8Mif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t rows, uint32_t cols)
Join two matrices with real elements into a matrix with complex elements - single precision.
- void [Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput1, xb_vecN_2xf32 *__restrict pOutput2, xb_vecN_2xf32 *pInput, uint32_t rows, uint32_t cols)
Split a complex matrix into two matrices first containing the real complex part and second the imaginary complex part - single precision.

3.45.1 Detailed Description

3.45.2 Function Documentation

3.45.2.1 Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8()

```
void Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8 (
    xb_vecN_2xf32 *__restrict pOutput1,
    xb_vecN_2xf32 *__restrict pOutput2,
    xb_vecN_2xf32 * pInput,
    uint32_t rows,
    uint32_t cols )
```

Split a complex matrix into two matrices first containing the real complex part and second the imaginary complex part - single precision.

Break input complex matrix in two output matrices containing real part and imaginary part.

First output matrix holds the real part of the complex input matrix.

Second output matrix holds the imaginary part of the complex input matrix.

This is the reverse function to Rsdk_DT_interl_Mof32cp8Mif32p8Mif32p8.

Output must not be in-place memory.

Parameters

out	<i>pOutput1</i>	First result real matrix size NxM..
out	<i>pOutput2</i>	Second result real matrix size NxM.
in	<i>pInput</i>	Operand complex matrix size NxM.
in	<i>rows</i>	Number of rows in input, output matrixes N.
in	<i>cols</i>	Number of columns in input, output matrixes M.

Note

deinterl(complex_matrix) = (real_matrix, real_matrix)

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: NxM <= 2000.

3.45.2.2 RsdK_DT_getIm_Vof32p8Vif32cp8()

```
void RsdK_DT_getIm_Vof32p8Vif32cp8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xcf32 * pInput,
    uint32_t size )
```

Convert complex vectors into real vectors by extracting the imaginary part of each vector element - single precision.

Extract and concatenate the imaginary part of the inputs vectors.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors size N.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors size N.
in	<i>size</i>	Size of input vector, N.

Note

getIm (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 2000.

3.45.2.3 RsdK_DT_getRe_Vof32p8Vif32cp8()

```
void RsdK_DT_getRe_Vof32p8Vif32cp8 (
    xb_vecN_2xf32 *__restrict pOutput,
```

```
xb_vecN_2xcf32 * pInput,
uint32_t size )
```

Convert complex vectors into real vectors by extracting the real part of each vector element - single precision.

Extract and concatenate the real part of the inputs vectors.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors size N.
in	<i>pInput</i>	First operand contains 8 parallel complex vectors size N.
in	<i>size</i>	Size of input vector, N.

Note

getRe (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 2000$.

3.45.2.4 RsdK_DT_interl_Mof32cp8Mif32p8Mif32p8()

```
void RsdK_DT_interl_Mof32cp8Mif32p8Mif32p8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t rows,
    uint32_t cols )
```

Join two matrices with real elements into a matrix with complex elements - single precision.

Interleave first matrix as real parts of the corresponding output complex elements and second matrix as the imaginary parts, respectively.

First input matrix holds the real part of the complex output matrix.

Second input matrix holds the imaginary part of the complex output matrix.

This is the reverse function to RsdK_DT_deinterl_Mof32p8Mof32p8Mif32cp8.

Output must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result complex matrix size NxM.
in	<i>pInput1</i>	First operand real matrix size NxM.
in	<i>pInput2</i>	Second operand real matrix size NxM.
in	<i>rows</i>	Number of rows in input, output matrixes N.
in	<i>cols</i>	Number of columns in input, output matrixes M.

Note

interl(real_matrix, real_matrix) = complex_matrix

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \times M \leq 2000$.

3.46 Dsp_lal_api_func_sort_vec

Functions

- void [Rsdk_AU_sortdOET_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sort descending - real single precision.
- void [Rsdk_AU_sortOET_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *pInput, uint32_t size)
Sort ascending - real single precision.
- void [Rsdk_AU_sortdOET_Viof32p8_k2](#) (xb_vecN_2xf32 *pInOut, uint32_t size)
Sort descending - real single precision.
- void [Rsdk_AU_sortOET_Viof32p8_k2](#) (xb_vecN_2xf32 *pInOut, uint32_t size)
Sort ascending - real single precision.
- void [Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput1, xb_vecN_2xf32 *pOutput2, xb_vecN_2xf32 *pInput1, xb_vecN_2xf32 *pInput2, uint32_t size)
Element-wise compare and compute minimum and maximum values - real single precision.

3.46.1 Detailed Description

3.46.2 Function Documentation

3.46.2.1 Rsdk_AU_sortdOET_Viof32p8_k2()

```
void Rsdk_AU_sortdOET_Viof32p8_k2 (
    xb_vecN_2xf32 * pInOut,
    uint32_t size )
```

Sort descending - real single precision.

Function sorts descending real data vectors using Odd-Even transposition or Odd-Even Sort.

Parameters

in, out	pInOut	Result/operand contains 8 parallel real vectors unsorted at the beginning of the function and sorted descending at the end, size N.
in	size	Size of inout vectors, N.

Note

sortd (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.46.2.2 Rsdk_AU_sortdOET_Vof32p8Vif32p8()

```
void Rsdk_AU_sortdOET_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sort descending - real single precision.

Function sorts descending real data vectors using Odd-Even transposition or Odd-Even Sort.
Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel sorted descending real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

sortd (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.46.2.3 Rsdk_AU_sortOET_Viof32p8_k2()

```
void Rsdk_AU_sortOET_Viof32p8_k2 (
    xb_vecN_2xf32 * pInOut,
    uint32_t size )
```

Sort ascending - real single precision.

Function sorts ascending real data vectors using Odd-Even transposition or Odd-Even Sort.

Parameters

in, out	<i>pInOut</i>	Result/operand contains 8 parallel real vectors unsorted at the beginning of the function and sorted ascending at the end, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

sorta (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.46.2.4 Rsdk_AU_sortOET_Vof32p8Vif32p8()

```
void Rsdk_AU_sortOET_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 * pInput,
    uint32_t size )
```

Sort ascending - real single precision.

Function sorts ascending real data vectors using Odd-Even transposition or Odd-Even Sort. Output must not be in-place memory vector.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel sorted ascending real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

sorta (vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.46.2.5 Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8()

```
void Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput1,
    xb_vecN_2xf32 * pOutput2,
    xb_vecN_2xf32 * pInput1,
    xb_vecN_2xf32 * pInput2,
    uint32_t size )
```

Element-wise compare and compute minimum and maximum values - real single precision.

Compare each element from the first input vector with the element at the same position from the second input vector, minimum result is copied in first output vector and maximum is copied in second output vector. Output must not be in-place memory vector.

Parameters

out	<i>pOutput1</i>	Result contains 8 parallel real vectors, representing the minimum values, size N.
out	<i>pOutput2</i>	Result contains 8 parallel real vectors, representing the maximum values, size N.
in	<i>pInput1</i>	Operand contains 8 parallel real vectors, size N.
in	<i>pInput2</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, N.

Note

minMax (vector, vector) = (vector, vector)
 v1 = a1, a2..., an;
 v2 = b1, b2..., bn;
 minMax1 (v1,v2) = (min (a1,b1), ..., min (an,bn);
 minMax2 (v1,v2) = (max (a1,b1), ..., max (an,bn);
 minMax = (minMax1, minMax2) The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 2000$.

3.47 Dsp_lal_api_func_sp_smooth

Functions

- void [RsdK_ML_spatialSm_Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 const *pInput, uint32_t rowsIn, uint32_t colsIn, uint32_t rowsWin, uint32_t colsWin)
2D Spatial smoothing correlation matrix construction for 8 parallel streams of matrices size NxM - complex single precision.

3.47.1 Detailed Description

3.47.2 Function Documentation

3.47.2.1 RsdK_ML_spatialSm_Mof32cp8Mif32cp8()

```

void RsdK_ML_spatialSm_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t rowsIn,
    uint32_t colsIn,
    uint32_t rowsWin,
    uint32_t colsWin )

```

2D Spatial smoothing correlation matrix construction for 8 parallel streams of matrices size NxM - complex single precision.

The function performs 2D Spatial smoothing required for ESPRIT algorithm implementation and outputs the correlation matrix between smoothed vectors.

The output matrix is constructed by sliding the matrix window, size PxQ, over the input antenna array. A vector is created for each position by concatenating the window columns, and its outer product ($v * v^H$) is computed. All outer products are accumulated in the output correlation matrix.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel square complex matrices, size (P*Q)x(P*Q).
in	<i>pInput</i>	First operand contains 8 parallel matrixes, size NxM.
in	<i>rowsIn</i>	Number of rows of the input matrix (N).
in	<i>colsIn</i>	Number of columns of the input matrix (M).
in	<i>rowsWin</i>	Number of rows of the smoothing window (P).
in	<i>colsWin</i>	Number of columns of the smoothing window (Q).

Note

The maximum size for the output matrix is 35x35 defined in MAX_SP_SM_OUT_SZ.

The maximum input matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \times M \leq 2800$.

rowsIn range is [2..N]

colsIn range is [2..M]

rowsWin range is [2..rowsIn]

colsWin range is [2..colsIn]

3.48 Dsp_lal_func_api_dist

Functions

- void [Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput1, xb_vecN_2xf32 const *pInput2, float32_t const *pInput3)

Squared Normalized Euclidean distance between two vectors with 2 variables.

- void [Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32](#) (xb_vecN_2xf32 *pOutput, float32_t const *pInput1, xb_vecN_2xf32 const *pInput2, float32_t const *pInput3)

Squared Normalized Euclidean distance between one observation with 2 variables and multiple references - real single precision.

3.48.1 Detailed Description

3.48.2 Function Documentation

3.48.2.1 Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32()

```
void Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32 (
    xb_vecN_2xf32 * pOutput,
    float32_t const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    float32_t const * pInput3 )
```

Squared Normalized Euclidean distance between one observation with 2 variables and multiple references - real single precision.

Compute order 2 squared Normalized Euclidean distance between one observation and multiple references of the same distribution with the inverse covariance matrix S pre-calculated and diagonal.

First input containf one observation.

Second input 8 parallel streams of references.

Third input contains pre-calculated symmetric inverse diagonal covariance matrix S stored as a vector c00, c11.

Output vector contains 8 scalar squared distances from the selected observation.

Parameters

out	<i>pOutput</i>	Result contains real vector size 8.
in	<i>pInput1</i>	First operand contains real vector size 2.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors size 2.
in	<i>pInput3</i>	Third operand contains real vector size 2.

Note

```

mahal2 (vector, vector, vector) = vector
v1 = obs_x, obs_y;
size(v1) = 1 x 2;
v2 = ref_x0, ref_x1, ..., ref_x7;
ref_y0, ref_y1, ..., ref_y7;
size(v2) = 8 x 2;
v3 = c00, c11; covariance diagonal
size(v3) = 2;
d1j = d((obs_x, obs_y), (ref_xj, ref_yj)) = (obs_x1-ref_xj)^2 * c00 + (obs_y1-ref_yj)^2 * c11; mahal2 (v1,v2,v3)
= d11, d12, ..., d18;

```

3.48.2.2 Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32()

```

void Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    float32_t const * pInput3 )

```

Squared Normalized Euclidean distance between two vectors with 2 variables.

- real single precision.

Squared Normalized Euclidean distance between two vectors of the same distribution with the inverse covariance matrix S pre-calculated and diagonal.

Input1 vector contains 8 parallel observations for 2 variables.

Input2 vector contains 8 parallel references for 2 variables.

Input3 contains pre-calculated symmetric inverse diagonal covariance matrix S stored as a vector c00, c11.

Output vector contains squared distances from first to second vector.

Parameters

out	<i>pOutput</i>	Result contains real vector size 8.
in	<i>pInput1</i>	First operand contains 8 parallel real vectors size 2.
in	<i>pInput2</i>	Second operand contains 8 parallel real vectors size 2.
in	<i>pInput3</i>	Third operand contains real vector size 2.

Note

```

mahal (vector, vector) = (vector)
v1 = obs_x1, obs_x2, ..., obs_x8;
obs_y1, obs_y2, ..., obs_y8;
v2 = ref_x1, ref_x2, ..., ref_x8;
ref_y1, ref_y2, ..., ref_y8;
s = c00, c11; covariance diagonal
di,j = (obs_xi-ref_xj)^2 * c00 + (obs_yi-ref_yj)^2 * c11;
mahal (v1,v2) = d1,1, d2,2, ..., d8,8;

```

3.49 Dsp_lal_api_func_prod_tot

Functions

- void [Rsdk_LA_prod_Sof32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput, uint32_t size)
Product of all vector elements for 8 parallel streams of vectors, size N - real single precision.
- void [Rsdk_LA_prod_Sof32cp8Vif32cp8](#) (xb_vecN_2xcf32 *pOutput, xb_vecN_2xcf32 const *pInput, uint32_t size)
Product of all vector elements for 8 parallel streams of vectors, size N - complex single precision.

3.49.1 Detailed Description

3.49.2 Function Documentation

3.49.2.1 Rsdk_LA_prod_Sof32cp8Vif32cp8()

```

void Rsdk_LA_prod_Sof32cp8Vif32cp8 (
    xb_vecN_2xcf32 * pOutput,
    xb_vecN_2xcf32 const * pInput,
    uint32_t size )

```

Product of all vector elements for 8 parallel streams of vectors, size N - complex single precision.

All elements in each vector are multiplied resulting a total product.

Parameters

out	<i>pOutput</i>	Result contains 8 complex scalars(the product of vectors' elements).
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Number of elements in vector (N).

Note

element eprod (vector) = scalar

n = elem_cnt

v = n, a1, a2..., an;

eprod(v) = a1 * a2 * ... * an; The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 4000.

3.49.2.2 Rsdk_LA_prod_Sof32p8Vif32p8()

```
void Rsdk_LA_prod_Sof32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput,
    uint32_t size )
```

Product of all vector elements for 8 parallel streams of vectors, size N - real single precision.

All elements in each vector are multiplied resulting a total product.

Parameters

out	<i>pOutput</i>	Result contains 8 real scalars(the product of vectors' elements).
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Number of elements in vector (N).

Note

element eprod (vector) = scalar

n = elem_cnt

v = n, a1, a2..., an;

eprod(v) = a1 * a2 * ... * an; The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 8000.

3.50 Dsp_lal_api_func_hessenberg_mat**Functions**

- void [Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8](#) (xb_vecN_2xf32 *pOutputP, xb_vecN_2xf32 *pOutputH, xb_vecN_2xf32 *pInput, xb_vecN_2xf32 *pBuffer, uint32_t size)
Upper Hessenberg reduction for NxN matrices - real single precision.
- void [Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8](#) (xb_vecN_2xcf32 *pOutputP, xb_vecN_2xcf32 *pOutputH, xb_vecN_2xcf32 *pInput, xb_vecN_2xcf32 *pBuffer, uint32_t size)
Upper Hessenberg reduction for NxN matrices - complex single precision.

3.50.1 Detailed Description**3.50.2 Function Documentation**

3.50.2.1 Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8()

```
void Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 * pOutputP,
    xb_vecN_2xcf32 * pOutputH,
    xb_vecN_2xcf32 * pInput,
    xb_vecN_2xcf32 * pBuffer,
    uint32_t size )
```

Upper Hessenberg reduction for NxN matrices - complex single precision.

Function reduces a complex general purpose matrix to the upper Hessenberg form using Householder reflection or Householder transformations.

$pOutputP * pOutputH * pOutputP^T = pInput$

$pOutputH$ and $pInput$ can point to the same address.

Parameters

out	<i>pOutputP</i>	Result contains 8 parallel orthogonal complex matrices, size N. pOutputP is a unitary matrix.
out	<i>pOutputH</i>	Result contains 8 parallel upper Hessenberg complex matrices, size N. They have zero entries below the first subdiagonal.
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size N.
in, out	<i>pBuffer</i>	Scratch area contains 8 parallel complex vectors, size N.
in	<i>size</i>	Size of input and output vectors, $N \geq 3$.

Note

$upHess(matrix) = (matrixP, matrixH)$

The maximum matrix/vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 35$.

3.50.2.2 Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8()

```
void Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8 (
    xb_vecN_2xf32 * pOutputP,
    xb_vecN_2xf32 * pOutputH,
    xb_vecN_2xf32 * pInput,
    xb_vecN_2xf32 * pBuffer,
    uint32_t size )
```

Upper Hessenberg reduction for NxN matrices - real single precision.

Function reduces a real general purpose matrix to the upper Hessenberg form using Householder reflection or Householder transformations.

$pOutputP * pOutputH * pOutputP^T = pInput$

$pOutputH$ and $pInput$ can point to the same address.

Parameters

out	<i>pOutputP</i>	Result contains 8 parallel orthogonal real matrices, size N. pOutputP is an orthogonal matrix.
out	<i>pOutputH</i>	Result contains 8 parallel upper Hessenberg real matrices, size N. They have zero entries below the first subdiagonal.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size N.
in, out	<i>pBuffer</i>	Scratch area contains 8 parallel real vectors, size N.
in	<i>size</i>	Size of input and output vectors, $N \geq 3$.

Note

upHess (matrix) = (matrixP, matrixH)

The maximum matrix/vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 50$.

3.51 Dsp_lal_api_func_copy

Functions

- void [Rsdk_DT_copy_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *__restrict pInput, uint32_t size)
Copy 8 parallel streams of vectors - real single precision.
- void [Rsdk_DT_copy_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *__restrict pInput, uint32_t size)
Copy 8 parallel streams of vectors - complex single precision.
- void [Rsdk_DT_copy_Mof32p8Mif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *__restrict pInput, uint32_t rows, uint32_t cols)
Copy matrices - real single precision.
- void [Rsdk_DT_copy_Mof32cp8Mif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 *__restrict pInput, uint32_t rows, uint32_t cols)
Copy matrices - complex single precision.
- void [Rsdk_DT_copyOffset_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *__restrict pInput, uint32_t numElem, uint32_t outOffset, uint32_t inOffset)
Copy a number of vector elements from an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer with/without offsets - real single precision.
- void [Rsdk_DT_copyOffset_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *__restrict pInput, uint32_t numElem, uint32_t outOffset, uint32_t inOffset)
Copy a number of vector elements from an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer with/without offsets - complex single precision.
- void [Rsdk_DT_copyInOffset_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *__restrict pInput, uint32_t numElem, uint32_t inOffset)
Copy a number of vector elements from an 8 parallel streams buffer starting with an offset at the beginning of the vectors in another 8 parallel streams buffer - real single precision.
- void [Rsdk_DT_copyInOffset_Vof32cp8Vif32cp8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *__restrict pInput, uint32_t numElem, uint32_t inOffset)
Copy a number of vector elements from an 8 parallel streams buffer starting with an offset at the beginning of the vectors in another 8 parallel streams buffer - complex single precision.
- void [Rsdk_DT_copyOutOffset_Vof32p8Vif32p8](#) (xb_vecN_2xf32 *__restrict pOutput, xb_vecN_2xf32 const *__restrict pInput, uint32_t numElem, uint32_t outOffset)

Copy a number of elements from the vectors in an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer starting with an offset - real single precision.

- void [Rsdk_DT_copyOutOffset_Vof32cp8Vif32cp8](#) (xb_vecN_2xcf32 *__restrict pOutput, xb_vecN_2xcf32 const *__restrict pInput, uint32_t numElem, uint32_t outOffset)

Copy a number of elements from the vectors in an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer starting with an offset - complex single precision.

3.51.1 Detailed Description

3.51.2 Function Documentation

3.51.2.1 RsdK_DT_copy_Mof32cp8Mif32cp8()

```
void RsdK_DT_copy_Mof32cp8Mif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 *__restrict pInput,
    uint32_t rows,
    uint32_t cols )
```

Copy matrices - complex single precision.

Function copy the values from input matrices into the output matrices.
Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex matrices, size NxM.
in	<i>pInput</i>	Operand contains 8 parallel complex matrices, size NxM.
in	<i>rows</i>	Rows of input and output matrices, N.
in	<i>cols</i>	Columns of input and output matrices, M.

Note

copy(matrix) = matrix

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: NxM ≤ 2000.

3.51.2.2 RsdK_DT_copy_Mof32p8Mif32p8()

```
void RsdK_DT_copy_Mof32p8Mif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 *__restrict pInput,
```

```
uint32_t rows,
uint32_t cols )
```

Copy matrices - real single precision.

Function copy the values from input matrices into the output matrices.
Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real matrices, size NxM.
in	<i>pInput</i>	Operand contains 8 parallel real matrices, size NxM.
in	<i>rows</i>	Rows of input and output matrices, N.
in	<i>cols</i>	Columns of input and output matrices, M.

Note

copy(matrix) = matrix

The maximum matrix size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: NxM <= 4000.

3.51.2.3 Rsdk_DT_copy_Vof32cp8Vif32cp8()

```
void Rsdk_DT_copy_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 *__restrict pInput,
    uint32_t size )
```

Copy 8 parallel streams of vectors - complex single precision.

Copies the vectors from an 8 parallel streams vectors buffer, pInput, to another 8 parallel streams vectors buffer, pOutput. Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel complex vectors, size N.
in	<i>size</i>	Number of elements in vector (N).

Note

copy(vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 2000.

3.51.2.4 RsdK_DT_copy_Vof32p8Vif32p8()

```
void RsdK_DT_copy_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 *__restrict pInput,
    uint32_t size )
```

Copy 8 parallel streams of vectors - real single precision.

Copies the vectors from an 8 parallel streams vectors buffer, pInput, to another 8 parallel streams vectors buffer, pOutput. Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size N.
in	<i>pInput</i>	Operand contains 8 parallel real vectors, size N.
in	<i>size</i>	Number of elements in vector (N).

Note

copy(vector) = vector

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 4000$.

3.51.2.5 RsdK_DT_copyInOffset_Vof32cp8Vif32cp8()

```
void RsdK_DT_copyInOffset_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t inOffset )
```

Copy a number of vector elements from an 8 parallel streams buffer starting with an offset at the beginning of the vectors in another 8 parallel streams buffer - complex single precision.

Copies numElem elements from the vectors in pInput buffer, starting from inOffset position, to pOutput buffer vectors, starting with the first element. Output vectors' elements in range [0, numElem - 1] are overwritten by numElem elements from the corresponding input vector, starting with inOffset index: [inOffset, inOffset + numElem - 1].

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is numElem and the output's length is $P < \text{numElem}$ and/or the input's length is $N < \text{inOffset} + \text{numElem}$, the copying will continue until all numElem elements are copied regardless of buffers' sizes.

If inOffset is 0, copying will start from the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vector size P.
in	<i>pInput</i>	Operand contains 8 parallel complex vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>inOffset</i>	Input vectors' element index from which the elements will be copied.

Note

copy(input, output) = output which contains the numElem elements from the input buffer, starting at inOffset, from outOffset onwards.

in = a0, a1, ..., a(N-1);

out = b0, b1, ..., b(P-1);

copy(in,out) = a(inOffset), a(inOffset+1), ..., a(inOffset+numElem-1), b(numElem), b(numElem+1),

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's dimensionality is m x n, it is indexed as a (m * n) size vector, from 0 to (m * n - 1). The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N+P <= 4000.
numElem range is [1..minimum(N,P)]
inOffset range is [0..N]

3.51.2.6 Rsdk_DT_copyInOffset_Vof32p8Vif32p8()

```
void Rsdk_DT_copyInOffset_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t inOffset )
```

Copy a number of vector elements from an 8 parallel streams buffer starting with an offset at the beginning of the vectors in another 8 parallel streams buffer - real single precision.

Copies numElem elements from the vectors in pInput buffer, starting from inOffset position, to pOutput buffer vectors, starting with the first element. Output vectors' elements in range [0, numElem - 1] are overwritten by numElem elements from the corresponding input vector, starting with inOffset index: [inOffset, inOffset + numElem - 1].

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is numElem and the output's length is P < numElem and/or the input's length is N < inOffset + numElem, the copying will continue until all numElem elements are copied regardless of buffers' sizes.

If inOffset is 0, copying will start from the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size P.
in	<i>pInput</i>	Operand contains 8 parallel real vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>inOffset</i>	Input vectors' element index from which the elements will be copied.

Note

`copy(input, output)` = output which contains the `numElem` elements from the input buffer, starting at `inOffset`, from `outOffset` onwards.

`in` = `a0, a1, ..., a(N-1)`;

`out` = `b0, b1, ..., b(P-1)`;

`copy(in,out)` = `a(inOffset), a(inOffset+1), ..., a(inOffset+numElem-1), b(numElem), b(numElem+1), .....`

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's dimensionality is $m \times n$, it is indexed as a $(m * n)$ size vector, from 0 to $(m * n - 1)$.
The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N+P \leq 8000$.
`numElem` range is $[1..\text{minimum}(N,P)]$
`inOffset` range is $[0..N]$

3.51.2.7 RsdK_DT_copyOffset_Vof32cp8Vif32cp8()

```
void RsdK_DT_copyOffset_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t outOffset,
    uint32_t inOffset )
```

Copy a number of vector elements from an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer with/without offsets - complex single precision.

Copies `numElem` elements from the vectors in `pInput` buffer, starting with `inOffset` position, to the `pOutput` buffer vectors, starting with the `outOffset` position. Output vectors' elements in range `[outOffset, outOffset + numElem - 1]` are overwritten by `numElem` elements from the corresponding input vector, starting with `inOffset` index: `[inOffset, inOffset + numElem - 1]`.

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is `numElem`, the output's length is $P < \text{numElem} + \text{outOffset}$ and/or the input's length is $N < \text{numElem} + \text{inOffset}$, the copying will continue until all `numElem` input elements are copied regardless of buffers' sizes.

If `inOffset` is 0, copying will start from the first element of the vectors.

If `outOffset` is 0, writing will start at the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vector size P.
in	<i>pInput</i>	Operand contains 8 parallel complex vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>outOffset</i>	Output vectors' element index where the input vector elements will be written.
in	<i>inOffset</i>	Input vectors' element index from which the elements will be copied.

Note

copy(input, output) = output which contains the numElem elements from the input buffer, starting at inOffset, from outOffset onwards.

in = a0, a1, ..., a(N-1);

out = b0, b1, ..., b(P-1);

copy(in,out) = b0, b1, b2,, b(outOffset-2), b(outOffset-1), a(inOffset), a(inOffset+1) ..., a(inOffset+numElem-1), b(outOffset-1+numElem), b(outOffset+numElem), ...

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's dimensionality is m x n, it is indexed as a (m * n) size vector, from 0 to (m * n - 1). The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N+P <= 4000.
numElem range is [1..minimum(N,P)]
outOffset range is [0..P]
inOffset range is [0..N]

3.51.2.8 RsdK_DT_copyOffset_Vof32p8Vif32p8()

```
void RsdK_DT_copyOffset_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t outOffset,
    uint32_t inOffset )
```

Copy a number of vector elements from an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer with/without offsets - real single precision.

Copies numElem elements from the vectors in pInput buffer, starting with inOffset position, to the pOutput buffer vectors, starting with the outOffset position. Output vectors' elements in range [outOffset, outOffset + numElem - 1] are overwritten by numElem elements from the corresponding input vector, starting with inOffset index: [inOffset, inOffset + numElem - 1].

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is numElem, the output's length is $P < \text{numElem} + \text{outOffset}$ and/or the input's length is $N < \text{numElem} + \text{inOffset}$, the copying will continue until all numElem input elements are copied regardless of buffers' sizes.

If inOffset is 0, copying will start from the first element of the vectors.

If outOffset is 0, writing will start at the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size P.
in	<i>pInput</i>	Operand contains 8 parallel real vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>outOffset</i>	Output vectors' element index where the input vector elements will be written.
in	<i>inOffset</i>	Input vectors' element index from which the elements will be copied.

Note

copy(input, output) = output which contains the numElem elements from the input buffer, starting at inOffset, from outOffset onwards.

in = a0, a1, ..., a(N-1);

out = b0, b1, ..., b(P-1);

copy(in,out) = b0, b1, b2,, b(outOffset-2), b(outOffset-1), a(inOffset), a(inOffset+1) ..., a(inOffset+numElem-1), b(outOffset-1+numElem), b(outOffset+numElem), ...

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's dimensionality is m x n, it is indexed as a (m * n) size vector, from 0 to (m * n - 1).
The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N+P \leq 8000$.
numElem range is [1.. minimum(N,P)]
outOffset range is [0..P]
inOffset range is [0..N]

3.51.2.9 RsdK_DT_copyOutOffset_Vof32cp8Vif32cp8()

```
void RsdK_DT_copyOutOffset_Vof32cp8Vif32cp8 (
    xb_vecN_2xcf32 *__restrict pOutput,
    xb_vecN_2xcf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t outOffset )
```

Copy a number of elements from the vectors in an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer starting with an offset - complex single precision.

Copies numElem elements from the vectors in the pInput buffer, starting with the first element, to the pOutput buffer vectors, starting with the outOffset position. Output vectors' elements in range [outOffset, outOffset + numElem - 1] are overwritten by numElem elements from the corresponding input vector, starting with inOffset index: [0, numElem - 1].

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is numElem and the output's useful length is $P < \text{numElem} + \text{outOffset}$ and the input's useful length is $N < \text{numElem}$, the copying will continue until all numElem input elements are copied regardless of buffers' sizes.

If outOffset is 0, writing will start at the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel complex vector size P.
in	<i>pInput</i>	Operand contains 8 parallel complex vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>outOffset</i>	Output vectors' element index where the input vector elements will be written.

Note

copy(input, output) = output which contains the numElem elements from the input buffer, starting at inOffset, from outOffset onwards.

in = a0, a1, ..., a(N-1);

out = b0, b1, ..., b(P-1);

copy(in,out) = b0, b1,, b(outOffset-1), a(inOffset), ..., a(inOffset+numElem-1), ..

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's (useful) dimensionality is $m \times n$, it is indexed as a $(m * n)$ size vector, from 0 to $(m * n - 1)$. The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N+P \leq 4000$.
numElem range is $[1..\text{minimum}(N,P)]$
outOffset range is $[0..P]$

3.51.2.10 RsdK_DT_copyOutOffset_Vof32p8Vif32p8()

```
void RsdK_DT_copyOutOffset_Vof32p8Vif32p8 (
    xb_vecN_2xf32 *__restrict pOutput,
    xb_vecN_2xf32 const *__restrict pInput,
    uint32_t numElem,
    uint32_t outOffset )
```

Copy a number of elements from the vectors in an 8 parallel streams buffer to the vectors in another 8 parallel streams buffer starting with an offset - real single precision.

Copies numElem elements from the vectors in the pInput buffer, starting with the first element, to the pOutput buffer vectors, starting with the outOffset position. Output vectors' elements in range [outOffset, outOffset + numElem - 1] are overwritten by numElem elements from the corresponding input vector, starting with inOffset index: [0, numElem - 1].

Both buffers are treated as linear infinite memory zones. This means that if the payload to copy is numElem, the output's useful length is $P < \text{numElem} + \text{outOffset}$ and the input's useful length is $N < \text{numElem}$, the copying will continue until all numElem input elements are copied regardless of buffers' sizes.

If outOffset is 0, writing will start at the first element of the vectors.

Output and input must not be in-place memory.

Parameters

out	<i>pOutput</i>	Result contains 8 parallel real vectors, size P.
in	<i>pInput</i>	Operand contains 8 parallel real vector size N.
in	<i>numElem</i>	Number of elements to copy from input vectors to output vectors.
in	<i>outOffset</i>	Output vectors' element index where the input vector elements will be written.

Note

copy(input, output) = output which contains the numElem elements from the input buffer, starting at inOffset, from outOffset onwards.

in = a0, a1, ..., a(N-1);

out = b0, b1, ..., b(P-1);

copy(in,out) = b0, b1,, b(outOffset-1), a(inOffset), ..., a(inOffset+numElem-1), ..

A practical use-case is copying elements from a vector input to a matrix output. For this example, note that:

- Dimensions for both matrix and vector are disregarded.
- If the matrix's (useful) dimensionality is $m \times n$, it is indexed as a $(m * n)$ size vector, from 0 to $(m * n - 1)$. The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N+P \leq 8000$.
numElem range is $[1..\text{minimum}(N,P)]$
outOffset range is $[0..P]$

3.52 Dsp_lal_api_func_polinom

Functions

- void [Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput1, xb_vecN_2xf32 const *pInput2, uint32_t degree)
Polynoms evaluation(the 8 polynoms are defined by 8 parallel streams of real coefficient vectors, size N+1) for 8 corresponding variable's values - real single precision.

3.52.1 Detailed Description

3.52.2 Function Documentation

3.52.2.1 Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8()

```
void Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    uint32_t degree )
```

Polynoms evaluation(the 8 polynoms are defined by 8 parallel streams of real coefficient vectors, size N+1) for 8 corresponding variable's values - real single precision.

The function evaluates the polynomial functions for a real value of the variable.

Each polynomial function is defined by the vector of the real values of the coefficients arranged in descending order.

The polynoms have the same degree, N.

Parameters

out	<i>pOutput</i>	Result contains the 8 parallel evaluation results P(x), real scalars.
in	<i>pInput1</i>	First operand contains 8 parallel real coefficients' vectors, size N+1.
in	<i>pInput2</i>	Second operand contains 8 real scalar variables.
in	<i>degree</i>	The polinomyals' degree, N.

Note

The maximum vector size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: $N \leq 8000$.

3.53 Dsp_lal_api_func_snr

Functions

- void [Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8](#) (xb_vecN_2xf32 *pOutput, xb_vecN_2xf32 const *pInput1, xb_vecN_2xf32 const *pInput2, uint32_t size)
Element-wise signal-to-noise ratio or SNR in dB - real single precision.

3.53.1 Detailed Description

3.53.2 Function Documentation

3.53.2.1 Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8()

```
void Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8 (
    xb_vecN_2xf32 * pOutput,
    xb_vecN_2xf32 const * pInput1,
    xb_vecN_2xf32 const * pInput2,
    uint32_t size )
```

Element-wise signal-to-noise ratio or SNR in dB - real single precision.

Compute Signal to noise ratio of signal and noise real vectors.

$output[k] = 10 \cdot \log_{10}(|input1[k]/input2[k]|)$

Parameters

out	<i>pOutput</i>	Result real vector size N.
in	<i>pInput1</i>	First operand real vector size N.
in	<i>pInput2</i>	Second operand real vector size N.
in	<i>size</i>	Number of elements in vector (N).

Note

signal-to-noise ratio (vector, vector) = vector
v1 = n, a1, a2, ..., an;
v2 = n, b1, b2, ..., bn;
SNR (v1,v2) = n, 10*log10(|a1/b1|),... ,10*log10(|an/bn|);
The maximum vectors' size is estimated in the ideal condition that the function use the whole memory of 256 kBytes: N <= 2730.

3.54 Dsp_lal_demos_api_func_add

Functions

- void [RsdkBbe32LalVectorAdd](#) (uintptr_t argList)
Function to call DSP LAL library functions All the functions from DSP LAL are grouped inside this function to show their integration with thw DSP Dispatcher.

3.54.1 Detailed Description

3.54.2 Function Documentation

3.54.2.1 RsdkBbe32LalVectorAdd()

```
void RsdkBbe32LalVectorAdd (  
    uintptr_t argList )
```

Function to call DSP LAL library functions All the functions from DSP LAL are grouped inside this function to show their integration with thw DSP Dispatcher.

Parameters

in	argList	- pointer to a data structure of type rsdkBbe32LalVectorAddParams_t containing the DSP LAL parameters and buffers.
----	---------	--

Chapter 4

Data Structure Documentation

4.1 dspDispCommParams_t Struct Reference

This structure is used to initialize the IPCF communication parameters used between the Dispatcher and the Host Driver.

```
#include <rsdk_dsp_dispatcher.h>
```

Data Fields

- uint8_t [ipcfInstance](#)
- uint8_t [ctrlChanId](#)
- uint8_t [asyncChanId](#)

4.1.1 Detailed Description

This structure is used to initialize the IPCF communication parameters used between the Dispatcher and the Host Driver.

They should be initialized by the user with the corresponding values defined in `ipcf_lp_Cfg.h`

4.1.2 Field Documentation

4.1.2.1 asyncChanId

```
uint8_t asyncChanId
```

IPCF Async Channel ID

4.1.2.2 ctrlChanId

```
uint8_t ctrlChanId
```

IPCF Control Channel ID

4.1.2.3 ipcfInstance

```
uint8_t ipcfInstance
```

IPCF instance ID

4.2 dspDisplnit_t Struct Reference

Initialization parameters for the Dispatcher.

```
#include <rsdk_dsp_dispatcher.h>
```

Data Fields

- [rsdkDsplsCb_t sptEvtlsrCb](#)
- [uint16_t sptEvtIntEnBitmap](#)
- [dspDispCommParams_t commParams](#)
- [rsdkDspFuncPtr_t pFuncTable](#)
- [uint32_t funcTableSize](#)
- [dspPrefetchAggrParams_t prefAggrParams](#)

4.2.1 Detailed Description

Initialization parameters for the Dispatcher.

This structure is only used by `RsdkSptInit()`. Must be allocated and initialized by the caller. Its content is not modified by the Dispatcher.

4.2.2 Field Documentation

4.2.2.1 commParams

```
dspDispCommParams\_t commParams
```

Communication parameters used between the Dispatcher and the Host Driver

4.2.2.2 funcTableSize

`uint32_t funcTableSize`

number of functions registered in the table

4.2.2.3 pFuncTable

`rsdkDspFuncPtr_t pFuncTable`

pointer to a table containing pointers to external DSP functions. The table itself is built starting from a list of function names, using the preprocessor. The list of function names is shared between the DSP and the host CPU which integrates the DSP image (e.g. A53). See /Apps/DSP_Dispatcher_example/src/dispatcher_func_list_dsp.h

4.2.2.4 prefAggrParams

`dspPrefetchAggrParams_t prefAggrParams`

prefetch aggressiveness parameters

4.2.2.5 sptEvtIntEnBitmap

`uint16_t sptEvtIntEnBitmap`

Allows the user to enable the SPT Event interrupts individually. Can be set by adding together the `RSDK_DSP_SPT_EVT0_INTEN_BIT.. RSDK_DSP_SPT_EVT12_INTEN_BIT` values

4.2.2.6 sptEvtIsrCb

`rsdkDspIsrCb_t sptEvtIsrCb`

Callback function to be called from the SPT Event interrupt handlers. Receives the event number as argument. If no callback is required then it can be initialized to NULL.

4.3 Dsphd_CommandAckType Struct Reference

This structure packs the information needed to send an ACK from the DSP Dispatcher to the core .

```
#include <Dsphd_Types.h>
```

Data Fields

- `uint32 commandNumber`
- `Dsphd_MsgTypeType commandType`

4.3.1 Detailed Description

This structure packs the information needed to send an ACK from the DSP Dispatcher to the core .

4.3.2 Field Documentation

4.3.2.1 commandNumber

```
uint32 commandNumber
```

Number of the command that receives ACK

4.3.2.2 commandType

```
Dsphd_MsgTypeType commandType
```

Specifies the command type that has been ACKed

4.4 Dsphd_JobType Struct Reference

This structure packs the information needed to call a function remotely on the BBE32 DSP, through the DSP Dispatcher .

```
#include <Dsphd_Types.h>
```

Data Fields

- uint8 [id](#)
- uint32 [arg](#)

4.4.1 Detailed Description

This structure packs the information needed to call a function remotely on the BBE32 DSP, through the DSP Dispatcher .

4.4.2 Field Documentation

4.4.2.1 arg

uint32 arg

Job argument. Can be either an immediate value or a 32-bit pointer to a data structure.

4.4.2.2 id

uint8 id

Job ID returned from RSDK_DSP_GET_FUNC_ID(function_name)

4.5 Dsphd_RuntimeCfgType Struct Reference

This structure packs the information needed to configure the runtime configuration of the DSP Dispatcher .

```
#include <Dsphd_Types.h>
```

Data Fields

- uint16 [numberOfChirps](#)
- [Dsphd_p1TaskConfigType](#) p1TaskConfig
- [Dsphd_fpuRoundConfigType](#) fpuRoundConfig
- volatile [Dsphd_DispatcherStateType](#) * pDispatcherState
- uint8 [reportFPUExceptions](#)
- uint8 [crc](#)

4.5.1 Detailed Description

This structure packs the information needed to configure the runtime configuration of the DSP Dispatcher .

It must be used as payload to the [DSPHD_MSG_UPDATE_DISPATCHER_CONFIG](#) message type

4.5.2 Field Documentation

4.5.2.1 crc

uint8 crc

CRC on this struct

4.5.2.2 fpuRoundConfig

`Dsphd_fpuRoundConfigType fpuRoundConfig`

Configure the behavior of the DSP via FCR register for rounding

4.5.2.3 numberOfChirps

`uint16 numberOfChirps`

Number of chirps expected from LINE_DONE interrupt after which all interrupts are re-enabled and Dispatcher exists P1

4.5.2.4 p1TaskConfig

`Dsphd_p1TaskConfigType p1TaskConfig`

Specifies P1 task operation via Dsphd_p1TaskConfigType enum

4.5.2.5 pDispatcherState

`volatile Dsphd_DispatcherStateType* pDispatcherState`

4.5.2.6 reportFPUExceptions

`uint8 reportFPUExceptions`

Global shared location where the DSP Dispatcher can report information about the DSP Dispatcher state Poll and throw FPU errors to the core after each BBE32 kernel; 0 means disabled; 1 means enabled

4.6 Dsphd_TraceCfgType Struct Reference

This structure packs the information needed to configure and start application trace logging in the DSP Dispatcher .

```
#include <Dsphd_Types.h>
```

Data Fields

- `uint8 * bufAddr`
- `uint16 bufSize`

4.6.1 Detailed Description

This structure packs the information needed to configure and start application trace logging in the DSP Dispatcher .

4.6.2 Field Documentation

4.6.2.1 bufAddr

```
uint8* bufAddr
```

Address of the trace log buffer

4.6.2.2 bufSize

```
uint16 bufSize
```

Size of the trace log buffer in bytes

4.7 dspPrefetchAggrParams_t Struct Reference

This structure is used to initialize the prefetch aggression parameters for HW Data, Hw Instr, Sw.

```
#include <rsdk_dsp_dispatcher.h>
```

Data Fields

- `uint8_t` [prefetchAggressionData](#)
- `uint8_t` [prefetchAggressionInstr](#)
- `uint8_t` [prefetchAggressionSW](#)

4.7.1 Detailed Description

This structure is used to initialize the prefetch aggression parameters for HW Data, Hw Instr, Sw.

They should be initialized by the user

4.7.2 Field Documentation

4.7.2.1 prefetchAggressionData

```
uint8_t prefetchAggressionData
```

level of prefetch aggression for data ; 0xFu -> most aggressive

4.7.2.2 prefetchAggressionInstr

```
uint8_t prefetchAggressionInstr
```

level of prefetch aggression for instruction ; 0xFu -> most aggressive

4.7.2.3 prefetchAggressionSW

```
uint8_t prefetchAggressionSW
```

level of prefetch aggression for SW ; 0xFu -> most aggressive

4.8 rsdkBbe32CaCfarParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_s32r41.h>
```

Data Fields

- uint16_t [rangeBins](#)
- uint16_t [dopplerBins](#)
- uint8_t [windowSize](#)
- uint8_t [guardInterval](#)
- uint16_t [thresholdFactor](#)
- uint32_t [pRdm](#)
- uint32_t [pThr](#)
- uint32_t [pDetectBitmap](#)

4.8.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for BBE32 radar signal processing algorithms.

4.8.2 Field Documentation

4.8.2.1 dopplerBins

```
uint16_t dopplerBins
```

[in] Number of doppler bins. Must be multiple of 16

4.8.2.2 guardInterval

uint8_t guardInterval

[in] Guard interval of CFAR algo.

4.8.2.3 pDetectBitmap

uint32_t pDetectBitmap

[out] Pointer to data buffer which will store peak detection bitmap results, of size $[(N_CHIRPS/16)*N_SAMPLES]$ It is represented as uint32_t to allow passing 32-bit addresses from A53 to BBE32

4.8.2.4 pRdm

uint32_t pRdm

[in] Pointer to data buffer containing 2-D range-doppler magnitude matrix of size $[N_CHIRPS*N_SAMPLES]$. It is represented as uint32_t to allow passing 32-bit addresses from A53 to BBE32. The buffer must be aligned at 32 bytes and size must be multiple of 32 bytes (16 16-bit magnitude values)

4.8.2.5 pThr

uint32_t pThr

[in] Pointer to scratch buffer used by RsdkBbe32Cfar() to store computed threshold values It is represented as uint32_t to allow passing 32-bit addresses from A53 to BBE32. The buffer must be aligned at 32 bytes and size must be multiple of 32 bytes

4.8.2.6 rangeBins

uint16_t rangeBins

[in] Number of range bins.

4.8.2.7 thresholdFactor

uint16_t thresholdFactor

[in] Threshold factor. SPT Mag2Log2 format. User must prevent overflow/saturation.

4.8.2.8 windowSize

uint8_t windowSize

[in] Window size for CFAR algo.

4.9 rsdkBbe32CmprDecTxpsParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pDeCmprOut](#)
- uint32_t [pInComprDat](#)
- uint32_t [pCcfg](#)
- uint32_t [pMantBwFlx](#)
- uint32_t [pMantBwEncTbl](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)
- uint16_t [numExpBits](#)
- uint16_t [numFillBits](#)
- int16_t [mantBwFix](#)

4.9.1 Field Documentation

4.9.1.1 mantBwFix

```
int16_t mantBwFix
```

[in] Fixed mantissa bit width, 1-15 (pMantBwFlx is ignored); -1 (data from pMantBwFlx is used)

4.9.1.2 noChirps

```
uint16_t noChirps
```

[in] Number of chirps.

4.9.1.3 noRanges

```
uint16_t noRanges
```

[in] Number of range bins in a chirp.

4.9.1.4 numExpBits

```
uint16_t numExpBits
```

[in] Number of exponent bits, must be 4 or 5

4.9.1.5 numFillBits

```
uint16_t numFillBits
```

[in] Number of fill bits, 0..7

4.9.1.6 pCcfg

```
uint32_t pCcfg
```

[in/out] Pointer to scratch data buffer containing the Compression configuration of size [noRanges * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.9.1.7 pDeCmprOut

```
uint32_t pDeCmprOut
```

[out] Pointer to data buffer containing output data vector(decompressed data) of size [noRanges*nChirp*4*8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.9.1.8 pInComprDat

```
uint32_t pInComprDat
```

[out] Pointer to data buffer containing input data vector(compressed data) of size [result of RsdK_EncTxpsInit()], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.9.1.9 pMantBwEncTbl

```
uint32_t pMantBwEncTbl
```

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each, only for numFillBits = 3 (15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.9.1.10 pMantBwFlx

```
uint32_t pMantBwFlx
```

[in] Pointer to data buffer containing input data representing the Bit width table of size [noRanges / 2], containing uint8_t size elements. Each uint8 contains 2 4-bit values as 16 - MBWInput concatenated. If mantBwFlx is set to -1, this is ignored). It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.10 rsdkBbe32CmprEncTxpsParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pCmprOut](#)
- uint32_t [pInDat](#)
- uint32_t [pCcfg](#)
- uint32_t [pMantBwFlx](#)
- uint32_t [pMantBwEncTbl](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)
- uint16_t [numExpBits](#)
- uint16_t [numFillBits](#)
- int16_t [mantBwFix](#)

4.10.1 Field Documentation

4.10.1.1 mantBwFix

int16_t mantBwFix

[in] Fixed mantissa bit width, 1-15 (pMantBwFlx is ignored); -1 (data from pMantBwFlx is used)

4.10.1.2 noChirps

uint16_t noChirps

[in] Number of chirps.

4.10.1.3 noRanges

uint16_t noRanges

[in] Number of range bins in a chirp.

4.10.1.4 numExpBits

uint16_t numExpBits

[in] Number of exponent bits, must be 4 or 5

4.10.1.5 numFillBits

uint16_t numFillBits

[in] Number of fill bits, 0..7

4.10.1.6 pCcfg

uint32_t pCcfg

[in/out] Pointer to scratch data buffer containing the Compression configuration of size [noRanges * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.10.1.7 pCmprOut

uint32_t pCmprOut

[out] Pointer to data buffer containing output data vector(compressed data) of size [result of RsdK_EncTxpsInit()], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.10.1.8 pInDat

uint32_t pInDat

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.10.1.9 pMantBwEncTbl

uint32_t pMantBwEncTbl

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each, only for numFillBits = 3 (15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.10.1.10 pMantBwFlx

uint32_t pMantBwFlx

[in] Pointer to data buffer containing input data representing the Bit width table of size [noRanges / 2], containing uint8_t size elements. Each uint8 contains 2 4-bit values as 16 - MBWInput concatenated. If mantBwFlx is set to -1, this is ignored). It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.11 rsdkBbe32CmprFixOneBufParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pCmprOut](#)
- uint32_t [pInDat](#)
- uint32_t [pMantBwEncTbl](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)
- uint16_t [numExpBits](#)
- uint16_t [numFillBits](#)
- int16_t [mantBwFix](#)

4.11.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for BBE32 radar signal processing algorithms.

4.11.2 Field Documentation

4.11.2.1 mantBwFix

```
int16_t mantBwFix
```

[in] Fixed mantissa bit width, 1-15

4.11.2.2 noChirps

```
uint16_t noChirps
```

[in] Number of chirps.

4.11.2.3 noRanges

```
uint16_t noRanges
```

[in] Number of range bins in a chirp.

4.11.2.4 numExpBits

```
uint16_t numExpBits
```

[in] Number of exponent bits, must be 4 or 5

4.11.2.5 numFillBits

uint16_t numFillBits

[in] Number of fill bits, 0..7

4.11.2.6 pCmprOut

uint32_t pCmprOut

[out] Pointer to data buffer containing output data vector(exponent/fill/mantissa bits) of size [result of RsdK_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.11.2.7 pInDat

uint32_t pInDat

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.11.2.8 pMantBwEncTbl

uint32_t pMantBwEncTbl

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each, only for numFillBits = 3 (15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.12 rsdkBbe32CmprFixTwoBuf4Params_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t pCmprOut
- uint32_t pCmprExpOut
- uint32_t pInDat
- uint16_t noRanges
- uint16_t noChirps
- int16_t mantBwFix

4.12.1 Field Documentation

4.12.1.1 mantBwFix

```
int16_t mantBwFix
```

[in] Fixed mantissa bit width, 1-15

4.12.1.2 noChirps

```
uint16_t noChirps
```

[in] Number of chirps.

4.12.1.3 noRanges

```
uint16_t noRanges
```

[in] Number of range bins in a chirp.

4.12.1.4 pCmprExpOut

```
uint32_t pCmprExpOut
```

[out] Pointer to data buffer containing output data vector(exponent/fill bits) of size [noRanges / 2], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.12.1.5 pCmprOut

```
uint32_t pCmprOut
```

[out] Pointer to data buffer containing output data vector(mantissa bits) of size [result of RsdK_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.12.1.6 pInDat

```
uint32_t pInDat
```

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.13 rsdkBbe32CmprFixTwoBuf8Params_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pCmprOut](#)
- uint32_t [pCmprExpOut](#)
- uint32_t [pInDat](#)
- uint32_t [pMantBwEncTbl](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)
- int16_t [mantBwFix](#)

4.13.1 Field Documentation

4.13.1.1 mantBwFix

```
int16_t mantBwFix
```

[in] Fixed mantissa bit width, 1-15

4.13.1.2 noChirps

```
uint16_t noChirps
```

[in] Number of chirps.

4.13.1.3 noRanges

```
uint16_t noRanges
```

[in] Number of range bins in a chirp.

4.13.1.4 pCmprExpOut

```
uint32_t pCmprExpOut
```

[out] Pointer to data buffer containing output data vector(exponent/fill bits) of size [noRanges], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.13.1.5 pCmprOut

```
uint32_t pCmprOut
```

[out] Pointer to data buffer containing output data vector(mantissa bits) of size [result of Rsdk_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.13.1.6 pInDat

uint32_t pInDat

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.13.1.7 pMantBwEncTbl

uint32_t pMantBwEncTbl

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each (15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.14 rsdkBbe32CmprFlxOneBufParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t pCmprOut
- uint32_t pInDat
- uint32_t pMantBwFlx
- uint32_t pMantBwEncTbl
- uint16_t noRanges
- uint16_t noChirps
- uint16_t numExpBits
- uint16_t numFillBits

4.14.1 Field Documentation

4.14.1.1 noChirps

uint16_t noChirps

[in] Number of chirps.

4.14.1.2 noRanges

uint16_t noRanges

[in] Number of range bins in a chirp.

4.14.1.3 numExpBits

```
uint16_t numExpBits
```

[in] Number of exponent bits, must be 4 or 5

4.14.1.4 numFillBits

```
uint16_t numFillBits
```

[in] Number of fill bits, 0..7

4.14.1.5 pCmprOut

```
uint32_t pCmprOut
```

[out] Pointer to data buffer containing output data vector(exponent/fill/mantissa bits) of size [result of RsdK_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.14.1.6 pInDat

```
uint32_t pInDat
```

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.14.1.7 pMantBwEncTbl

```
uint32_t pMantBwEncTbl
```

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each, only for numFillBits = 3 (15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.14.1.8 pMantBwFlx

```
uint32_t pMantBwFlx
```

[in] Pointer to data buffer containing input data representing the Bit width table of size [noRanges], containing uint8_t size elements. Each value can take values from 1...15. IMPORTANT: Use RsdK_CompressCompPmantBwFlx() to compute the p_mantBwFlx before calling this. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.15 rsdkBbe32CmprFlxTwoBuf4Params_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pCmprOut](#)
- uint32_t [pCmprExpOut](#)
- uint32_t [pInDat](#)
- uint32_t [pMantBwFlx](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)

4.15.1 Field Documentation

4.15.1.1 noChirps

uint16_t noChirps

[in] Number of chirps.

4.15.1.2 noRanges

uint16_t noRanges

[in] Number of range bins in a chirp.

4.15.1.3 pCmprExpOut

uint32_t pCmprExpOut

[out] Pointer to data buffer containing output data vector(exponent/fill bits) of size [noRanges/ 2], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.15.1.4 pCmprOut

uint32_t pCmprOut

[out] Pointer to data buffer containing output data vector(mantissa bits) of size [result of RsdK_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.15.1.5 pInDat

uint32_t pInDat

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.15.1.6 pMantBwFlx

uint32_t pMantBwFlx

[in] Pointer to data buffer containing input data representing the Bit width table of size [noRanges], containing uint8_t size elements. Each value can take values from 1...15. IMPORTANT: Use RsdK_CompressCompPmantBwFlx() to compute the p_mantBwFlx before calling this. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.16 rsdkBbe32CmprFlxTwoBuf8Params_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t pCmprOut
- uint32_t pCmprExpOut
- uint32_t plnDat
- uint32_t pMantBwFlx
- uint32_t pMantBwEncTbl
- uint16_t noRanges
- uint16_t noChirps

4.16.1 Field Documentation

4.16.1.1 noChirps

uint16_t noChirps

[in] Number of chirps.

4.16.1.2 noRanges

uint16_t noRanges

[in] Number of range bins in a chirp.

4.16.1.3 pCmprExpOut

uint32_t pCmprExpOut

[out] Pointer to data buffer containing output data vector(exponent/fill bits) of size [noRanges], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.16.1.4 pCmprOut

```
uint32_t pCmprOut
```

[out] Pointer to data buffer containing output data vector(mantissa bits) of size [result of RsdK_EncSeqBytesPerChirp() * noChirps], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.16.1.5 pInDat

```
uint32_t pInDat
```

[in] Pointer to data buffer containing input data vector of size [noRanges*noChirps * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.16.1.6 pMantBwEncTbl

```
uint32_t pMantBwEncTbl
```

[in] Fill bit encoding, 15 entries for mant BW = 1..15, 3 bit each(15 elements size of uint8_t) It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.16.1.7 pMantBwFlx

```
uint32_t pMantBwFlx
```

[in] Pointer to data buffer containing input data representing the Bit width table of size [noRanges], containing uint8_t size elements. Each value can take values from 1...15. IMPORTANT: Use RsdK_CompressCompPmantBwFlx() to compute the p_mantBwFlx before calling this. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.17 rsdkBbe32CmprSplnitParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t numBytesPerChirp
- uint32_t nRange
- uint32_t pMantBwFlx
- uint8_t comprType

4.17.1 Field Documentation

4.17.1.1 comprType

uint8_t comprType

[in] Compression mode for which the initialization is done, see rsdkBbe32CmprModeSpt_t enum for available possibilities.

4.17.1.2 nRange

uint32_t nRange

[in] Number of range bins in a chirp. (N)

4.17.1.3 numBytesPerChirp

uint32_t numBytesPerChirp

[out] Number of bytes per chirps (P) IMPORTANT: this parameter must be passed as in put for the actual compression through the [rsdkBbe32CmprSptParams_t](#) structure.

4.17.1.4 pMantBwFlx

uint32_t pMantBwFlx

[in] Pointer to data buffer containing input data representing the Bit width table of size [N], containing uint8_t size elements. Each uint8 contains an 8 bit value, that can be 3/5/7/11. IMPORTANT: Set to NULL for RsdkBbe32EncCp4d, RsdkBbe32EncCp4dFmta, RsdkBbe32EncCp4dFmtb. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.18 rsdkBbe32CmprSptParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pCmprOut](#)
- uint32_t [pInDat](#)
- uint32_t [pMantBwFlx](#)
- uint32_t [numBytesPerChirp](#)
- uint16_t [noRanges](#)
- uint16_t [noChirps](#)

4.18.1 Field Documentation

4.18.1.1 noChirps

```
uint16_t noChirps
```

[in] Number of chirps. (M)

4.18.1.2 noRanges

```
uint16_t noRanges
```

[in] Number of range bins in a chirp. (N)

4.18.1.3 numBytesPerChirp

```
uint32_t numBytesPerChirp
```

[in] Number of bytes per chirps (P) IMPORTANT: The required value is obtained by calling [RsdkBbe32CmprSptInit\(\)](#) and copying the parameter with the same name.

4.18.1.4 pCmprOut

```
uint32_t pCmprOut
```

[out] Pointer to data buffer containing output data vector(exponent/fill/mantissa bits) of size [P * M], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.18.1.5 pInDat

```
uint32_t pInDat
```

[in] Pointer to data buffer containing input data vector of size [N * M * 4 * 8], containing uint8_t size elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.18.1.6 pMantBwFlx

```
uint32_t pMantBwFlx
```

[in] Pointer to data buffer containing input data representing the Bit width mantissa table of size [N], containing uint8_t size elements. Each uint8_t contains 2 4-bit values as 16 - MBWInput concatenated. IMPORTANT: Set to NULL for RsdkBbe32EncCp4d, RsdkBbe32EncCp4dFmta, RsdkBbe32EncCp4dFmtb. IMPORTANT: FOR RsdkBbe32EncFlxSpt34, the required values are obtained by calling [RsdkBbe32CmprSptInit\(\)](#) with the same pointer. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.19 rsdkBbe32DoaParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_s32r41.h>
```

Data Fields

- uint32_t [pOutMag](#)
- uint32_t [pDetectBitmap](#)
- uint32_t [pln](#)
- uint32_t [pScratch1Area](#)
- uint32_t [pScratch2Area](#)
- uint32_t [pTwiddlesArea](#)
- uint32_t [pWindowArea](#)
- uint16_t [noAntennas](#)
- uint16_t [noPeaks](#)
- uint16_t [shift](#)
- uint32_t [thresholdFactor](#)

4.19.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for complex DoA algorithms based on FFTs.

4.19.2 Field Documentation

4.19.2.1 noAntennas

uint16_t noAntennas

[in] Input. Number of virtual antennas.

4.19.2.2 noPeaks

uint16_t noPeaks

[in] Input. Number of input peaks.

4.19.2.3 pDetectBitmap

uint32_t pDetectBitmap

[out] Output. Local max Bitmap, size should be noPeaks * doaSize(128/256) * 2 / 16 bytes. Each elements of the bitmap is represented on 16 bits. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.4 pIn

```
uint32_t pIn
```

[in] Input. Size should be $\text{noPeaks} * \text{noAntennas} * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits (q0.23 format), but stored on 32 bits. It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.5 pOutMag

```
uint32_t pOutMag
```

[out] Output. Magnitude values $((a^2 + b^2) / 2)$. Utilize data size should be $\text{noPeaks} * \text{doaSize}(128/256) * 4$ bytes, but the allocated space must be $\text{noPeaks} * \text{doaSize}(128/256) * 8$ bytes. Each element is represented on 24 bits, but stored on 32 bits. The output value should be scaled with $(\text{shift}^4)^2 / 256$. It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.6 pScratch1Area

```
uint32_t pScratch1Area
```

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Size should be $128 * 8$ bytes for `doa128`. Size should be $256 * 8$ bytes for `doa256`. Each element is represented on 24 bits, but stored on 32 bits. **OPTIONAL:** For best performance this should point to a DRAM area. It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.7 pScratch2Area

```
uint32_t pScratch2Area
```

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Each element is represented on 24 bits, but stored on 32 bits. Size should be $128 * 8$ bytes for `doa128`. Size should be $256 * 8$ bytes for `doa256`. **OPTIONAL:** For best performance this should point to a DRAM area (if possible in different memory from `pScratch1Area`). It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.8 pTwiddlesArea

```
uint32_t pTwiddlesArea
```

[scratch] < > Scratch. Pointer to area where twiddles will be stored. Size should be $352 * 2$ bytes for `doa128`. Size should be $832 * 2$ bytes for `doa256`. Each element is stored on 16 bits, (q0.15) format. **OPTIONAL:** For the initialization of the twiddles values, `RsdkBbe32FftInit()` must be called before with the same addresses for twiddles and window areas. **OPTIONAL:** For best performance this should point to a DRAM area. It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.9 pWindowArea

uint32_t pWindowArea

[scratch] < > Scratch. Pointer to area where window coefficients will be stored. Size should be [doasize(128/256) / 2 * 2] bytes for DOA128/256. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For the initialization of the window coefficients, [RsdkBbe32FftInit\(\)](#) must be called before with the same addresses for twiddles and window areas. OPTIONAL: For best performance this should point to a DRAM area. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.19.2.10 shift

uint16_t shift

[in] Shift amount for FFTs. RSDK_BBE32_FFT_SHIFT_2 should be used usually to prevent saturation. The available shift amounts are described in rsdkBbe32FftShift_t enum. User must prevent overflow/saturation.

4.19.2.11 thresholdFactor

uint32_t thresholdFactor

[in] Input. The threshold factor is multiplied with the global maximum magnitude value for comparison with the local maximum values. The value represents a fixed-point value between [0,1] saved on 24 bits(q0.23). It is represented as uint32_t to allow passing from M7 to BBE32.

4.20 rsdkBbe32Fft2RealParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pOut1](#)
- uint32_t [pOut2](#)
- uint32_t [pIn1](#)
- uint32_t [pIn2](#)
- uint32_t [pScratch1Area](#)
- uint32_t [pScratch2Area](#)
- uint32_t [pScratch3Area](#)
- uint32_t [pTwiddlesArea](#)
- uint32_t [pWindowArea](#)
- uint16_t [shift](#)
- uint16_t [size](#)

4.20.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for 2 real input FFT algorithm.

4.20.2 Field Documentation

4.20.2.1 pln1

`uint32_t pIn1`

[in] Input buffer containing the first real input signal. Size should be $N * 2$ bytes. Each element is represented on 16 bits(q0.15 format). **IMPORTANT:** pOut1 and pln1 can point to the same memory location! It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.2 pln2

`uint32_t pIn2`

[in] Input buffer containing the second real input signal. Size should be $N * 2$ bytes. Each element is represented on 16 bits(q0.15 format). **IMPORTANT:** pOut2 and pln2 can point to the same memory location! It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.3 pOut1

`uint32_t pOut1`

[out] Output buffer, containing the complex output lower half of FFT corresponding to the first input signal. Size should be $(N / 2) * 8$ bytes. Each element is represented on 24 bits(q0.23 format), but stored on 32 bits. **IMPORTANT:** pOut1 and pln1 can point to the same memory location! It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.4 pOut2

`uint32_t pOut2`

[out] Output buffer, containing the complex output lower half of FFT corresponding to the second input signal. Size should be $(N / 2) * 8$ bytes. Each element is represented on 24 bits(q0.23 format), but stored on 32 bits. **IMPORTANT:** pOut2 and pln2 can point to the same memory location! It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.5 pScratch1Area

`uint32_t pScratch1Area`

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Size should be $N * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. **OPTIONAL:** For best performance this should point to a buffer in the DRAM area **IMPORTANT:** Buffer can be overwritten/reused between multiple kernels or runs. It is represented as `uint32_t` to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.6 pScratch2Area

```
uint32_t pScratch2Area
```

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Size should be $N * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. OPTIONAL: For best performance this should point to a buffer in the DRAM area(if possible in different memory from pScratch1Area). IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.7 pScratch3Area

```
uint32_t pScratch3Area
```

[scratch] < > Scratch. Pointer to scratch area used for combining the two input signals. Size should be $(N + 8) * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. OPTIONAL: For best performance this should point to a buffer in the DRAM area IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.8 pTwiddlesArea

```
uint32_t pTwiddlesArea
```

[scratch] < > Scratch. Pointer to area where twiddles will be stored. Size should be $352 * 2$ bytes for $N = 128$. Size should be $832 * 2$ bytes for $N = 256$. Size should be $1536 * 2$ bytes for $N = 512$. Size should be $3584 * 2$ bytes for $N = 1024$. Size should be $6656 * 2$ bytes for $N = 2048$. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be kept at CONSTANT ADDRESS between multiple chirps processing. IMPORTANT: For the initialization of the twiddles values, [RsdkBbe32FftInit\(\)](#) must be called before with the same addresses for twiddles and window areas. IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.9 pWindowArea

```
uint32_t pWindowArea
```

[scratch] < > Scratch. Pointer to area where window coefficients will be stored. Size should be $(N / 2) * 2$ bytes. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be kept at CONSTANT ADDRESS between multiple chirps processing. IMPORTANT: For the initialization of the window values, [RsdkBbe32FftInit\(\)](#) must be called before with the same addresses for twiddles and window areas. IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.20.2.10 shift

```
uint16_t shift
```

[in] Shift amount. RSDK_BBE32_FFT_SHIFT_2 should be used usually to prevent saturation between radix rounds. The available shift amounts are described in rsdkBbe32FftShift_t enum

4.20.2.11 size

uint16_t size

[in] Size of the FFTs/input signals(N). The available sizes are described in rsdkBbe32FftSize_t enum

4.21 rsdkBbe32FftCplxParams_t Struct Reference

This struture is used to pass the window and twiddles area, for the [RsdkBbe32FftInit\(\)](#) wrapper.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pInOut](#)
- uint32_t [pScratch1Area](#)
- uint32_t [pScratch2Area](#)
- uint32_t [pTwiddlesArea](#)
- uint32_t [pWindowArea](#)
- uint16_t [shift](#)
- uint16_t [size](#)

4.21.1 Detailed Description

This struture is used to pass the window and twiddles area, for the [RsdkBbe32FftInit\(\)](#) wrapper.

<

The data types accepted by the BBE32 algos.

Parameters for complex FFT algorithms.

4.21.2 Field Documentation

4.21.2.1 pInOut

uint32_t pInOut

[in/out] Input/Output buffer pointer. The operation is done inplace, so buffer is used both for holding input signal and the corresponding output FFT. Size should be $N * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.21.2.2 pScratch1Area

```
uint32_t pScratch1Area
```

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Size should be $N * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. OPTIONAL: For best performance this should point to a buffer in the DRAM area. IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.21.2.3 pScratch2Area

```
uint32_t pScratch2Area
```

[scratch] < > Scratch. Pointer to scratch area used in radix rounds of FFTs. Size should be $N * 8$ bytes. Both real and imaginary parts of each element is represented on 24 bits(q0.23 format), but stored on 32 bits. OPTIONAL: For best performance this should point to a buffer in the DRAM area(if possible in different memory from pScrach1Area). IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.21.2.4 pTwiddlesArea

```
uint32_t pTwiddlesArea
```

[scratch] < > Scratch. Pointer to area where twiddles will be stored. Size should be $352 * 2$ bytes for $N = 128$. Size should be $832 * 2$ bytes for $N = 256$. Size should be $1536 * 2$ bytes for $N = 512$. Size should be $3584 * 2$ bytes for $N = 1024$. Size should be $6656 * 2$ bytes for $N = 2048$. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be kept at CONSTANT ADDRESS between multiple chirps processing. IMPORTANT: For the initialization of the twiddles values, [RsdkBbe32FftInit\(\)](#) must be called before with the same addresses for twiddles and window areas. IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.21.2.5 pWindowArea

```
uint32_t pWindowArea
```

[scratch] < > Scratch. Pointer to area where window coefficients will be stored. Size should be $(N / 2) * 2$ bytes. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be kept at CONSTANT ADDRESS between multiple chirps processing. IMPORTANT: For the initialization of the window values, [RsdkBbe32FftInit\(\)](#) must be called before with the same addresses for twiddles and window areas. IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.21.2.6 shift

```
uint16_t shift
```

[in] Shift amount. RSDK_BBE32_FFT_SHIFT_2 should be used usually to prevent saturation between radix rounds. The available shift amounts are described in rsdkBbe32FftShift_t enum

4.21.2.7 size

uint16_t size

[in] Size of the FFTs/input signals(N). The available sizes are described in rsdkBbe32FftSize_t enum

4.22 rsdkBbe32FftInitParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pTwiddlesArea](#)
- uint32_t [pWindowArea](#)
- uint32_t [size](#)

4.22.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for FFT initialization for twiddles and window coefficients

4.22.2 Field Documentation

4.22.2.1 pTwiddlesArea

uint32_t pTwiddlesArea

[scratch] < > Scratch. Pointer to area where twiddles will be stored. Size should be 352 * 2 bytes for N = 128. Size should be 832 * 2 bytes for N = 256. Size should be 1536 * 2 bytes for N = 512. Size should be 3584 * 2 bytes for N = 1024. Size should be 6656 * 2 bytes for N = 2048. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be called only once at the start of the application or when changing the used FFT size. IMPORTANT: Same buffer address must be used for both the initialization(RsdkBbe32FftInit()) and the actual FFT([RsdkBbe32Fft2Real\(\)](#)/[RsdkBbe32Fft2Cplx\(\)](#)). IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.22.2.2 pWindowArea

```
uint32_t pWindowArea
```

[scratch] < > Scratch. Pointer to area where window coefficients will be stored. Size should be $(N / 2) * 2$ bytes. Each element is stored on 16 bits, (q0.15) format. OPTIONAL: For best performance this should point to a buffer in DRAM area, and should be called only once at the start of the application or when changing the used FFT size. IMPORTANT: Same buffer address must be used for both the initialization(RsdkBbe32FftInit()) and the actual FFT(RsdkBbe32Fft2Real()/RsdkBbe32Fft2Cplx()). IMPORTANT: Buffer must not be overwritten or have its address changed. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.22.2.3 size

```
uint32_t size
```

4.23 rsdkBbe32HistParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t pOut
- uint32_t pln
- uint32_t pEdges
- uint16_t size
- uint16_t sizeEdges

4.23.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for Histogram algorithm.

4.23.2 Field Documentation

4.23.2.1 pEdges

```
uint32_t pEdges
```

[in] Edges Input buffer. Size should be sizeEdges * 4 bytes.

4.23.2.2 pIn

`uint32_t pIn`

[in] Input buffer. Size should be `size * 4` bytes.

4.23.2.3 pOut

`uint32_t pOut`

[out] Output buffer. Size should be `sizeEdges * 4` bytes.

4.23.2.4 size

`uint16_t size`

[in] Size of input buffer.

4.23.2.5 sizeEdges

`uint16_t sizeEdges`

[in] Size of the edges buffer.

4.24 rsdkBbe32LalVectorAddParams_t Struct Reference

```
#include <rsdk_dsp_lal_vector_add.h>
```

Data Fields

- `uint32_t` [size](#)
- `uint32_t` [pInput1](#)
- `uint32_t` [pInput2](#)
- `uint32_t` [pOutput](#)

4.24.1 Field Documentation

4.24.1.1 pInput1

`uint32_t pInput1`

[in] Pointer to input data 1.

4.24.1.2 pInput2

```
uint32_t pInput2
```

[in] Pointer to input data 2.

4.24.1.3 pOutput

```
uint32_t pOutput
```

[in] Pointer to output data.

4.24.1.4 size

```
uint32_t size
```

[in] Size of the vector.

4.25 rsdkBbe32OsCfarParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_s32r41.h>
```

Data Fields

- uint16_t [dopplerBins](#)

4.25.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for OS-CFAR algorithm.

4.25.2 Field Documentation

4.25.2.1 dopplerBins

```
uint16_t dopplerBins
```

[in] Number of doppler bins. Must be multiple of 16

4.26 rsdkBbe32ReordTile16to4ChParams_t Struct Reference

The data types accepted by the BBE32 algos.

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pOutDat](#)
- uint32_t [pInDat](#)
- uint8_t [doubleInBufEn](#)
- uint16_t [noElements](#)

4.26.1 Detailed Description

The data types accepted by the BBE32 algos.

Parameters for Reorder algorithm.

4.26.2 Field Documentation

4.26.2.1 doubleInBufEn

```
uint8_t doubleInBufEn
```

[in] Control flag to specify whether [RsdkBbe32ReordTile16to4Ch\(\)](#) should toggle the pInDat pointer in a ping-pong fashion, to match with the CSI2 double-buffer data acquisition pattern. Valid values: "0" = disable, "non-0" = enable.

4.26.2.2 noElements

```
uint16_t noElements
```

4.26.2.3 pInDat

```
uint32_t pInDat
```

[in] Pointer to data buffer containing input data vector of 4 Channels tile 16(16 elem ch1, 16 elem ch2, 16 elem ch3, 16 elem ch4, 16 ch1, 16 ch2, 16 ch3, 16 ch4 ...) of size [noElements], containing elements on 16 bits. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.26.2.4 pOutDat

uint32_t pOutDat

[out] Pointer to data buffer containing output data vector concatenated for all antennas, containing 16-bit elements. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.27 rsdkBbe32TdlmParams_t Struct Reference

```
#include <rsdk_dsp_radar_bb_algos_saf86xx.h>
```

Data Fields

- uint32_t [pln](#)
- uint32_t [pOut](#)
- uint32_t [plnDataAbs](#)
- uint32_t [pTotalSum](#)
- uint16_t [samples](#)
- uint8_t [thrPercent](#)
- uint16_t [scalingFactor](#)

4.27.1 Field Documentation

4.27.1.1 pln

uint32_t pln

[in] Pointer to data buffer containing input data vector for Range*4ch. Input is ADC samples in order (CH1,CH2,CH3,CH4). The size of the buffer is (4 * N) * 2 bytes. Each element is stored on 16 bits. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.27.1.2 plnDataAbs

uint32_t plnDataAbs

[scratch] Pointer to scratch area buffer that will hold the absolute values of the input data vector. The size of the buffer is (4 * N) * 2 bytes. Each element is stored on 16 bits. OPTIONAL: For best performance this should point to a buffer in DRAM area. IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.27.1.3 pOut

uint32_t pOut

[out] Pointer to data buffer containing output data vector for Range*4ch. Output is ADC samples in order (CH1,CH2,CH3,CH4). The size of the buffer is (4 * N) * 2 bytes. Each element is stored on 16 bits. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.27.1.4 pTotalSum

uint32_t pTotalSum

[scratch] Pointer to scratch area buffer used for holding the sum of the absolute values for all 4 channels. The size of the buffer is $N * 2$ bytes. Each element is stored on 16 bits. OPTIONAL: For best performance this should point to a buffer in DRAM area. IMPORTANT: Buffer can be overwritten/reused between multiple kernels or runs. It is represented as uint32_t to allow passing 32-bit addresses from M7 to BBE32.

4.27.1.5 samples

uint16_t samples

4.27.1.6 scalingFactor

uint16_t scalingFactor

[in] Threshold as % of samples (e.g. 15 => $\text{round}(\text{samples} * 0.15)$ as threshold

4.27.1.7 thrPercent

uint8_t thrPercent

[in] Number of samples for Range(N).

Index

arg
 Dsphd_JobType, 146

asyncChanId
 dspDispCommParams_t, 143

BBE32_ALWAYS_INLINE
 DSP LAL Constants, 34

BBE32_NO_INLINE
 DSP LAL Constants, 34

BBE_BATCH_WIDTH
 DSP LAL Demo Constants, 37

BBE_SIMD_WIDTH
 DSP Radar Algo Constants, 24

bufAddr
 Dsphd_TraceCfgType, 149

bufSize
 Dsphd_TraceCfgType, 149

commandNumber
 Dsphd_CommandAckType, 146

commandType
 Dsphd_CommandAckType, 146

commParams
 dspDisplnit_t, 144

comprType
 rsdkBbe32CmprSplnitParams_t, 164

crc
 Dsphd_RuntimeCfgType, 147

ctrlChanId
 dspDispCommParams_t, 143

dopplerBins
 rsdkBbe32CaCfarParams_t, 150
 rsdkBbe32OsCfarParams_t, 177

doubleInBufEn
 rsdkBbe32ReordTile16to4ChParams_t, 178

DSP, 5

DSP Dispatcher API, 5

DSP Dispatcher Constants, 6
 RSDK_DSP_SPT_EVT0_INTEN_BIT, 6
 RSDK_DSP_SPT_EVT10_INTEN_BIT, 6
 RSDK_DSP_SPT_EVT11_INTEN_BIT, 6
 RSDK_DSP_SPT_EVT12_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT1_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT2_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT3_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT4_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT5_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT6_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT7_INTEN_BIT, 7
 RSDK_DSP_SPT_EVT8_INTEN_BIT, 8
 RSDK_DSP_SPT_EVT9_INTEN_BIT, 8

DSP Dispatcher Data Types, 8
 rsdkDspFuncPtr_t, 9
 rsdkDsplsCb_t, 9

DSP Dispatcher Functions, 9
 RsdkDspDispatcherInit, 10
 RsdkDspDispatcherRun, 10

DSP Host Driver API, 25

DSP Host Driver Constants, 26
 DSPHD_SIZEOF_JOB_LIST, 26
 DSPHD_SIZEOF_MSG, 26

DSP Host Driver Data Types, 26
 DSPHD_ACK_OUT_OF_ORDER, 28
 DSPHD_ACK_RECEIVED, 28
 DSPHD_ACK_TIMEOUT, 28
 DSPHD_ACK_WAITING, 28
 Dsphd_AckStatusType, 27
 DSPHD_CFG_FPU_EXTEND_ENUM_TO_32_BITS, 28
 DSPHD_CFG_FPU_ROUND_CEIL, 28
 DSPHD_CFG_FPU_ROUND_FLOOR, 28
 DSPHD_CFG_FPU_ROUND_TO_NEAREST, 28
 DSPHD_CFG_FPU_ROUND_TRUNC, 28
 DSPHD_CFG_P1_TASK_EXTEND_ENUM_TO_32_BITS, 30
 DSPHD_CFG_P1_TASK_NON_PREEMPT, 30
 DSPHD_CFG_P1_TASK_PREEMPT, 30
 DSPHD_DISP_STATE_ERROR, 28
 DSPHD_DISP_STATE_IDLE, 28
 DSPHD_DISP_STATE_INIT, 28
 DSPHD_DISP_STATE_RUNNING, 28
 Dsphd_DispatcherStateType, 28
 Dsphd_fpuRoundConfigType, 28
 DSPHD_MSG_CHECK_DISPATCHER_ALIVE, 29
 DSPHD_MSG_CLEAR_CE_JOB_LIST, 29
 DSPHD_MSG_CLEAR_LONG_JOB_LIST, 29
 DSPHD_MSG_CLEAR_RC_JOB_LIST, 29
 DSPHD_MSG_DSP_TRACE_END, 29
 DSPHD_MSG_DSP_TRACE_START, 29
 DSPHD_MSG_EXTEND_ENUM_TO_32_BITS, 29
 DSPHD_MSG_LAST, 29
 DSPHD_MSG_RUN_ASYNC_JOB, 29
 DSPHD_MSG_UPDATE_CE_JOB_LIST, 29
 DSPHD_MSG_UPDATE_DISPATCHER_CONFIG, 29
 DSPHD_MSG_UPDATE_LONG_JOB_LIST, 29
 DSPHD_MSG_UPDATE_RC_JOB_LIST, 29
 Dsphd_MsgTypeType, 28

- Dsphd_p1TaskConfigType, 29
- DSPHD_SIG_ASYNC_DONE, 30
- DSPHD_SIG_ASYNC_DONE_EXTEND_ENUM_TO_DSP, 30
- Dsphd_SignalsType, 30
- DSP Host Driver Functions, 30
 - Dsphd_CreateJobList, 31
 - Dsphd_Init, 31
 - Dsphd_SendMsg, 31
- DSP LAL API, 32
- DSP LAL Constants, 33
 - BBE32_ALWAYS_INLINE, 34
 - BBE32_NO_INLINE, 34
 - EXP_MASK, 34
 - INFINITY_SP, 34
 - MANTISSA_MASK, 34
 - NAN_SP, 34
 - OPTIMIZE_0, 34
 - OPTIMIZE_1, 35
 - OPTIMIZE_2, 35
 - OPTIMIZE_3, 35
 - OPTIMIZE_S, 35
 - RSDK_MAG2N_2XF32_V1, 35
 - RSDK_MAG2N_2XF32_V2, 36
 - SP_pi, 36
 - SP_pi_over_2, 36
- DSP LAL Demo Constants, 37
 - BBE_BATCH_WIDTH, 37
 - SIZE_VEC, 37
- DSP LAL Demo Data, 33
- DSP LAL Demo Functions, 38
- DSP LAL Functions, 37
- DSP Radar Algo Constants, 24
 - BBE_SIMD_WIDTH, 24
- DSP Radar Algo Data Types, 24
- DSP Radar Algo Functions, 11
 - RsdkBbe32CaCfar, 12
 - RsdkBbe32CmprSptInit, 13
 - RsdkBbe32DecTxps, 13
 - RsdkBbe32Doa128, 14
 - RsdkBbe32Doa256, 14
 - RsdkBbe32EncCp4d, 15
 - RsdkBbe32EncCp4dFmta, 16
 - RsdkBbe32EncCp4dFmtb, 16
 - RsdkBbe32EncFlxSpt34, 16
 - RsdkBbe32EncSeqFixOnebuf, 17
 - RsdkBbe32EncSeqFixTwobuf4, 17
 - RsdkBbe32EncSeqFixTwobuf8, 18
 - RsdkBbe32EncSeqFlxOnebuf, 18
 - RsdkBbe32EncSeqFlxTwobuf4, 19
 - RsdkBbe32EncSeqFlxTwobuf8, 19
 - RsdkBbe32EncTxps, 20
 - RsdkBbe32Fft2Real, 20
 - RsdkBbe32FftCplx, 21
 - RsdkBbe32FftInit, 21
 - RsdkBbe32Hist, 22
 - RsdkBbe32OsCfar, 22
 - RsdkBbe32ReordTile16to4Ch, 23
 - RsdkBbe32Tdlm, 23
- DSP Radar Baseband Algo Library API, 11
- DSP User Guide, 5
- Dsp_lal_api_func_add_vec, 38
 - Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8, 38
 - Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2, 39
 - Rsdk_LA_add_Vof32p8Vif32p8Vif32p8, 39
 - Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2, 40
- Dsp_lal_api_func_cholesky_dec_mat, 86
 - Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8, 86
 - Rsdk_LA_decLLCholPd_Mof32p8Mif32p8, 86
- Dsp_lal_api_func_conj_vec, 87
 - Rsdk_LA_conj_Vof32cp8Vif32cp8, 87
- Dsp_lal_api_func_conversions_vec_mat, 118
 - Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8, 118
 - Rsdk_DT_getlm_Vof32p8Vif32cp8, 120
 - Rsdk_DT_getRe_Vof32p8Vif32cp8, 120
 - Rsdk_DT_interl_Mof32cp8Mif32p8Mif32p8, 121
- Dsp_lal_api_func_copy, 131
 - Rsdk_DT_copy_Mof32cp8Mif32cp8, 132
 - Rsdk_DT_copy_Mof32p8Mif32p8, 132
 - Rsdk_DT_copy_Vof32cp8Vif32cp8, 133
 - Rsdk_DT_copy_Vof32p8Vif32p8, 133
 - Rsdk_DT_copyInOffset_Vof32cp8Vif32cp8, 134
 - Rsdk_DT_copyInOffset_Vof32p8Vif32p8, 135
 - Rsdk_DT_copyOffset_Vof32cp8Vif32cp8, 136
 - Rsdk_DT_copyOffset_Vof32p8Vif32p8, 137
 - Rsdk_DT_copyOutOffset_Vof32cp8Vif32cp8, 138
 - Rsdk_DT_copyOutOffset_Vof32p8Vif32p8, 139
- Dsp_lal_api_func_det_mat, 81
 - Rsdk_LA_detSq_Sof32cp8Mif32cp8, 82
 - Rsdk_LA_detSq_Sof32p8Mif32p8, 82
- Dsp_lal_api_func_div_vec, 50
 - Rsdk_LA_div_Vof32cp8Vif32cp8Vif32cp8, 50
 - Rsdk_LA_div_Vof32p8Vif32p8Vif32p8, 51
 - Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2, 51
- Dsp_lal_api_func_eig_jacobi_mat, 107
 - Rsdk_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8, 107
 - Rsdk_LA_eigSy_Mof32p8Mof32p8Mif32p8, 108
- Dsp_lal_api_func_eig_ql_mat, 108
 - Rsdk_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8, 109
 - Rsdk_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8, 110
 - Rsdk_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8, 110
- Dsp_lal_api_func_eig_QRfrancis_mat, 111
 - Rsdk_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8, 111
 - Rsdk_LA_eigValUpHess_Vof32cp8Mif32cp8, 112
- Dsp_lal_api_func_fill, 61
 - Rsdk_EF_fill_Vof32cp8Sif32c, 62
 - Rsdk_EF_fill_Vof32cp8Sif32cp8, 63
 - Rsdk_EF_fill_Vof32p8Sif32, 63
 - Rsdk_EF_fill_Vof32p8Sif32p8, 64
 - Rsdk_EF_fillZeros_Mof32cp8, 64

- Rsdk_EF_fillZeros_Mof32p8, 65
- Rsdk_EF_fillZeros_Vof32cp8, 65
- Rsdk_EF_fillZeros_Vof32p8, 66
- Rsdk_EF_fillZerosDg_Mof32cp8Sif32cp8, 66
- Rsdk_EF_fillZerosDg_Mof32cp8Vif32cp8, 67
- Rsdk_EF_fillZerosDg_Mof32p8Sif32p8, 67
- Rsdk_EF_fillZerosDg_Mof32p8Vif32p8, 68
- Rsdk_EF_identity_Mof32cp8, 68
- Rsdk_EF_identity_Mof32p8, 69
- Rsdk_SP_fillStgRadix2_Vof32cp8, 69
- Dsp_lal_api_func_hessenberg_mat, 129
 - Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8, 129
 - Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8, 130
- Dsp_lal_api_func_inv_gauss_mat, 104
 - Rsdk_LA_invGausCPNs_Mof32cp8Miof32cp8, 104
 - Rsdk_LA_invGausCPNs_Mof32p8Miof32p8, 105
 - Rsdk_LA_invNsGaus_Mof32cp8Miof32cp8, 105
 - Rsdk_LA_invNsGaus_Mof32p8Miof32p8, 106
- Dsp_lal_api_func_inv_low_triangular_mat, 99
 - Rsdk_LA_invLNs_Mof32cp8Mif32cp8, 99
 - Rsdk_LA_invLNs_Mof32p8Mif32p8, 100
 - Rsdk_LA_invRectLNs_Mof32cp8Mif32cp8, 101
 - Rsdk_LA_invRectLNs_Mof32p8Mif32p8, 101
- Dsp_lal_api_func_inv_qr_mat, 102
 - Rsdk_LA_invHousQRNs_Mof32cp8Mif32cp8, 102
 - Rsdk_LA_invHousQRNs_Mof32p8Mif32p8, 103
- Dsp_lal_api_func_inv_up_triangular_mat, 96
 - Rsdk_LA_invRectUNs_Mof32cp8Mif32cp8, 96
 - Rsdk_LA_invRectUNs_Mof32p8Mif32p8, 97
 - Rsdk_LA_invUNs_Mof32cp8Mif32cp8, 98
 - Rsdk_LA_invUNs_Mof32p8Mif32p8, 98
- Dsp_lal_api_func_log_vec, 75
 - Rsdk_EF_log10_Vof32p8Vif32p8, 75
 - Rsdk_EF_log2_Vof32p8Vif32p8, 76
 - Rsdk_EF_log_Vof32p8Vif32p8, 76
- Dsp_lal_api_func_mean_vec, 113
 - Rsdk_LA_avg_Sof32cp8Vif32cp8, 113
 - Rsdk_LA_avg_Sof32p8Vif32p8, 114
 - Rsdk_LA_havg_Sof32cp8Vif32cp8, 114
 - Rsdk_LA_havg_Sof32p8Vif32p8, 115
 - Rsdk_LA_rms_Sof32p8Vif32cp8, 115
 - Rsdk_LA_rms_Sof32p8Vif32p8, 116
 - Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8, 117
 - Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8, 117
- Dsp_lal_api_func_min_max_vec, 88
 - Rsdk_LA_max_Sou32p8Vif32p8, 88
 - Rsdk_LA_maxAbs_Sou32p8Vif32cp8, 89
 - Rsdk_LA_min_Sou32p8Vif32p8, 90
 - Rsdk_LA_minAbs_Sou32p8Vif32cp8, 90
- Dsp_lal_api_func_mul_vec_mat, 43
 - Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8, 43
 - Rsdk_LA_mul_Mof32p8Mif32p8Mif32p8, 45
 - Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8, 46
 - Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2, 46
 - Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8, 47
- Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2, 47
- Rsdk_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8, 48
- Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8, 49
- Dsp_lal_api_func_op_vec_mat, 53
 - Rsdk_EF_exp_Vof32p8Vif32p8, 54
 - Rsdk_EF_mod_Vof32p8Vif32p8Vif32p8, 55
 - Rsdk_EF_sqrt_Vof32cp8Vif32cp8, 55
 - Rsdk_EF_sqrt_Vof32p8Vif32p8, 56
 - Rsdk_EF_sqrtAbs_Vof32p8Vif32p8, 56
 - Rsdk_LA_hypot_Vof32p8Vif32p8Vif32p8, 57
 - Rsdk_LA_rcp_Vof32cp8Vif32cp8, 58
 - Rsdk_LA_rcp_Vof32p8Vif32p8, 58
 - Rsdk_LA_thresh_Vof32p8Vif32p8Sif32, 59
 - Rsdk_LA_thresh_Vof32p8Vif32p8Vif32p8, 59
 - Rsdk_LA_threshAbs_Vof32p8Vif32cp8Sif32, 60
 - Rsdk_LA_threshAbs_Vof32p8Vif32cp8Vif32p8, 61
- Dsp_lal_api_func_polinom, 140
 - Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8, 140
- Dsp_lal_api_func_prod_tot, 128
 - Rsdk_LA_prod_Sof32cp8Vif32cp8, 128
 - Rsdk_LA_prod_Sof32p8Vif32p8, 129
- Dsp_lal_api_func_prod_vec, 78
 - Rsdk_LA_cross_Vof32cp8Vif32cp8Vif32cp8, 78
 - Rsdk_LA_cross_Vof32p8Vif32p8Vif32p8, 79
 - Rsdk_LA_crossconj_Vof32cp8Vif32cp8Vif32cp8, 79
 - Rsdk_LA_dot_Sof32p8Vif32p8Vif32p8, 80
 - Rsdk_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8, 80
 - Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8, 81
- Dsp_lal_api_func_qr_dec_mat, 83
 - Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8, 83
 - Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8, 84
- Dsp_lal_api_func_snr, 141
 - Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8, 141
- Dsp_lal_api_func_sort_vec, 122
 - Rsdk_AU_sortdOET_Viof32p8_k2, 122
 - Rsdk_AU_sortdOET_Vof32p8Vif32p8, 123
 - Rsdk_AU_sortOET_Viof32p8_k2, 123
 - Rsdk_AU_sortOET_Vof32p8Vif32p8, 124
 - Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8, 124
- Dsp_lal_api_func_sp_smooth, 125
 - Rsdk_ML_spatialSm_Mof32cp8Mif32cp8, 125
- Dsp_lal_api_func_sub_vec, 40
 - Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8, 41
 - Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2, 41
 - Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8, 42
 - Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2, 42
- Dsp_lal_api_func_sum_vec, 91
 - Rsdk_LA_sum2_Sof32cp8Vif32cp8, 91

- Rsdk_LA_sum2_Sof32p8Vif32p8, 92
- Rsdk_LA_sum3_Sof32cp8Vif32cp8, 93
- Rsdk_LA_sum3_Sof32p8Vif32p8, 93
- Rsdk_LA_sum_Sof32cp8Vif32cp8, 94
- Rsdk_LA_sum_Sof32p8Vif32p8, 94
- Rsdk_LA_sumRcp_Sof32cp8Vif32cp8, 95
- Rsdk_LA_sumRcp_Sof32p8Vif32p8, 95
- Dsp_lal_api_func_trig_vec, 70
 - Rsdk_EF_acos_Vof32p8Vif32p8, 70
 - Rsdk_EF_asin_Vof32p8Vif32p8, 71
 - Rsdk_EF_atan2_Vof32p8Vif32p8Vif32p8, 72
 - Rsdk_EF_atan_Vof32p8Vif32p8, 72
 - Rsdk_EF_cos_Vof32p8Vif32p8, 73
 - Rsdk_EF_sin_Vof32p8Vif32p8, 73
 - Rsdk_LA_deg2rad_Vof32p8Vif32p8, 74
 - Rsdk_LA_rad2deg_Vof32p8Vif32p8, 74
- Dsp_lal_api_func_xcorr_vec, 52
 - Rsdk_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8, 52
 - Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8, 53
- Dsp_lal_demos_api_func_add, 142
 - RsdkBbe32LalVectorAdd, 142
- Dsp_lal_func_api_dist, 126
 - Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32p8, 126
 - Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32p8, 127
- dspDispCommParams_t, 143
 - asyncChanId, 143
 - ctrlChanId, 143
 - ipcflInstance, 144
- dspDisplnit_t, 144
 - commParams, 144
 - funcTableSize, 144
 - pFuncTable, 145
 - prefAggrParams, 145
 - sptEvtIntEnBitmap, 145
 - sptEvtIsrCb, 145
- DSPHD_ACK_OUT_OF_ORDER
 - DSP Host Driver Data Types, 28
- DSPHD_ACK_RECEIVED
 - DSP Host Driver Data Types, 28
- DSPHD_ACK_TIMEOUT
 - DSP Host Driver Data Types, 28
- DSPHD_ACK_WAITING
 - DSP Host Driver Data Types, 28
- Dsphd_AckStatusType
 - DSP Host Driver Data Types, 27
- DSPHD_CFG_FPU_EXTEND_ENUM_TO_32_BITS
 - DSP Host Driver Data Types, 28
- DSPHD_CFG_FPU_ROUND_CEIL
 - DSP Host Driver Data Types, 28
- DSPHD_CFG_FPU_ROUND_FLOOR
 - DSP Host Driver Data Types, 28
- DSPHD_CFG_FPU_ROUND_TO_NEAREST
 - DSP Host Driver Data Types, 28
- DSPHD_CFG_FPU_ROUND_TRUNC
 - DSP Host Driver Data Types, 28
- DSPHD_CFG_P1_TASK_EXTEND_ENUM_TO_32_BITS
 - DSP Host Driver Data Types, 30
- DSPHD_CFG_P1_TASK_NON_PREEMPT
 - DSP Host Driver Data Types, 30
- DSPHD_CFG_P1_TASK_PREEMPT
 - DSP Host Driver Data Types, 30
- Dsphd_CommandAckType, 145
 - commandNumber, 146
 - commandType, 146
- Dsphd_CreateJobList
 - DSP Host Driver Functions, 31
- DSPHD_DISP_STATE_ERROR
 - DSP Host Driver Data Types, 28
- DSPHD_DISP_STATE_IDLE
 - DSP Host Driver Data Types, 28
- DSPHD_DISP_STATE_INIT
 - DSP Host Driver Data Types, 28
- DSPHD_DISP_STATE_RUNNING
 - DSP Host Driver Data Types, 28
- Dsphd_DispatcherStateType
 - DSP Host Driver Data Types, 28
- Dsphd_fpuRoundConfigType
 - DSP Host Driver Data Types, 28
- Dsphd_Init
 - DSP Host Driver Functions, 31
- Dsphd_JobType, 146
 - arg, 146
 - id, 147
- DSPHD_MSG_CHECK_DISPATCHER_ALIVE
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_CLEAR_CE_JOB_LIST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_CLEAR_LONG_JOB_LIST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_CLEAR_RC_JOB_LIST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_DSP_TRACE_END
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_DSP_TRACE_START
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_EXTEND_ENUM_TO_32_BITS
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_LAST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_RUN_ASYNC_JOB
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_UPDATE_CE_JOB_LIST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_UPDATE_DISPATCHER_CONFIG
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_UPDATE_LONG_JOB_LIST
 - DSP Host Driver Data Types, 29
- DSPHD_MSG_UPDATE_RC_JOB_LIST
 - DSP Host Driver Data Types, 29
- Dsphd_MsgTypeType
 - DSP Host Driver Data Types, 28
- Dsphd_p1TaskConfigType
 - DSP Host Driver Data Types, 29
- Dsphd_RuntimeCfgType, 147

- crc, [147](#)
- fpuRoundConfig, [147](#)
- numberOfChirps, [148](#)
- p1TaskConfig, [148](#)
- pDispatcherState, [148](#)
- reportFPUExceptions, [148](#)
- Dsphd_SendMsg
 - DSP Host Driver Functions, [31](#)
- DSPHD_SIG_ASYNC_DONE
 - DSP Host Driver Data Types, [30](#)
- DSPHD_SIG_ASYNC_DONE_EXTEND_ENUM_TO_32_BITS
 - DSP Host Driver Data Types, [30](#)
- Dsphd_SignalsType
 - DSP Host Driver Data Types, [30](#)
- DSPHD_SIZEOF_JOB_LIST
 - DSP Host Driver Constants, [26](#)
- DSPHD_SIZEOF_MSG
 - DSP Host Driver Constants, [26](#)
- Dsphd_TraceCfgType, [148](#)
 - bufAddr, [149](#)
 - bufSize, [149](#)
- dspPrefetchAggrParams_t, [149](#)
 - prefetchAggressionData, [149](#)
 - prefetchAggressionInstr, [149](#)
 - prefetchAggressionSW, [150](#)
- EXP_MASK
 - DSP LAL Constants, [34](#)
- fpuRoundConfig
 - Dsphd_RuntimeCfgType, [147](#)
- funcTableSize
 - dspDisplnit_t, [144](#)
- guardInterval
 - rsdkBbe32CaCfarParams_t, [150](#)
- id
 - Dsphd_JobType, [147](#)
- INFINITY_SP
 - DSP LAL Constants, [34](#)
- ipcflInstance
 - dspDispCommParams_t, [144](#)
- mantBwFix
 - rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
 - rsdkBbe32CmprFixOneBufParams_t, [156](#)
 - rsdkBbe32CmprFixTwoBuf4Params_t, [157](#)
 - rsdkBbe32CmprFixTwoBuf8Params_t, [159](#)
- MANTISSA_MASK
 - DSP LAL Constants, [34](#)
- NAN_SP
 - DSP LAL Constants, [34](#)
- noAntennas
 - rsdkBbe32DoaParams_t, [167](#)
- noChirps
 - rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
- rsdkBbe32CmprFixOneBufParams_t, [156](#)
- rsdkBbe32CmprFixTwoBuf4Params_t, [158](#)
- rsdkBbe32CmprFixTwoBuf8Params_t, [159](#)
- rsdkBbe32CmprSptParams_t, [165](#)
- noElements
 - rsdkBbe32ReordTile16to4ChParams_t, [178](#)
- noPeaks
 - rsdkBbe32DoaParams_t, [167](#)
- noRanges
 - rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
 - rsdkBbe32CmprFixOneBufParams_t, [156](#)
 - rsdkBbe32CmprFixTwoBuf4Params_t, [158](#)
 - rsdkBbe32CmprFixTwoBuf8Params_t, [159](#)
 - rsdkBbe32CmprFixOneBufParams_t, [160](#)
 - rsdkBbe32CmprFixTwoBuf4Params_t, [162](#)
 - rsdkBbe32CmprFixTwoBuf8Params_t, [163](#)
 - rsdkBbe32CmprSptParams_t, [166](#)
- nRange
 - rsdkBbe32CmprSpInitParams_t, [165](#)
- numberOfChirps
 - Dsphd_RuntimeCfgType, [148](#)
- numBytesPerChirp
 - rsdkBbe32CmprSpInitParams_t, [165](#)
 - rsdkBbe32CmprSptParams_t, [166](#)
- numExpBits
 - rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
 - rsdkBbe32CmprFixOneBufParams_t, [156](#)
 - rsdkBbe32CmprFixOneBufParams_t, [160](#)
- numFillBits
 - rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
 - rsdkBbe32CmprFixOneBufParams_t, [156](#)
 - rsdkBbe32CmprFixOneBufParams_t, [161](#)
- OPTIMIZE_0
 - DSP LAL Constants, [34](#)
- OPTIMIZE_1
 - DSP LAL Constants, [35](#)
- OPTIMIZE_2
 - DSP LAL Constants, [35](#)
- OPTIMIZE_3
 - DSP LAL Constants, [35](#)
- OPTIMIZE_S
 - DSP LAL Constants, [35](#)
- p1TaskConfig
 - Dsphd_RuntimeCfgType, [148](#)
- pCcfig
 - rsdkBbe32CmprDecTxpsParams_t, [153](#)
 - rsdkBbe32CmprEncTxpsParams_t, [154](#)
- pCmprExpOut
 - rsdkBbe32CmprFixTwoBuf4Params_t, [158](#)
 - rsdkBbe32CmprFixTwoBuf8Params_t, [159](#)
 - rsdkBbe32CmprFixTwoBuf4Params_t, [162](#)

- rsdkBbe32CmprFlxTwoBuf8Params_t, 163
- pCmprOut
 - rsdkBbe32CmprEncTxpsParams_t, 155
 - rsdkBbe32CmprFixOneBufParams_t, 157
 - rsdkBbe32CmprFixTwoBuf4Params_t, 158
 - rsdkBbe32CmprFixTwoBuf8Params_t, 159
 - rsdkBbe32CmprFlxOneBufParams_t, 161
 - rsdkBbe32CmprFlxTwoBuf4Params_t, 162
 - rsdkBbe32CmprFlxTwoBuf8Params_t, 163
 - rsdkBbe32CmprSptParams_t, 166
- pDeCmprOut
 - rsdkBbe32CmprDecTxpsParams_t, 153
- pDetectBitmap
 - rsdkBbe32CaCfarParams_t, 151
 - rsdkBbe32DoaParams_t, 167
- pDispatcherState
 - Dsphd_RuntimeCfgType, 148
- pEdges
 - rsdkBbe32HistParams_t, 175
- pFuncTable
 - dspDisplnit_t, 145
- pln
 - rsdkBbe32DoaParams_t, 167
 - rsdkBbe32HistParams_t, 175
 - rsdkBbe32TdlmParams_t, 179
- pln1
 - rsdkBbe32Fft2RealParams_t, 170
- pln2
 - rsdkBbe32Fft2RealParams_t, 170
- plnComprDat
 - rsdkBbe32CmprDecTxpsParams_t, 153
- plnDat
 - rsdkBbe32CmprEncTxpsParams_t, 155
 - rsdkBbe32CmprFixOneBufParams_t, 157
 - rsdkBbe32CmprFixTwoBuf4Params_t, 158
 - rsdkBbe32CmprFixTwoBuf8Params_t, 159
 - rsdkBbe32CmprFlxOneBufParams_t, 161
 - rsdkBbe32CmprFlxTwoBuf4Params_t, 162
 - rsdkBbe32CmprFlxTwoBuf8Params_t, 164
 - rsdkBbe32CmprSptParams_t, 166
 - rsdkBbe32ReordTile16to4ChParams_t, 178
- plnDataAbs
 - rsdkBbe32TdlmParams_t, 179
- plnOut
 - rsdkBbe32FftCplxParams_t, 172
- plnInput1
 - rsdkBbe32LalVectorAddParams_t, 176
- plnInput2
 - rsdkBbe32LalVectorAddParams_t, 176
- pMantBwEncTbl
 - rsdkBbe32CmprDecTxpsParams_t, 153
 - rsdkBbe32CmprEncTxpsParams_t, 155
 - rsdkBbe32CmprFixOneBufParams_t, 157
 - rsdkBbe32CmprFixTwoBuf8Params_t, 160
 - rsdkBbe32CmprFlxOneBufParams_t, 161
 - rsdkBbe32CmprFlxTwoBuf8Params_t, 164
- pMantBwFlx
 - rsdkBbe32CmprDecTxpsParams_t, 153
- rsdkBbe32CmprEncTxpsParams_t, 155
- rsdkBbe32CmprFlxOneBufParams_t, 161
- rsdkBbe32CmprFlxTwoBuf4Params_t, 162
- rsdkBbe32CmprFlxTwoBuf8Params_t, 164
- rsdkBbe32CmprSptParams_t, 166
- pOut
 - rsdkBbe32HistParams_t, 176
 - rsdkBbe32TdlmParams_t, 179
- pOut1
 - rsdkBbe32Fft2RealParams_t, 170
- pOut2
 - rsdkBbe32Fft2RealParams_t, 170
- pOutDat
 - rsdkBbe32ReordTile16to4ChParams_t, 178
- pOutMag
 - rsdkBbe32DoaParams_t, 168
- pOutput
 - rsdkBbe32LalVectorAddParams_t, 177
- pRdm
 - rsdkBbe32CaCfarParams_t, 151
- prefAggrParams
 - dspDisplnit_t, 145
- prefetchAggressionData
 - dspPrefetchAggrParams_t, 149
- prefetchAggressionInstr
 - dspPrefetchAggrParams_t, 149
- prefetchAggressionSW
 - dspPrefetchAggrParams_t, 150
- pScratch1Area
 - rsdkBbe32DoaParams_t, 168
 - rsdkBbe32Fft2RealParams_t, 170
 - rsdkBbe32FftCplxParams_t, 172
- pScratch2Area
 - rsdkBbe32DoaParams_t, 168
 - rsdkBbe32Fft2RealParams_t, 170
 - rsdkBbe32FftCplxParams_t, 173
- pScratch3Area
 - rsdkBbe32Fft2RealParams_t, 171
- pThr
 - rsdkBbe32CaCfarParams_t, 151
- pTotalSum
 - rsdkBbe32TdlmParams_t, 179
- pTwiddlesArea
 - rsdkBbe32DoaParams_t, 168
 - rsdkBbe32Fft2RealParams_t, 171
 - rsdkBbe32FftCplxParams_t, 173
 - rsdkBbe32FftlntParams_t, 174
- pWindowArea
 - rsdkBbe32DoaParams_t, 168
 - rsdkBbe32Fft2RealParams_t, 171
 - rsdkBbe32FftCplxParams_t, 173
 - rsdkBbe32FftlntParams_t, 174
- rangeBins
 - rsdkBbe32CaCfarParams_t, 151
- reportFPUEXceptions
 - Dsphd_RuntimeCfgType, 148
- Rsdk_AU_sortdOET_Viof32p8_k2

- Dsp_lal_api_func_sort_vec, [122](#)
- Rsdk_AU_sortdOET_Vof32p8Vif32p8
 - Dsp_lal_api_func_sort_vec, [123](#)
- Rsdk_AU_sortOET_Viof32p8_k2
 - Dsp_lal_api_func_sort_vec, [123](#)
- Rsdk_AU_sortOET_Vof32p8Vif32p8
 - Dsp_lal_api_func_sort_vec, [124](#)
- RSDK_DSP_SPT_EVT0_INTEN_BIT
 - DSP Dispatcher Constants, [6](#)
- RSDK_DSP_SPT_EVT10_INTEN_BIT
 - DSP Dispatcher Constants, [6](#)
- RSDK_DSP_SPT_EVT11_INTEN_BIT
 - DSP Dispatcher Constants, [6](#)
- RSDK_DSP_SPT_EVT12_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT1_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT2_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT3_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT4_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT5_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT6_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT7_INTEN_BIT
 - DSP Dispatcher Constants, [7](#)
- RSDK_DSP_SPT_EVT8_INTEN_BIT
 - DSP Dispatcher Constants, [8](#)
- RSDK_DSP_SPT_EVT9_INTEN_BIT
 - DSP Dispatcher Constants, [8](#)
- Rsdk_DT_copy_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_copy, [132](#)
- Rsdk_DT_copy_Mof32p8Mif32p8
 - Dsp_lal_api_func_copy, [132](#)
- Rsdk_DT_copy_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_copy, [133](#)
- Rsdk_DT_copy_Vof32p8Vif32p8
 - Dsp_lal_api_func_copy, [133](#)
- Rsdk_DT_copyInOffset_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_copy, [134](#)
- Rsdk_DT_copyInOffset_Vof32p8Vif32p8
 - Dsp_lal_api_func_copy, [135](#)
- Rsdk_DT_copyOffset_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_copy, [136](#)
- Rsdk_DT_copyOffset_Vof32p8Vif32p8
 - Dsp_lal_api_func_copy, [137](#)
- Rsdk_DT_copyOutOffset_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_copy, [138](#)
- Rsdk_DT_copyOutOffset_Vof32p8Vif32p8
 - Dsp_lal_api_func_copy, [139](#)
- Rsdk_DT_deinterl_Mof32p8Mof32p8Mif32cp8
 - Dsp_lal_api_func_conversions_vec_mat, [118](#)
- Rsdk_DT_getIm_Vof32p8Vif32cp8
 - Dsp_lal_api_func_conversions_vec_mat, [120](#)
- Rsdk_DT_getRe_Vof32p8Vif32cp8
 - Dsp_lal_api_func_conversions_vec_mat, [120](#)
- Rsdk_DT_interl_Mof32cp8Mif32p8Mif32p8
 - Dsp_lal_api_func_conversions_vec_mat, [121](#)
- Rsdk_DT_minMax_Vof32p8Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_sort_vec, [124](#)
- Rsdk_EF_acos_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [70](#)
- Rsdk_EF_asin_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [71](#)
- Rsdk_EF_atan2_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [72](#)
- Rsdk_EF_atan_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [72](#)
- Rsdk_EF_cos_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [73](#)
- Rsdk_EF_exp_Vof32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [54](#)
- Rsdk_EF_fill_Vof32cp8Sif32c
 - Dsp_lal_api_func_fill, [62](#)
- Rsdk_EF_fill_Vof32cp8Sif32cp8
 - Dsp_lal_api_func_fill, [63](#)
- Rsdk_EF_fill_Vof32p8Sif32
 - Dsp_lal_api_func_fill, [63](#)
- Rsdk_EF_fill_Vof32p8Sif32p8
 - Dsp_lal_api_func_fill, [64](#)
- Rsdk_EF_fillZeros_Mof32cp8
 - Dsp_lal_api_func_fill, [64](#)
- Rsdk_EF_fillZeros_Mof32p8
 - Dsp_lal_api_func_fill, [65](#)
- Rsdk_EF_fillZeros_Vof32cp8
 - Dsp_lal_api_func_fill, [65](#)
- Rsdk_EF_fillZeros_Vof32p8
 - Dsp_lal_api_func_fill, [66](#)
- Rsdk_EF_fillZerosDg_Mof32cp8Sif32cp8
 - Dsp_lal_api_func_fill, [66](#)
- Rsdk_EF_fillZerosDg_Mof32cp8Vif32cp8
 - Dsp_lal_api_func_fill, [67](#)
- Rsdk_EF_fillZerosDg_Mof32p8Sif32p8
 - Dsp_lal_api_func_fill, [67](#)
- Rsdk_EF_fillZerosDg_Mof32p8Vif32p8
 - Dsp_lal_api_func_fill, [68](#)
- Rsdk_EF_identity_Mof32cp8
 - Dsp_lal_api_func_fill, [68](#)
- Rsdk_EF_identity_Mof32p8
 - Dsp_lal_api_func_fill, [69](#)
- Rsdk_EF_log10_Vof32p8Vif32p8
 - Dsp_lal_api_func_log_vec, [75](#)
- Rsdk_EF_log2_Vof32p8Vif32p8
 - Dsp_lal_api_func_log_vec, [76](#)
- Rsdk_EF_log_Vof32p8Vif32p8
 - Dsp_lal_api_func_log_vec, [76](#)
- Rsdk_EF_mod_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [55](#)
- Rsdk_EF_sin_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [73](#)
- Rsdk_EF_sqrt_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_op_vec_mat, [55](#)
- Rsdk_EF_sqrt_Vof32p8Vif32p8

- Dsp_lal_api_func_op_vec_mat, [56](#)
- Rsdk_EF_sqrtAbs_Vof32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [56](#)
- Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_add_vec, [38](#)
- Rsdk_LA_add_Vof32cp8Vif32cp8Vif32cp8_k2
 - Dsp_lal_api_func_add_vec, [39](#)
- Rsdk_LA_add_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_add_vec, [39](#)
- Rsdk_LA_add_Vof32p8Vif32p8Vif32p8_k2
 - Dsp_lal_api_func_add_vec, [40](#)
- Rsdk_LA_avg_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_mean_vec, [113](#)
- Rsdk_LA_avg_Sof32p8Vif32p8
 - Dsp_lal_api_func_mean_vec, [114](#)
- Rsdk_LA_conj_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_conj_vec, [87](#)
- Rsdk_LA_cross_Vo3f32cp8Vi3f32cp8Vi3f32cp8
 - Dsp_lal_api_func_prod_vec, [78](#)
- Rsdk_LA_cross_Vo3f32p8Vi3f32p8Vi3f32p8
 - Dsp_lal_api_func_prod_vec, [79](#)
- Rsdk_LA_crossconj_Vo3f32cp8Vi3f32cp8Vi3f32cp8
 - Dsp_lal_api_func_prod_vec, [79](#)
- Rsdk_LA_decLLCholPd_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_cholesky_dec_mat, [86](#)
- Rsdk_LA_decLLCholPd_Mof32p8Mif32p8
 - Dsp_lal_api_func_cholesky_dec_mat, [86](#)
- Rsdk_LA_decQRHous_Mof32cp8Mof32cp8Mif32cp8
 - Dsp_lal_api_func_qr_dec_mat, [83](#)
- Rsdk_LA_decQRHous_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_qr_dec_mat, [84](#)
- Rsdk_LA_decQUpHessQHousSq_Mof32cp8Mof32cp8Mif32cp8
 - Dsp_lal_api_func_hessenberg_mat, [129](#)
- Rsdk_LA_decQUpHessQHousSq_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_hessenberg_mat, [130](#)
- Rsdk_LA_deg2rad_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [74](#)
- Rsdk_LA_detSq_Sof32cp8Mif32cp8
 - Dsp_lal_api_func_det_mat, [82](#)
- Rsdk_LA_detSq_Sof32p8Mif32p8
 - Dsp_lal_api_func_det_mat, [82](#)
- Rsdk_LA_dist2MahalDg2_Sof32p8Vi2f32Vif32p8Vi2f32
 - Dsp_lal_func_api_dist, [126](#)
- Rsdk_LA_dist2MahalDg2_Sof32p8Vif32p8Vif32p8Vi2f32
 - Dsp_lal_func_api_dist, [127](#)
- Rsdk_LA_div_Vof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_div_vec, [50](#)
- Rsdk_LA_div_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_div_vec, [51](#)
- Rsdk_LA_div_Vof32p8Vif32p8Vif32p8_k2
 - Dsp_lal_api_func_div_vec, [51](#)
- Rsdk_LA_dot_Sof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_prod_vec, [80](#)
- Rsdk_LA_dotFirstConj_Sof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_prod_vec, [80](#)
- Rsdk_LA_dotSecondConj_Sof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_prod_vec, [81](#)
- Rsdk_LA_eigHm_Mof32cp8Mof32cp8Mif32cp8
 - Dsp_lal_api_func_eig_jacobi_mat, [107](#)
- Rsdk_LA_eigSy_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_eig_jacobi_mat, [108](#)
- Rsdk_LA_eigTriDgQHousSy_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_eig_ql_mat, [109](#)
- Rsdk_LA_eigTriDgSy_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_eig_ql_mat, [110](#)
- Rsdk_LA_eigUpHess_Vof32cp8Mof32cp8Mif32cp8
 - Dsp_lal_api_func_eig_QRfrancis_mat, [111](#)
- Rsdk_LA_eigValTriDgSy_Mof32p8Mof32p8Mif32p8
 - Dsp_lal_api_func_eig_ql_mat, [110](#)
- Rsdk_LA_eigValUpHess_Vof32cp8Mif32cp8
 - Dsp_lal_api_func_eig_QRfrancis_mat, [112](#)
- Rsdk_LA_havg_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_mean_vec, [114](#)
- Rsdk_LA_havg_Sof32p8Vif32p8
 - Dsp_lal_api_func_mean_vec, [115](#)
- Rsdk_LA_hypot_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [57](#)
- Rsdk_LA_invGausCPNs_Mof32cp8Miof32cp8
 - Dsp_lal_api_func_inv_gauss_mat, [104](#)
- Rsdk_LA_invGausCPNs_Mof32p8Miof32p8
 - Dsp_lal_api_func_inv_gauss_mat, [105](#)
- Rsdk_LA_invHousQRNs_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_inv_qr_mat, [102](#)
- Rsdk_LA_invHousQRNs_Mof32p8Mif32p8
 - Dsp_lal_api_func_inv_qr_mat, [103](#)
- Rsdk_LA_invLNs_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_inv_low_triangular_mat, [99](#)
- Rsdk_LA_invLNs_Mof32p8Mif32p8
 - Dsp_lal_api_func_inv_low_triangular_mat, [100](#)
- Rsdk_LA_invNsGaus_Mof32cp8Miof32cp8
 - Dsp_lal_api_func_inv_gauss_mat, [105](#)
- Rsdk_LA_invNsGaus_Mof32p8Miof32p8
 - Dsp_lal_api_func_inv_gauss_mat, [106](#)
- Rsdk_LA_invRectLNs_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_inv_low_triangular_mat, [101](#)
- Rsdk_LA_invRectLNs_Mof32p8Mif32p8
 - Dsp_lal_api_func_inv_low_triangular_mat, [101](#)
- Rsdk_LA_invRectUNs_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_inv_up_triangular_mat, [96](#)
- Rsdk_LA_invRectUNs_Mof32p8Mif32p8
 - Dsp_lal_api_func_inv_up_triangular_mat, [97](#)
- Rsdk_LA_invUNs_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_inv_up_triangular_mat, [98](#)
- Rsdk_LA_invUNs_Mof32p8Mif32p8
 - Dsp_lal_api_func_inv_up_triangular_mat, [98](#)
- Rsdk_LA_max_Sou32p8Vif32p8
 - Dsp_lal_api_func_min_max_vec, [88](#)
- Rsdk_LA_maxAbs_Sou32p8Vif32cp8
 - Dsp_lal_api_func_min_max_vec, [89](#)
- Rsdk_LA_min_Sou32p8Vif32p8
 - Dsp_lal_api_func_min_max_vec, [90](#)
- Rsdk_LA_minAbs_Sou32p8Vif32cp8
 - Dsp_lal_api_func_min_max_vec, [90](#)
- Rsdk_LA_mul_Mof32cp8Mif32cp8Mif32cp8
 - Dsp_lal_api_func_mul_vec_mat, [43](#)
- Rsdk_LA_mul_Mof32p8Mif32p8Mif32p8

- Dsp_lal_api_func_mul_vec_mat, [45](#)
- Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_mul_vec_mat, [46](#)
- Rsdk_LA_mul_Vof32cp8Vif32cp8Vif32cp8_k2
 - Dsp_lal_api_func_mul_vec_mat, [46](#)
- Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_mul_vec_mat, [47](#)
- Rsdk_LA_mul_Vof32p8Vif32p8Vif32p8_k2
 - Dsp_lal_api_func_mul_vec_mat, [47](#)
- Rsdk_LA_mulSecondH_Mof32cp8Mif32cp8Mif32cp8
 - Dsp_lal_api_func_mul_vec_mat, [48](#)
- Rsdk_LA_mulSecondT_Mof32p8Mif32p8Mif32p8
 - Dsp_lal_api_func_mul_vec_mat, [49](#)
- Rsdk_LA_polinomEval_Sof32p8Vif32p8Sif32p8
 - Dsp_lal_api_func_polinom, [140](#)
- Rsdk_LA_prod_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_prod_tot, [128](#)
- Rsdk_LA_prod_Sof32p8Vif32p8
 - Dsp_lal_api_func_prod_tot, [129](#)
- Rsdk_LA_rad2deg_Vof32p8Vif32p8
 - Dsp_lal_api_func_trig_vec, [74](#)
- Rsdk_LA_rcp_Vof32cp8Vif32cp8
 - Dsp_lal_api_func_op_vec_mat, [58](#)
- Rsdk_LA_rcp_Vof32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [58](#)
- Rsdk_LA_rms_Sof32p8Vif32cp8
 - Dsp_lal_api_func_mean_vec, [115](#)
- Rsdk_LA_rms_Sof32p8Vif32p8
 - Dsp_lal_api_func_mean_vec, [116](#)
- Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_sub_vec, [41](#)
- Rsdk_LA_sub_Vof32cp8Vif32cp8Vif32cp8_k2
 - Dsp_lal_api_func_sub_vec, [41](#)
- Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_sub_vec, [42](#)
- Rsdk_LA_sub_Vof32p8Vif32p8Vif32p8_k2
 - Dsp_lal_api_func_sub_vec, [42](#)
- Rsdk_LA_sum2_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_sum_vec, [91](#)
- Rsdk_LA_sum2_Sof32p8Vif32p8
 - Dsp_lal_api_func_sum_vec, [92](#)
- Rsdk_LA_sum3_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_sum_vec, [93](#)
- Rsdk_LA_sum3_Sof32p8Vif32p8
 - Dsp_lal_api_func_sum_vec, [93](#)
- Rsdk_LA_sum_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_sum_vec, [94](#)
- Rsdk_LA_sum_Sof32p8Vif32p8
 - Dsp_lal_api_func_sum_vec, [94](#)
- Rsdk_LA_sumRcp_Sof32cp8Vif32cp8
 - Dsp_lal_api_func_sum_vec, [95](#)
- Rsdk_LA_sumRcp_Sof32p8Vif32p8
 - Dsp_lal_api_func_sum_vec, [95](#)
- Rsdk_LA_thresh_Vof32p8Vif32p8Sif32
 - Dsp_lal_api_func_op_vec_mat, [59](#)
- Rsdk_LA_thresh_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [59](#)
- Rsdk_LA_threshAbs_Vof32p8Vif32cp8Sif32
 - Dsp_lal_api_func_op_vec_mat, [60](#)
- Rsdk_LA_threshAbs_Vof32p8Vif32cp8Vif32p8
 - Dsp_lal_api_func_op_vec_mat, [61](#)
- Rsdk_LA_wavg_Sof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_mean_vec, [117](#)
- Rsdk_LA_wavg_Sof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_mean_vec, [117](#)
- Rsdk_LA_xcorr_Vof32cp8Vif32cp8Vif32cp8
 - Dsp_lal_api_func_xcorr_vec, [52](#)
- Rsdk_LA_xcorr_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_xcorr_vec, [53](#)
- RSDK_MAG2N_2XF32_V1
 - DSP LAL Constants, [35](#)
- RSDK_MAG2N_2XF32_V2
 - DSP LAL Constants, [36](#)
- Rsdk_ML_spatialSm_Mof32cp8Mif32cp8
 - Dsp_lal_api_func_sp_smooth, [125](#)
- Rsdk_SP_fillStgRadix2_Vof32cp8
 - Dsp_lal_api_func_fill, [69](#)
- Rsdk_SP_SNR_Vof32p8Vif32p8Vif32p8
 - Dsp_lal_api_func_snr, [141](#)
- RsdkBbe32CaCfar
 - DSP Radar Algo Functions, [12](#)
- rsdkBbe32CaCfarParams_t, [150](#)
 - dopplerBins, [150](#)
 - guardInterval, [150](#)
 - pDetectBitmap, [151](#)
 - pRdm, [151](#)
 - pThr, [151](#)
 - rangeBins, [151](#)
 - thresholdFactor, [151](#)
 - windowSize, [151](#)
- rsdkBbe32CmprDecTxpsParams_t, [152](#)
 - mantBwFix, [152](#)
 - noChirps, [152](#)
 - noRanges, [152](#)
 - numExpBits, [152](#)
 - numFillBits, [152](#)
 - pCcfg, [153](#)
 - pDeCmprOut, [153](#)
 - pInComprDat, [153](#)
 - pMantBwEncTbl, [153](#)
 - pMantBwFlx, [153](#)
- rsdkBbe32CmprEncTxpsParams_t, [153](#)
 - mantBwFix, [154](#)
 - noChirps, [154](#)
 - noRanges, [154](#)
 - numExpBits, [154](#)
 - numFillBits, [154](#)
 - pCcfg, [154](#)
 - pCmprOut, [155](#)
 - pInDat, [155](#)
 - pMantBwEncTbl, [155](#)
 - pMantBwFlx, [155](#)
- rsdkBbe32CmprFixOneBufParams_t, [155](#)
 - mantBwFix, [156](#)
 - noChirps, [156](#)
 - noRanges, [156](#)

- numExpBits, 156
- numFillBits, 156
- pCmprOut, 157
- pInDat, 157
- pMantBwEncTbl, 157
- rsdkBbe32CmprFixTwoBuf4Params_t, 157
 - mantBwFix, 157
 - noChirps, 158
 - noRanges, 158
 - pCmprExpOut, 158
 - pCmprOut, 158
 - pInDat, 158
- rsdkBbe32CmprFixTwoBuf8Params_t, 158
 - mantBwFix, 159
 - noChirps, 159
 - noRanges, 159
 - pCmprExpOut, 159
 - pCmprOut, 159
 - pInDat, 159
 - pMantBwEncTbl, 160
- rsdkBbe32CmprFlxOneBufParams_t, 160
 - noChirps, 160
 - noRanges, 160
 - numExpBits, 160
 - numFillBits, 161
 - pCmprOut, 161
 - pInDat, 161
 - pMantBwEncTbl, 161
 - pMantBwFlx, 161
- rsdkBbe32CmprFlxTwoBuf4Params_t, 161
 - noChirps, 162
 - noRanges, 162
 - pCmprExpOut, 162
 - pCmprOut, 162
 - pInDat, 162
 - pMantBwFlx, 162
- rsdkBbe32CmprFlxTwoBuf8Params_t, 163
 - noChirps, 163
 - noRanges, 163
 - pCmprExpOut, 163
 - pCmprOut, 163
 - pInDat, 164
 - pMantBwEncTbl, 164
 - pMantBwFlx, 164
- rsdkBbe32CmprSplnitParams_t, 164
 - comprType, 164
 - nRange, 165
 - numBytesPerChirp, 165
 - pMantBwFlx, 165
- RsdkBbe32CmprSptInit
 - DSP Radar Algo Functions, 13
- rsdkBbe32CmprSptParams_t, 165
 - noChirps, 165
 - noRanges, 166
 - numBytesPerChirp, 166
 - pCmprOut, 166
 - pInDat, 166
 - pMantBwFlx, 166
- RsdkBbe32DecTxps
 - DSP Radar Algo Functions, 13
- RsdkBbe32Doa128
 - DSP Radar Algo Functions, 14
- RsdkBbe32Doa256
 - DSP Radar Algo Functions, 14
- rsdkBbe32DoaParams_t, 166
 - noAntennas, 167
 - noPeaks, 167
 - pDetectBitmap, 167
 - pIn, 167
 - pOutMag, 168
 - pScratch1Area, 168
 - pScratch2Area, 168
 - pTwiddlesArea, 168
 - pWindowArea, 168
 - shift, 169
 - thresholdFactor, 169
- RsdkBbe32EncCp4d
 - DSP Radar Algo Functions, 15
- RsdkBbe32EncCp4dFmta
 - DSP Radar Algo Functions, 16
- RsdkBbe32EncCp4dFmtb
 - DSP Radar Algo Functions, 16
- RsdkBbe32EncFlxSpt34
 - DSP Radar Algo Functions, 16
- RsdkBbe32EncSeqFixOnebuf
 - DSP Radar Algo Functions, 17
- RsdkBbe32EncSeqFixTwobuf4
 - DSP Radar Algo Functions, 17
- RsdkBbe32EncSeqFixTwobuf8
 - DSP Radar Algo Functions, 18
- RsdkBbe32EncSeqFlxOnebuf
 - DSP Radar Algo Functions, 18
- RsdkBbe32EncSeqFlxTwobuf4
 - DSP Radar Algo Functions, 19
- RsdkBbe32EncSeqFlxTwobuf8
 - DSP Radar Algo Functions, 19
- RsdkBbe32EncTxps
 - DSP Radar Algo Functions, 20
- RsdkBbe32Fft2Real
 - DSP Radar Algo Functions, 20
- rsdkBbe32Fft2RealParams_t, 169
 - pIn1, 170
 - pIn2, 170
 - pOut1, 170
 - pOut2, 170
 - pScratch1Area, 170
 - pScratch2Area, 170
 - pScratch3Area, 171
 - pTwiddlesArea, 171
 - pWindowArea, 171
 - shift, 171
 - size, 171
- RsdkBbe32FftCplx
 - DSP Radar Algo Functions, 21
- rsdkBbe32FftCplxParams_t, 172
 - pInOut, 172

- pScratch1Area, [172](#)
 - pScratch2Area, [173](#)
 - pTwiddlesArea, [173](#)
 - pWindowArea, [173](#)
 - shift, [173](#)
 - size, [173](#)
- RsdkBbe32FftInit
 - DSP Radar Algo Functions, [21](#)
- rsdkBbe32FftInitParams_t, [174](#)
 - pTwiddlesArea, [174](#)
 - pWindowArea, [174](#)
 - size, [175](#)
- RsdkBbe32Hist
 - DSP Radar Algo Functions, [22](#)
- rsdkBbe32HistParams_t, [175](#)
 - pEdges, [175](#)
 - pIn, [175](#)
 - pOut, [176](#)
 - size, [176](#)
 - sizeEdges, [176](#)
- RsdkBbe32LalVectorAdd
 - Dsp_lal_demos_api_func_add, [142](#)
- rsdkBbe32LalVectorAddParams_t, [176](#)
 - pInput1, [176](#)
 - pInput2, [176](#)
 - pOutput, [177](#)
 - size, [177](#)
- RsdkBbe32OsCfar
 - DSP Radar Algo Functions, [22](#)
- rsdkBbe32OsCfarParams_t, [177](#)
 - dopplerBins, [177](#)
- RsdkBbe32ReordTile16to4Ch
 - DSP Radar Algo Functions, [23](#)
- rsdkBbe32ReordTile16to4ChParams_t, [178](#)
 - doubleInBufEn, [178](#)
 - noElements, [178](#)
 - pInDat, [178](#)
 - pOutDat, [178](#)
- RsdkBbe32Tdlm
 - DSP Radar Algo Functions, [23](#)
- rsdkBbe32TdlmParams_t, [179](#)
 - pIn, [179](#)
 - pInDataAbs, [179](#)
 - pOut, [179](#)
 - pTotalSum, [179](#)
 - samples, [180](#)
 - scalingFactor, [180](#)
 - thrPercent, [180](#)
- RsdkDspDispatcherInit
 - DSP Dispatcher Functions, [10](#)
- RsdkDspDispatcherRun
 - DSP Dispatcher Functions, [10](#)
- rsdkDspFuncPtr_t
 - DSP Dispatcher Data Types, [9](#)
- rsdkDsplsCb_t
 - DSP Dispatcher Data Types, [9](#)
- samples
 - rsdkBbe32TdlmParams_t, [180](#)
- scalingFactor
 - rsdkBbe32TdlmParams_t, [180](#)
- shift
 - rsdkBbe32DoaParams_t, [169](#)
 - rsdkBbe32Fft2RealParams_t, [171](#)
 - rsdkBbe32FftCplxParams_t, [173](#)
- size
 - rsdkBbe32Fft2RealParams_t, [171](#)
 - rsdkBbe32FftCplxParams_t, [173](#)
 - rsdkBbe32FftInitParams_t, [175](#)
 - rsdkBbe32HistParams_t, [176](#)
 - rsdkBbe32LalVectorAddParams_t, [177](#)
- SIZE_VEC
 - DSP LAL Demo Constants, [37](#)
- sizeEdges
 - rsdkBbe32HistParams_t, [176](#)
- SP_pi
 - DSP LAL Constants, [36](#)
- SP_pi_over_2
 - DSP LAL Constants, [36](#)
- sptEvtIntEnBitmap
 - dspDisplnit_t, [145](#)
- sptEvtIsrCb
 - dspDisplnit_t, [145](#)
- thresholdFactor
 - rsdkBbe32CaCfarParams_t, [151](#)
 - rsdkBbe32DoaParams_t, [169](#)
- thrPercent
 - rsdkBbe32TdlmParams_t, [180](#)
- windowSize
 - rsdkBbe32CaCfarParams_t, [151](#)

How to Reach Us:**Home Page:**nxp.com**Web Support:**nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Converge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and μ Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2024 NXP B.V.

